第9章 查找

提纲

CONTENTS

9.1 查找的基本概念

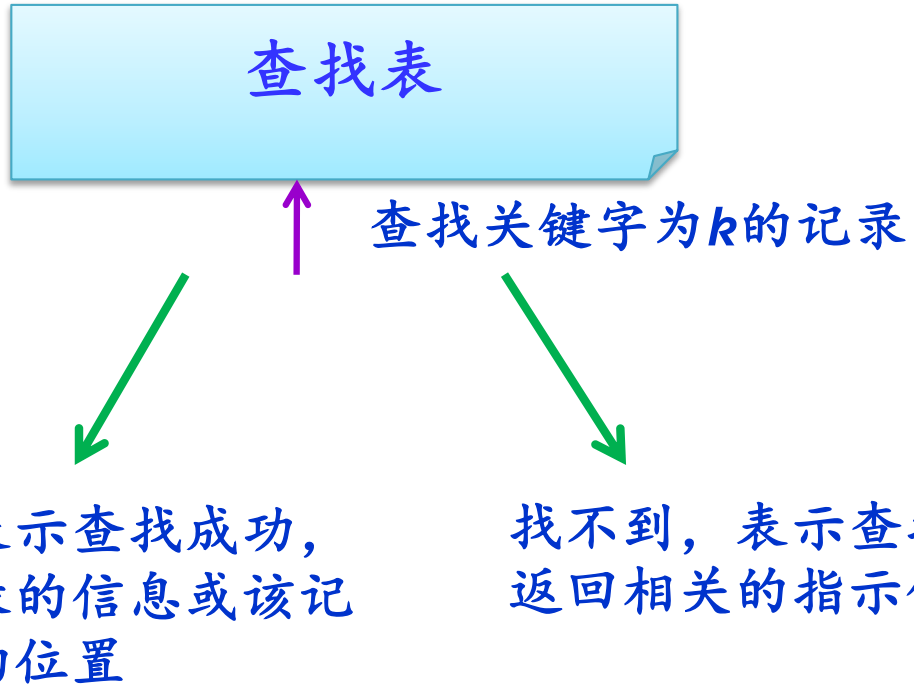
9.2 线性表的查找

9.3 树表的查找

9.4 哈希表查找

9.1 查找的基本概念

- 一般情况下，被查找的对象称为**查找表**，查找表包含一组元素（或记录），每个元素由若干个数据项组成，并假设有能唯一标识元素的数据项，称为主关键字（默认按主关键字查找）。



查找表按照操作方式分为静态查找表和动态查找表两类。

- **静态查找表**是只作查找操作的查找表，主要操作有查询某个“特定的”数据元素是否在查找表中，检索某个“特定的”数据元素及其属性。
- **动态查找表**是在查找过程中同时插入查找表中不存在的数据元素，或者从查找表中删除已经存在的某个数据元素。

查找有内查找和外查找之分。

- 若整个查找过程都在内存进行，则称之为**内查找**。
- 反之，若查找过程中需要访问外存，则称之为**外查找**。

查找性能评价

- 查找算法中的主要操作是关键字之间的比较，所以通常把查找过程中关键字平均比较次数也就是平均查找长度作为衡量一个查找算法效率优劣的依据。
- 平均查找长度ASL（Average Search Length）定义为

$$\mathrm{ASL}=\sum_{i=1}^{n} p_i c_i$$

其中，n是查找表中元素的个数，p_i是查找第i个元素的概率，一般地，除特别指出外，均认为每个元素的查找概率相等，即$p_i=1/n$（$1\leqslant i\leqslant n$），c_i是查找到第i个元素所需的关键字比较次数。

由于查找的结果有查找成功和不成功两种情况，所以平均查找长度也分为成功情况下的平均查找长度和不成功情况下的平均查找长度。

- 成功情况下的平均查找长度指在查找表中找到指定关键字*k*的元素平均所需关键字比较的次数。
- 不成功情况下的平均查找长度指在查找表中确定找不到关键字*k*的元素平均所需关键字比较的次数。

查找表T：含有*n*个元素。

成功情况下（概率相等）的平均查找长度$ASL_{成功}$是指找到T中任一记录平均需要的关键字比较次数。

例如：

关键字	5	1	4	8	7	9	2	4	3
找到时的比较次数	1	2	3	4	5	6	7	8	9

$$ASL_{成功}=\frac{1+2+3+4+5+6+7+8+9}{9}=5$$

查找表T：含有*n*个元素。

不成功情况下的平均查找长度$ASL_{不成功}$是指查找失败（在T中未查找到）平均需要的关键字比较次数。

$\forall x \notin T$

通过关键字比较后确定不在T中

平均关键字比较次数

第9章 查找

提纲

CONTENTS

9.1 查找的基本概念

9.2 线性表的查找

9.3 树表的查找

9.4 哈希表查找

9.2 线性表的查找

- 线性表采用顺序表存储，由于顺序表不适合数据修改操作（插入和删除元素几乎需要移动一半的元素）⇨ 顺序表是一种静态查找表。
- 三种线性表查找方法，即顺序查找、折半查找和分块查找算法。

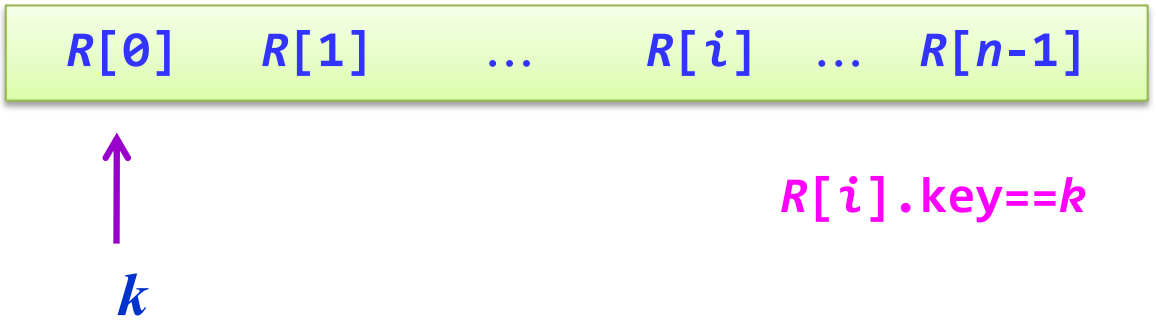
顺序表组织：

- 若待查找的顺序表仅由若干整数构成，直接采用vector<int>向量表示。
- 如10个整数关键字序列表示为：*R*={1,6,2,5,3,7,9,8,10,4}（不存在相同的关键字）。
- 若待排序表中每个元素除整数关键字外还有其他数据项，可以采用向量vector<T>表示，T为相应的元素类型。
- 如3个学生元素，每个元素由学号（每个学生的学号是唯一的）和姓名组成，*R*表示为：*R*={[1,"Mary"],[3,"John"],[2,"Smith"]}。

```
struct T
{   int no;                 //学号:关键字
    string name;            //姓名
};
```

9.2.1 顺序查找

1. 顺序查找算法



基本思路

- 从顺序表的一端开始依次遍历，将遍历的元素关键字和给定值**k**相比较，
- 若两者相等，则查找成功，返回该元素的序号。
- 若遍历结束后，仍未找到关键字等于**k**的元素，则查找失败，返回**-1**。
- 默认从顺序表的前端开始遍历。

关键字比较

```
int SeqSearch1(vector<int>& R,int k) //顺序查找算法1
{  int n=R.size();
   int i=0;
   while (i<n && R[i]!=k)
      i++;                                  //从表头往后找
   if (i>=n) return -1;                     //未找到返回-1
   else return i;                           //找到后返回其序号i
}
```

简单比较方法

设置一个哨兵　　关键字比较

```
int SeqSearch2(vector<int>& R,int k) //顺序查找算法2
{  int n=R.size();
   R.push_back(k);                   //末尾添加一个哨兵
   int i=0;
   while (R[i]!=k) i++;              //从表头往后找
   if (i==n) return -1;              //未找到返回-1
   else return i;                    //找到后返回其序号i
}
```

由于查找算法主要考虑关键字之间的比较，两个算法的效率差不多

2．顺序查找算法分析

1）仅考虑查找成功的情况

- 第1个元素即R[0]=k，1次关键字比较。
- 第2个元素即R[1]=k，2次关键字比较。
- 以此类推，第n个元素即R[n-1]=k，n次关键字比较

$c_i=i+1$

共有n种查找成功的情况，在等概率时，$p_i=1/n$。

$$\mathrm{ASL}_{成功}=\sum_{i=0}^{n-1}p_i\,c_i=\frac{1}{n}\sum_{i=0}^{n-1}(i+1)=\frac{1}{n}\times\frac{n(n+1)}{2}=\frac{n+1}{2}$$

查找成功时的平均比较次数约为表长的一半 。

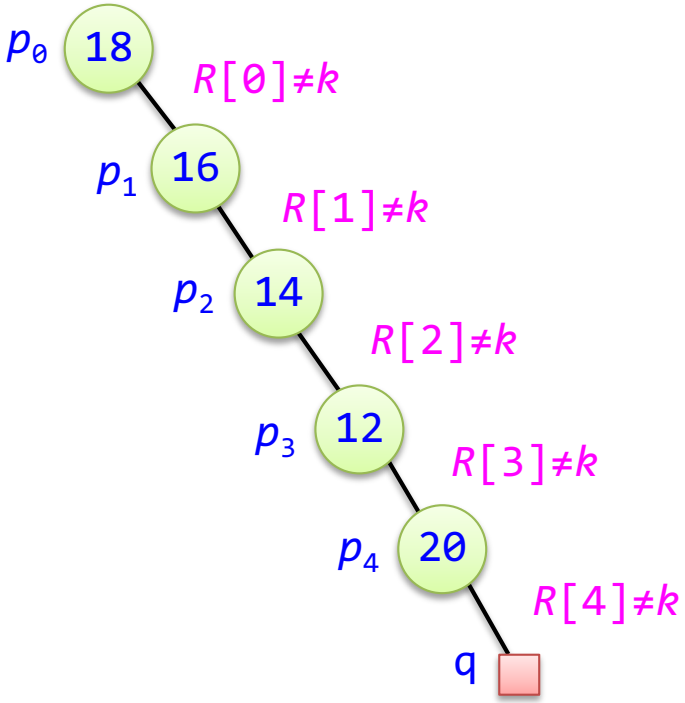
2）仅考虑查找不成功的情况

若*k*值不在表中，则总是需要*n*次比较之后才能确定查找失败，所以仅仅考虑查找不成功时对应的平均查找长度为：

$$ASL_{不成功} = n$$

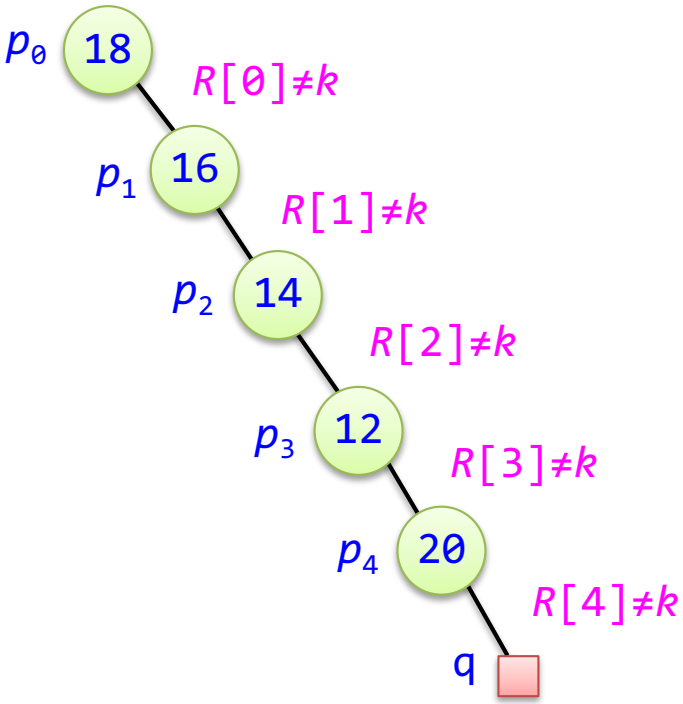
3）既考虑查找成功又考虑查找不成功的情况

- 一个顺序表中顺序查找的全部情况（查找成功和失败）可以用一棵判定树或比较树（这里是单支二叉树）来描述。
- 例如，*n*=5的关键字序列（18，16，14，12，20）的顺序查找过程对应的判定树如下：



- 设所有成功查找的概率为$p=\sum p_i$，q表示不成功查找的概率，当既考虑查找成功又考虑查找不成功的情况时有$p+q=1$。
- 不妨假设$p=q=0.5$，并且所有关键字成功查找的概率相同，即$p_i=0.5/n$，则成功情况下的平均查找长度为：

$$\mathrm{ASL}_{成功}=\sum_{i=0}^{n-1}p_i c_i=\sum_{i=0}^{n-1}\left(\frac{0.5}{n}\times(i+1)\right)=0.5\times\frac{n+1}{2}$$



假设所有不成功查找的情况为m种，它们的查找概率相同，即$q_i=0.5/m$，则不成功情况下的平均查找长度为：

$$\mathrm{ASL}_{\text{不成功}}=\sum_{i=1}^{m}q_i\times n=\sum_{i=1}^{m}(\frac{0.5}{m}\times n)=0.5n$$

合起来：

$$\mathrm{ASL}=\mathrm{ASL}_{\text{成功}}+\mathrm{ASL}_{\text{不成功}}=0.5\times\frac{n+1}{2}+0.5\times n=\frac{3n+1}{4}$$

可以推出：顺序查找的时间复杂度为$O(n)$。

顺序查找总结

- 顺序查找的优点是算法简单，且对查找表的存储结构无特殊要求，无论是用顺序表还是用链表来存放元素，也无论是元素之间是否按关键字有序，它都同样适用。
- 顺序查找的缺点是查找效率低，因此，当*n*较大时不宜采用顺序查找。

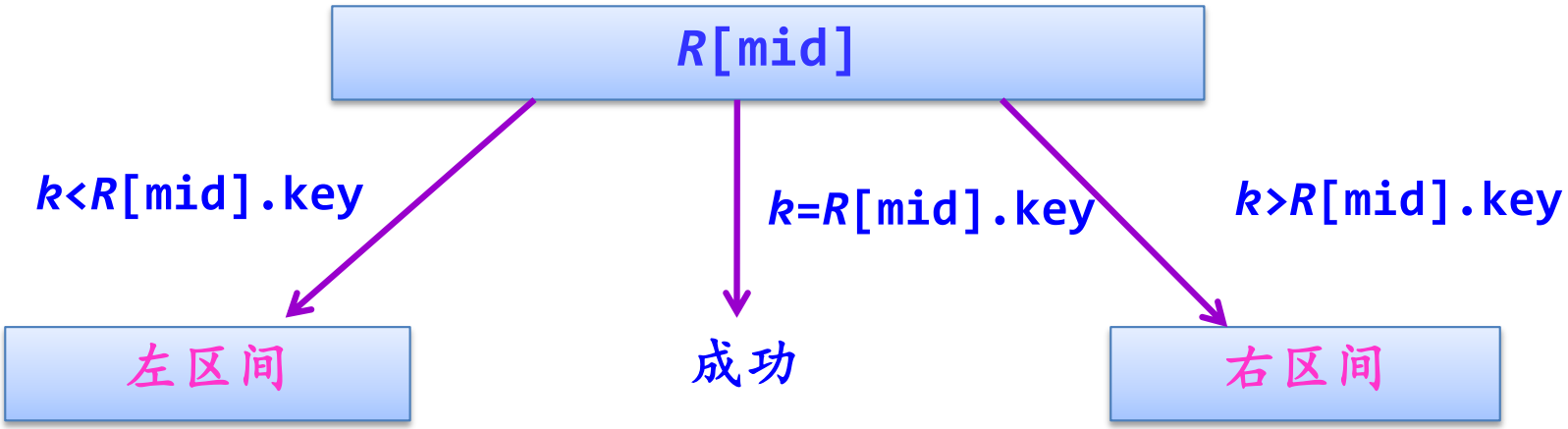
9.2.2 折半查找

1.折半查找算法

- 折半查找也称为二分查找：

基本思路 查找表R[0..n-1]为递增有序顺序表

- R[low..high]是当前的非空查找区间（下界为low，上界为high），中点位置mid=⌊(low+high)/2⌋（或者mid=(low+high)>>1），k值与R[mid]比较：



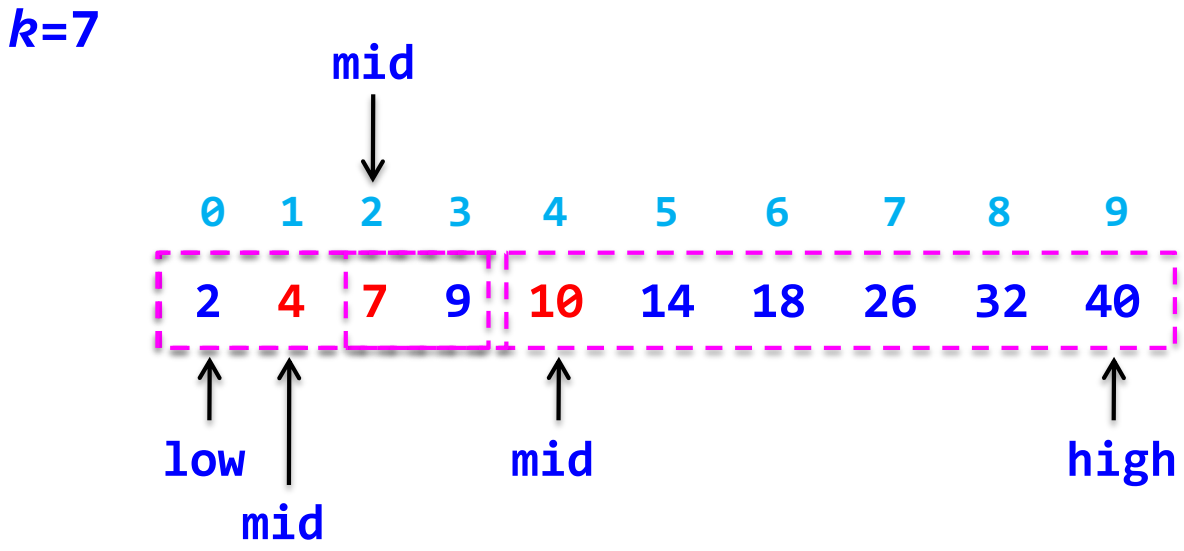
9.2.2 折半查找

1.折半查找算法

基本思路 查找表R[0..n-1]为递增有序顺序表

- R[low..high]是当前的非空查找区间（下界为low，上界为high），中点位置mid=⌊(low+high)/2⌋（或者mid=(low+high)>>1），k值与R[mid]比较：
  - ① 若k=R[mid]，则查找成功并返回该元素的序号mid。
  - ② 若k<R[mid]，则在左子表R[low..mid-1]中查找，即下界不变，上界改为mid-1。
  - ③ 若k>R[mid]，则在右子表R[mid+1..high]中查找，即下界改为mid+1，上界不变。
- 下一次查找是针对非空新查找区间进行的，其过程与上述过程类似。若新查找区间为空，表示查找失败，返回-1。

【例9.2】在关键字有序序列（2，4，7，9，10，14，18，26，32，40）中采用折半查找方法查找关键字为7的元素。



R[mid]=k ⇨ 查找成功

```
int BinSearch1(vector<int>& R,int k) //拆半查找非递归算法
{  int n=R.size();
   int low=0,high=n-1;
   while (low<=high)                  //当前区间非空时
   {  int mid=(low+high)/2;           //求查找区间的中间位置
      if (k==R[mid])                  //查找成功返回其序号mid
         return mid;
      if (k<R[mid])                   //继续在R[low..mid-1]中查找
         high=mid-1;
      else                            //k>R[mid]
         low=mid+1;                   //继续在R[mid+1..high]中查找
   }
   return -1;                         //当前查找区间空时返回-1
}
```

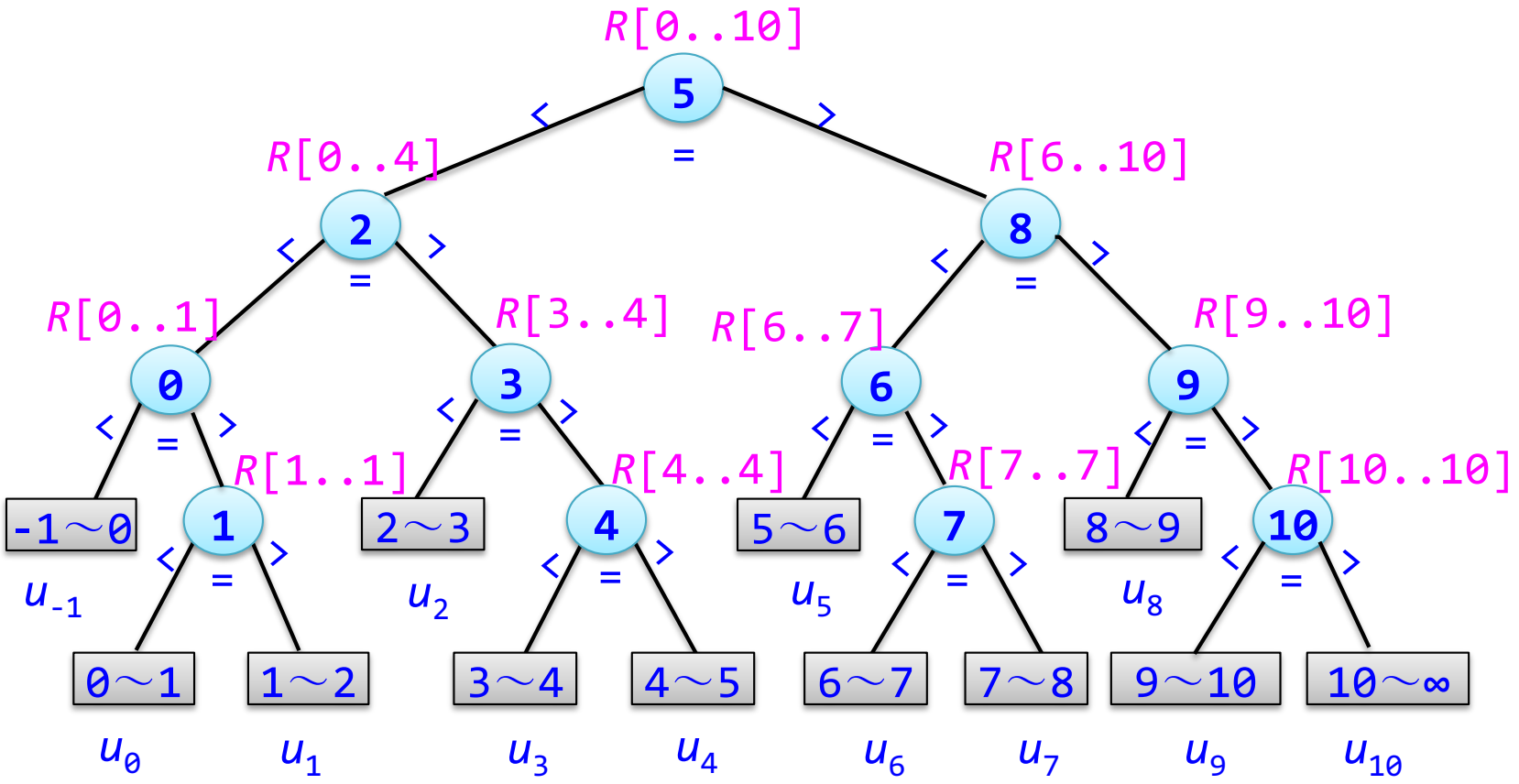
```
int BinSearch2(vector<int>& R,int k)          //拆半查找递归算法
{
   return BinSearch21(R,0,R.size()-1,k);
}

int BinSearch21(vector<int>& R,int low,int high,int k)
//被BinSearch2方法调用
{  if (low<=high)                             //当前查找区间非空时
   {  int mid=(low+high)/2;                   //求查找区间的中间位置
      if (k==R[mid])                          //查找成功返回其序号mid
         return mid;
      if (k<R[mid])                           //递归在左区间中查找
         return BinSearch21(R,low,mid-1,k);
      else                                    //k>R[mid],递归在右区间中查找
         return BinSearch21(R,mid+1,high,k);
   }
   else return -1;                            //当前查找区间空时返回-1
}
```

2．折半查找算法分析

- 一个有序顺序表R中所有元素的折半查找过程可用一棵判定树或比较树（这里是二叉树）来描述。
- 查找区间为R[low..high]的判定树T(low，high)定义为，当low>high时，T(low，high)为空树；当low≤high时，根结点为中间序号mid=(low+high)/2的元素，其左子树是R[low..mid-1]对应的判定树T(low，mid-1)，其右子树是R[mid+1，high]对应的判定树T(mid+1，high)。

具有11个元素（R[0..10]）的有序表可用下图的判定树来表示



- 内部结点中的数字表示该元素在有序表中的下标。
- 外部结点中的两个值表示查找不成功时关键字对应的元素序号范围

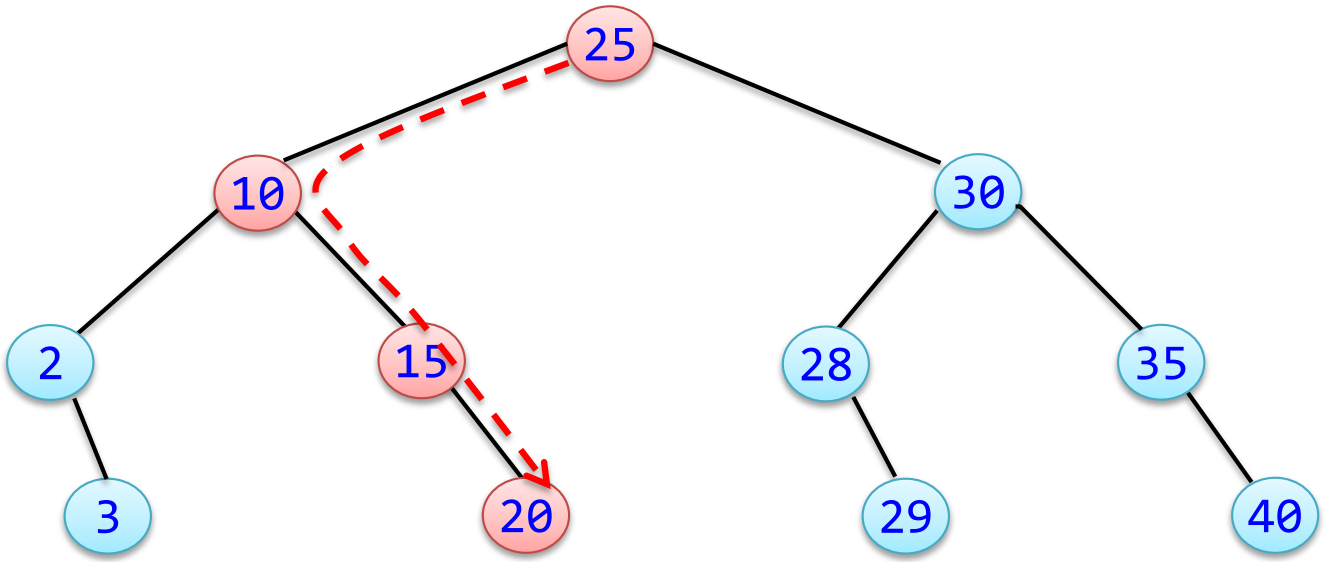
【例9.4】给定11个元素的有序表（2，3，10，15，20，25，28，29，30，35，40），采用折半查找，试问：

（1）若查找给定值为20的元素，将依次与表中哪些元素比较？

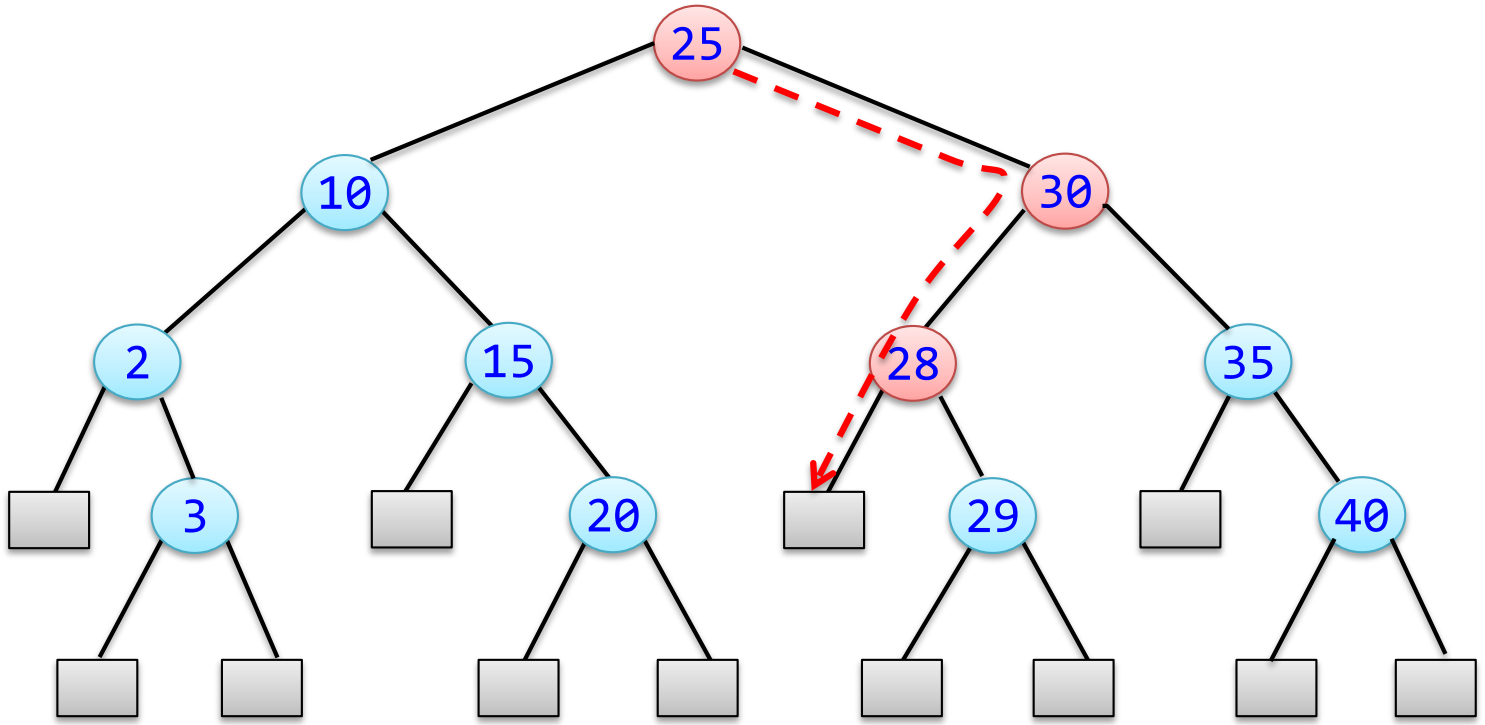
（2）若查找给定值为26的元素，将依次与哪些元素比较？

（3）假设查找表中每个元素的概率相同，求查找成功时的平均查找长度和查找不成功时的平均查找长度。

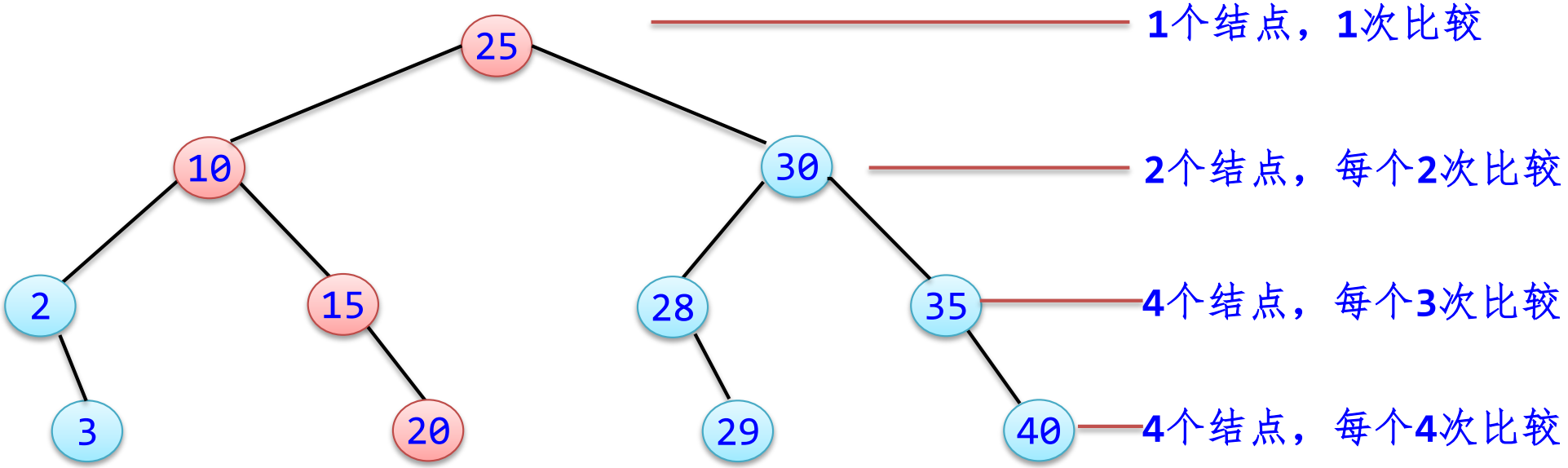
（1）有序表为（2，3，10，15，20，25，28，29，30，35，40），查找给定值为20的元素，将依次与表中哪些元素比较？



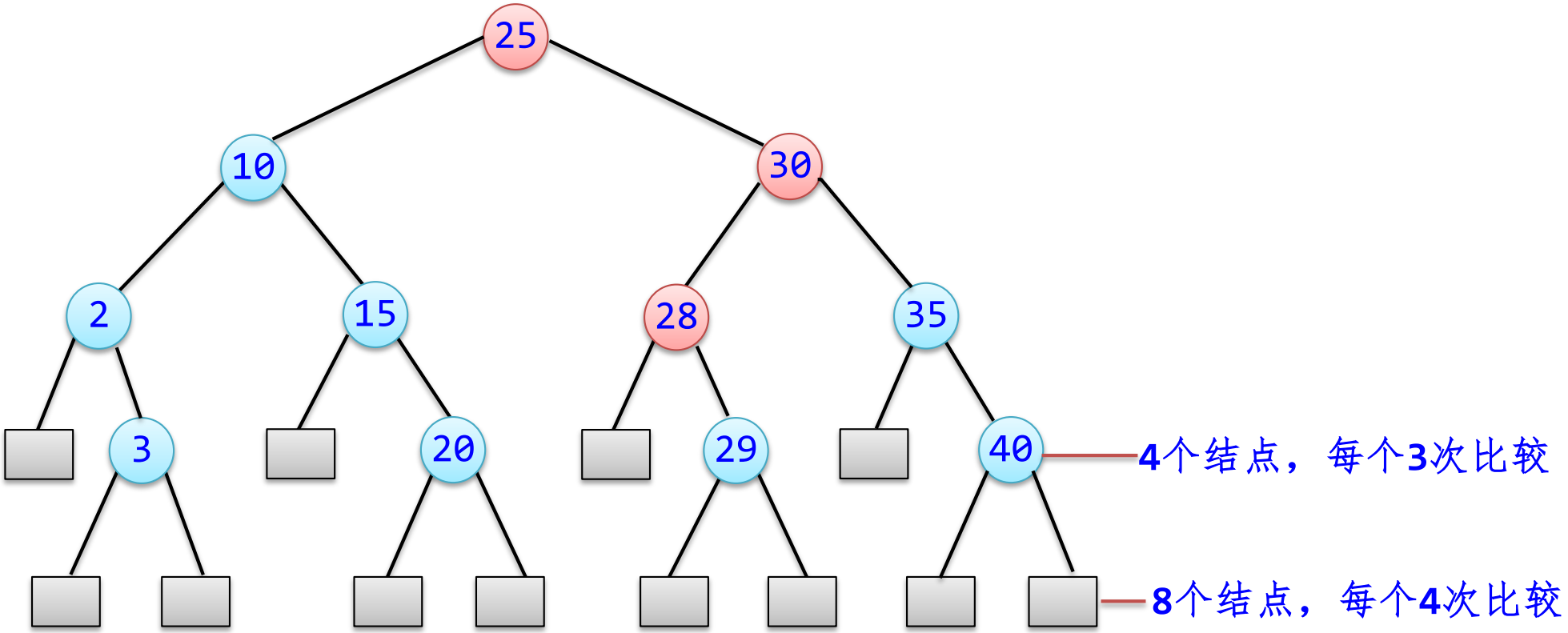
（2）有序表为（2，3，10，15，20，25，28，29，30，35，40），查找给定值为26的元素，将依次与哪些元素比较？



（3）假设查找表中每个元素的概率相同，求查找成功时的平均查找长度和查找不成功时的平均查找长度。



$$ASL_{成功} = \frac{1\times1+2\times2+4\times3+4\times4}{11} = 3$$



$$ASL_{不成功} = \frac{4 \times 3 + 8 \times 4}{12} = 3.67$$

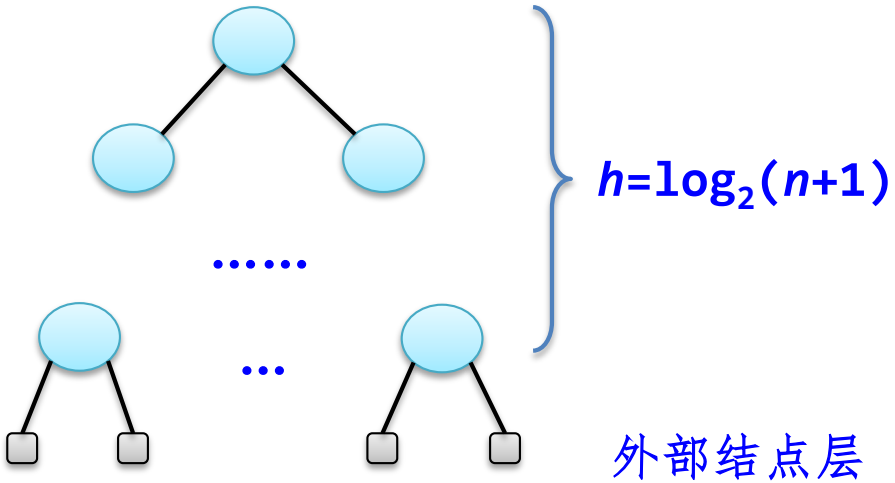
不妨设关键字序列为（k_0，k_1，…，k_{n-1}），并有$k_0<k_1<\cdots<k_{n-1}$，对应n个内部结点，查找关键字k_i的概率为p_i，则成功情况下的平均查找长度为：

$$\mathrm{ASL}_{\text{成功}} = \sum_{i=0}^{n-1} p_i \times level(k_i)$$

不在判定树中的关键字可分为$n+1$类E_i（$-1 \leqslant i \leqslant n-1$），对应$n+1$个外部结点，设$q_i$是查找属于$E_i$中关键字的概率，那么不成功的平均查找长度为：

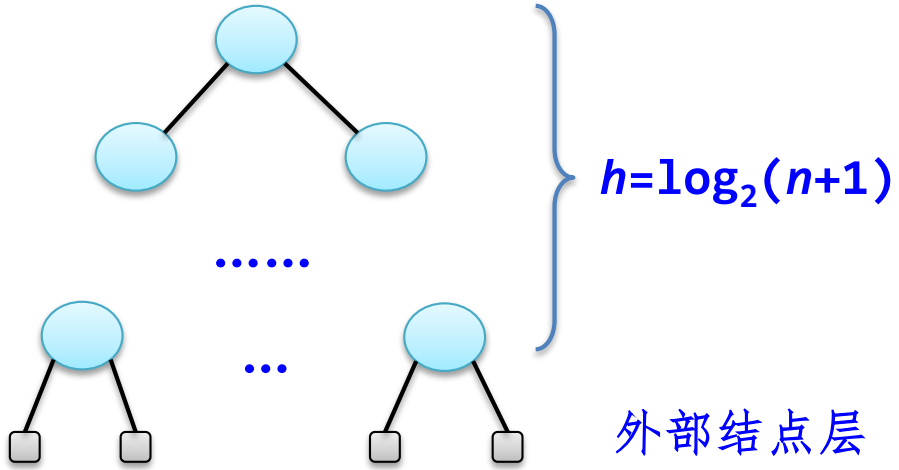
$$\mathrm{ASL}_{\text{不成功}} = \sum_{i=-1}^{n-1} q_i \times (level(u_i) - 1)$$

- 借助一棵二叉判定树很容易求得折半查找的平均查找长度。
- 为讨论方便起见，不妨设内部结点的总数为$n=2^h-1$，这样的判定树是高度为$h=\log_2(n+1)$的满二叉树（高度h不计外部结点）。该满二叉树中第j（$1\leqslant j\leqslant h$）层上的结点个数为2^{j-1}，查找该层上的每个结点需要进行j次比较。



因此，在等概率假设下，折半查找成功情况下的平均查找长度为：

$$\mathrm{ASL}_{\text{成功}}=\sum_{i=0}^{n-1}p_i c_i=\frac{1}{n}\sum_{j=1}^{h}2^{j-1}\times j=\frac{n+1}{n}\log_2(n+1)\approx\log_2(n+1)-1$$



层次最大的外部结点就是不成功查找所需关键字比较次数最多的结点，它一定是层次最大的内部结点的孩子结点，其关键字比较次数恰好是 $h+1-1=h$。

从成功和不成功情况下的平均查找长度看出，折半查找的时间复杂度为$O(\log_2 n)$，是一种高效的查找算法。

结论

- 当n不等于2^h-1时，其折半查找判定树不一定为满二叉树，但可以证明n个结点的判定树的高度与n个结点的完全二叉树的高度相等，即h为$\lceil \log_2(n+1) \rceil$或者$\lfloor \log_2 n \rfloor+1$。
- 查找成功时关键字比较次数最多为判定树的高度h。
- 查找不成功时关键字比较次数最多也为判定树的高度h。

3．STL中的折半查找算法

对于以数组为低层结构的有序表（如数组、vector或者deque容器等），STL中提供了一系列以折半查找为基础的快速查找通用算法：

- binary_search(beg，end，*x*，[comp])在[beg，end)范围内查找*x*，如果找到则返回true，否则返回false。其中comp是与排序一致的比较函数，省略时使用底层类型的小于运算符。
- lower_bound(beg，end，*x*，[comp])在[beg，end)范围内查找第一个大于等于*x*的元素地址。其中comp是与排序一致的比较函数，省略时使用底层类型的小于运算符。
- upper_bound(beg，end，*x*，[comp])在[beg，end)范围内查找第一个大于*x*的元素地址，即插入点位置。其中comp是与排序一致的比较函数，省略时使用底层类型的小于运算符。
- equal_range(beg，end，*x*，[comp])返回一对地址，第一个即first为lower_bound的结果，第二个second为upper_bound的结果。其中comp是与排序一致的比较函数，省略时使用底层类型的小于运算符。

```
#include<iostream>
#include<vector>
#include<deque>
#include<algorithm>
using namespace std;
int main()
{  int a[]={1,2,2,2,3};
   int n=sizeof(a)/sizeof(a[0]);
   bool flag=binary_search(a,a+n,2);
   printf(“%d\n”,flag);                    //输出1⇨数组a中存在元素2
   int first=lower_bound(a,a+n,2)-a;       //通过-a得到查找元素的序号
   printf("%d\n",first);                   //输出1 ⇨a[1]是第一个>=2的元素
   int last=upper_bound(a,a+n,2)-a;
   printf("%d\n",last);                    //输出4 ⇨a[4]是第一个>2的元素
   pair<int* ,int* > ia=equal_range(a,a+n,2);
   printf("%d %d\n",ia.first-a,ia.second-a);  //输出1和4
```

```
    vector<int> v={1,2,2,2,3};
    flag=binary_search(v.begin(),v.end(),2);
    printf("%d\n",flag);                    //输出1,向量v中存在元素2
    first=lower_bound(v.begin(),v.end(),2)-v.begin();
    printf("%d\n",first);                   //输出1,v[1]是第一个>=2的元素
    last=upper_bound(v.begin(),v.end(),2)-v.begin();
    printf("%d\n",last);                    //输出4,v[4]是第一个>2的元素
    pair<vector<int>::iterator,vector<int>::iterator> its
                =equal_range(v.begin(),v.end(),2);
    printf("%d %d\n",its.first-v.begin(),its.second-v.begin());
                                            //输出1和4
    return 0;
}
```

4*．折半查找算法的变形算法设计

- 前面基本折半查找是在关键字有序且不重复的查找表中查找关键字为*k*的一个元素，每次比较产生3个分支。
- 当关键字*k*重复时基本折半查找一定能够找到一个关键字为*k*的元素，但不能确定是哪一个关键字为*k*的元素。需要利用折半查找算法的变形来实现，像STL中lower_bound通用算法查找第一个大于等于*x*的元素，upper_bound通用算法查找第一个大于*x*的元素，它们都是折半查找的变形算法。

设计折半查找的变形算法需要注意如下几点：

① 确定边界。while循环的条件是low<=high（查找区间不空时循环）还是low<high（查找区间有2个或者以上元素时循环）。

② 通常折半查找变形算法每次比较产生两个分支，每个分支对应的low或者high的修改是什么？如果边界是low<=high，mid=(low+high)/2，若某个分支修改low是low=mid，当low=high时，下一步的查找区间没有变化会导致死循环，必须避免出现这样的情况。

③ 在循环结束时，分析此时目标元素的形态，确定算法返回位置的正确表示形式。

折半查找变形算法的通用设计方法

- 用谓词$p(x)$表示解空间中结点x满足的条件，该谓词是一个bool函数，不同的问题中该谓词也不同。
- 在解空间（这里的解空间就是判定树）按谓词的真假来选择一个分支。

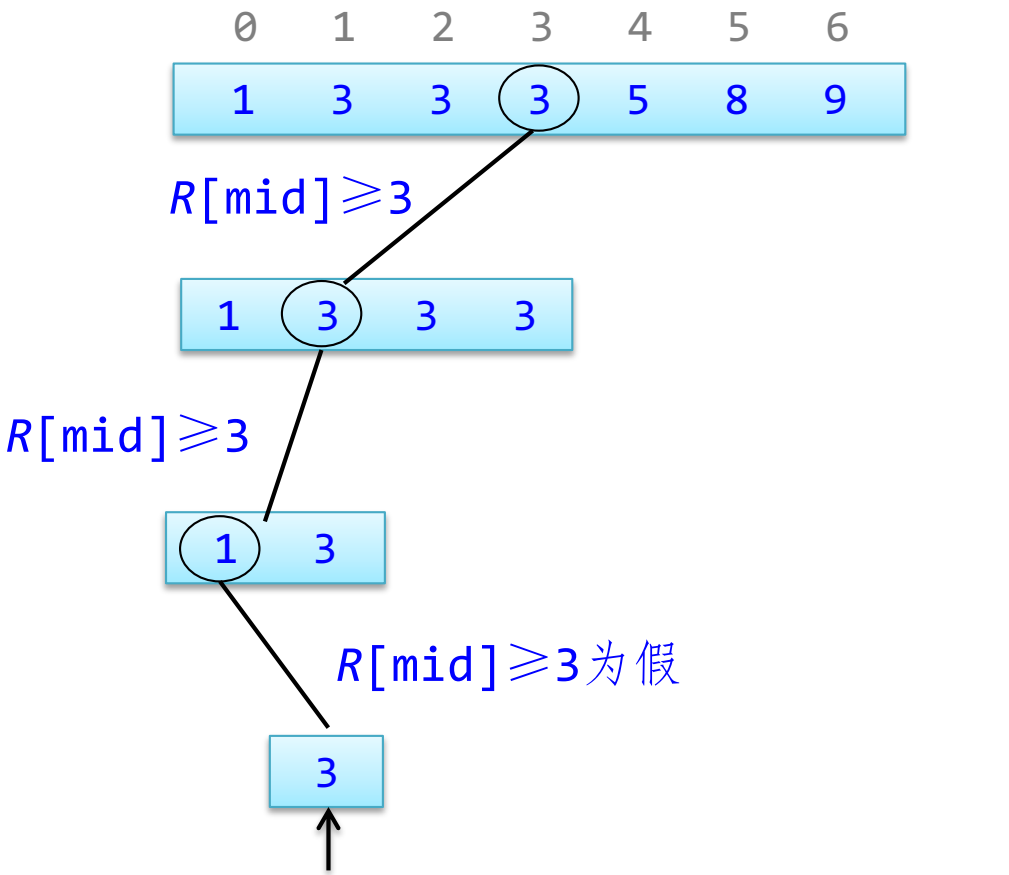
1）lower_bound查找算法设计

- 该算法是在有序表*R*中查找第一个大于等于*k*的元素（*R*中可能没有关键字*k*的元素，也有可能有多个），简单地说就是查找*k*的插入点，关键字*k*的插入点定义为将*k*插入*R*中使其有序的第一个位置。
- 默认*R*是递增有序的。
- 例如，*R*=（1，3，3，3，5，8），*k*=0的插入点为0，*k*=3的插入点为1，*k*=5的插入点为4，*k*=10的插入点为6。

- 对于R[0..n-1]，插入点可能是0～n-1，还可能是n（当k大于R中所有元素时），所以初始查找区间是[0，n]而不是[0，n-1]。
- 设置p(x)为“x>=k”，即查找关键字大于等于k的元素。
- 如何保证找到p(x)为真的第一个元素呢？若查找区间为[low，high]，置mid=(low+high)/2。
- 当p(R[mid])为真时，需要在左区间[low，mid]（含mid）中继续查找，即修改high=mid。当p(R[mid])为假时与基本折半查找一样在右区间中查找。
- 查找区间至少含2个元素（因为low=high时出现死循环），所以确定边界是low<high。
- 这样在最后查找区间为[low，low]时，low就是插入点。

查找第一个大于等于3的过程

$p(x)$是$R[mid]\geqslant 3$



第一个$p(x)$为假的仅含一个元素的区间

```
int* lower_bound1(vector<int>& R,int n,int k)
{  int low=0,high=n;
   while (low<high)
   {  int mid=(low+high)/2;
      if (R[mid]>=k)                    //p(x)="x>=k",谓词为true
         high=mid;                      //在左区间中查找
      else                              //谓词为false
         low=mid+1;                     //在右区间中查找
   }
   return& R[low];                      //返回R[low]元素地址
}
```

STL中对应的简化版算法

```
int* lower_bound2(vector<int>& R,int n,int k)        //STL版本
{  int low=0,mid;
   int half,len;
   len=n;
   while (len>0)
   {  half=len/2;
      mid=low+half;
      if (R[mid]>=k)                 //p(x)="x>=k", 谓词为true
         len=half;                   //左区间(以R[low]开始的len个元素
                                     //含R[mid])中查找,low不变
      else                           //谓词为false
      {  low=mid+1;                  //修改low
         len=len-half-1;             //在右区间中查找
      }
   }
   return& R[low];                   //返回R[low]元素地址
}
```

也可以这样设计：

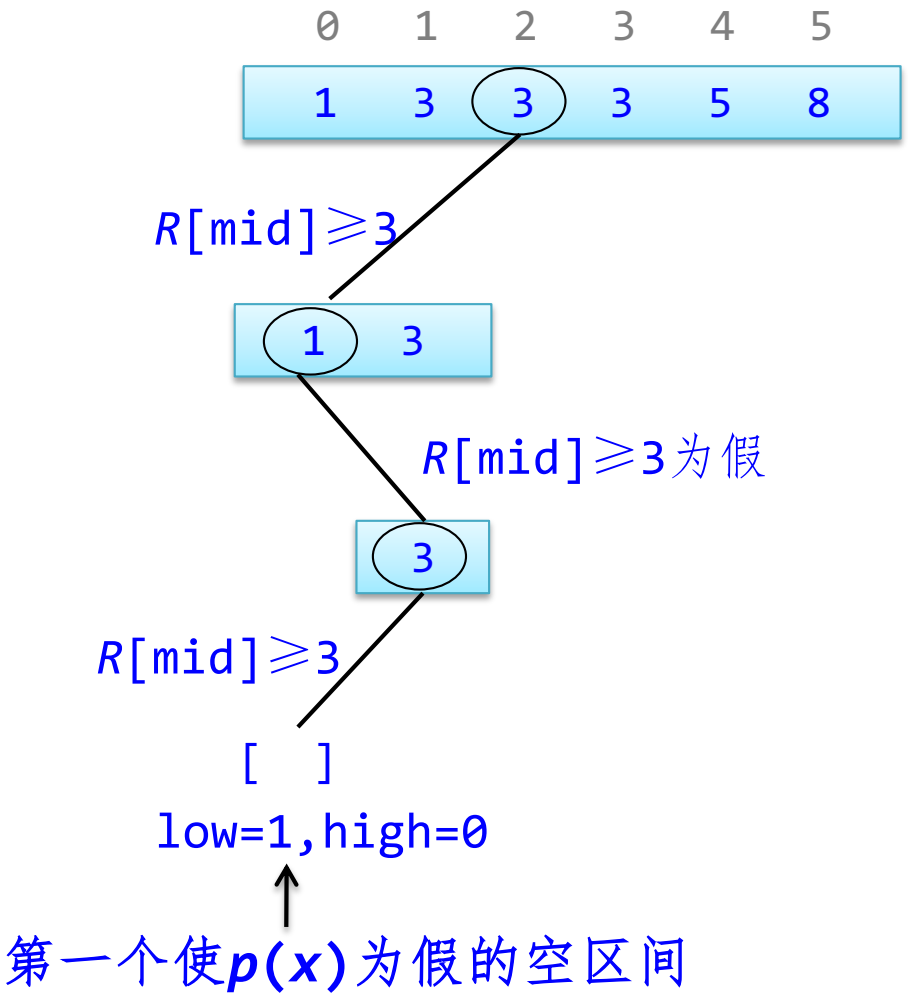
① 确定边界为low<=high（循环执行到空为止）。

② 确定两个分支的变化，*p*(*x*)为true修改high=mid-1（不含mid，左分支的查找区间可能不包含*p*(*x*)为真的整数），*p*(*x*)为false修改low=mid+1（右分支的查找区间可能包含*p*(*x*)为真的整数）。

③ 在循环结束时，查找区间[low，high]为空（一定不包含*p*(*x*)为真的整数）而右区间[low..*n*-1]一定包含*p*(*x*)为真的整数，其首位置low即为所求，实际上此时有low=high+1，返回high+1亦可。

查找第一个大于等于3的过程

$p(x)$是$R[mid]\geqslant 3$



```
int* lower_bound3(vector<int>& R,int n,int k)
{  int low=0,high=n-1;
   while (low<=high)              //当前区间至少有一个元素时
   {  int mid=(low+high)/2;       //求查找区间的中间位置
      if (R[mid]>=k)              //p(x)为x>=k,谓词为true
         high=mid-1;              //在R[low..mid-1]中查找,low不变
      else                        //谓词为false
         low=mid+1;               //在R[mid+1..high]中查找
   }
   return& R[low];                //返回R[low]或者R[high+1]元素地址
}
```

2）upper_bound查找算法设计

upper_bound算法是查找第一个大于*x*的元素，设计思路与lower_bound相同，只是将谓词*p*(*x*)改为“x>k”。

9.2.3 索引存储结构和分块查找

1．索引存储结构

- 索引存储结构是在采用数据表存储数据的同时，还建立附加的索引表。
- 索引表中的每一项称为索引项，索引项的一般形式为（关键字，地址），其中，关键字唯一标识一个元素，地址为该关键字元素在数据表中的存储地址，整个索引表按关键字有序排列。

索引存储结构 = 数据表 + 索引表

数据表

地址	学号	姓名	分数
0	2018001	王华	90
1	2018010	刘丽	62
2	2018006	陈明	54
3	2018009	张强	95
4	2018007	许兵	76
5	2018012	李萍	88
6	2018005	李英	82

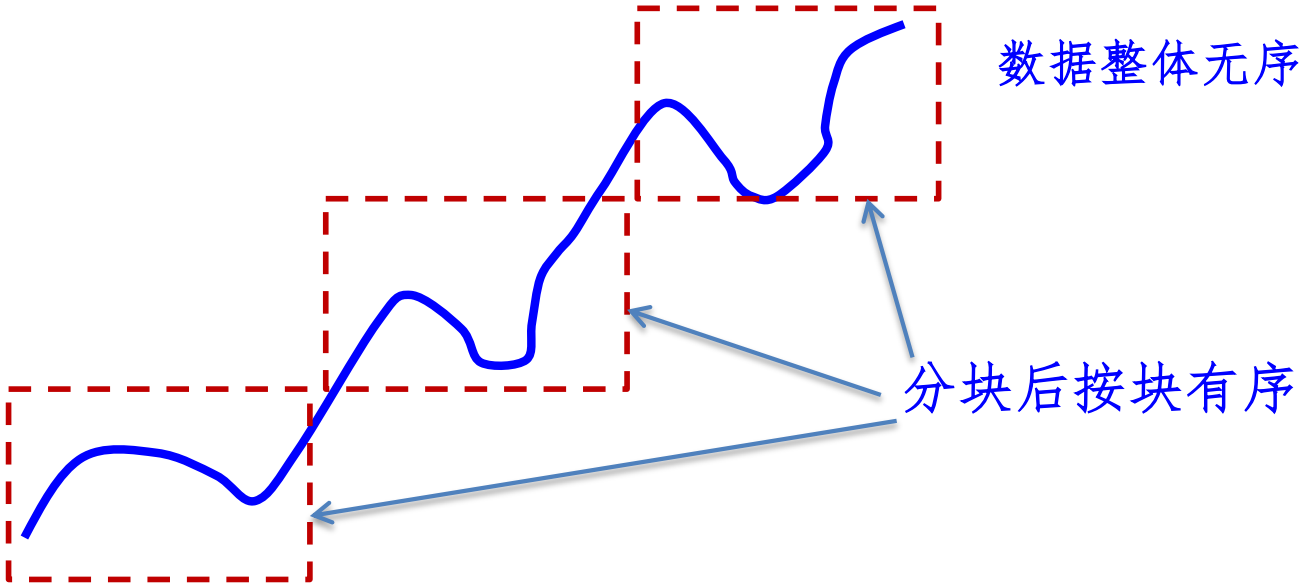
索引表

学号	地址
2018001	0
2018005	6
2018006	2
2018007	4
2018009	3
2018010	1
2018012	5

按关键字*k*的查找过程：

- 先在索引表按折半查找方法找到关键字为*k*的索引项，得到其地址，所花时间为$O(\log_2 n)$。
- 再通过地址在数据表中找到对应的元素，所花时间为$O(1)$，合起来的查找时间为$O(\log_2 n)$。

2．分块查找

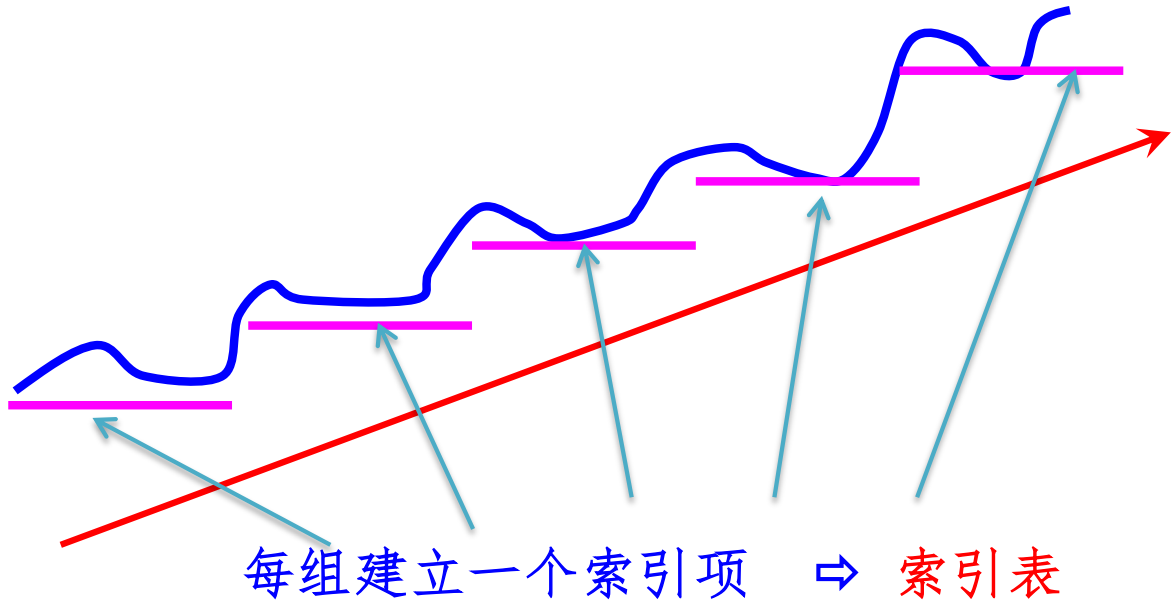


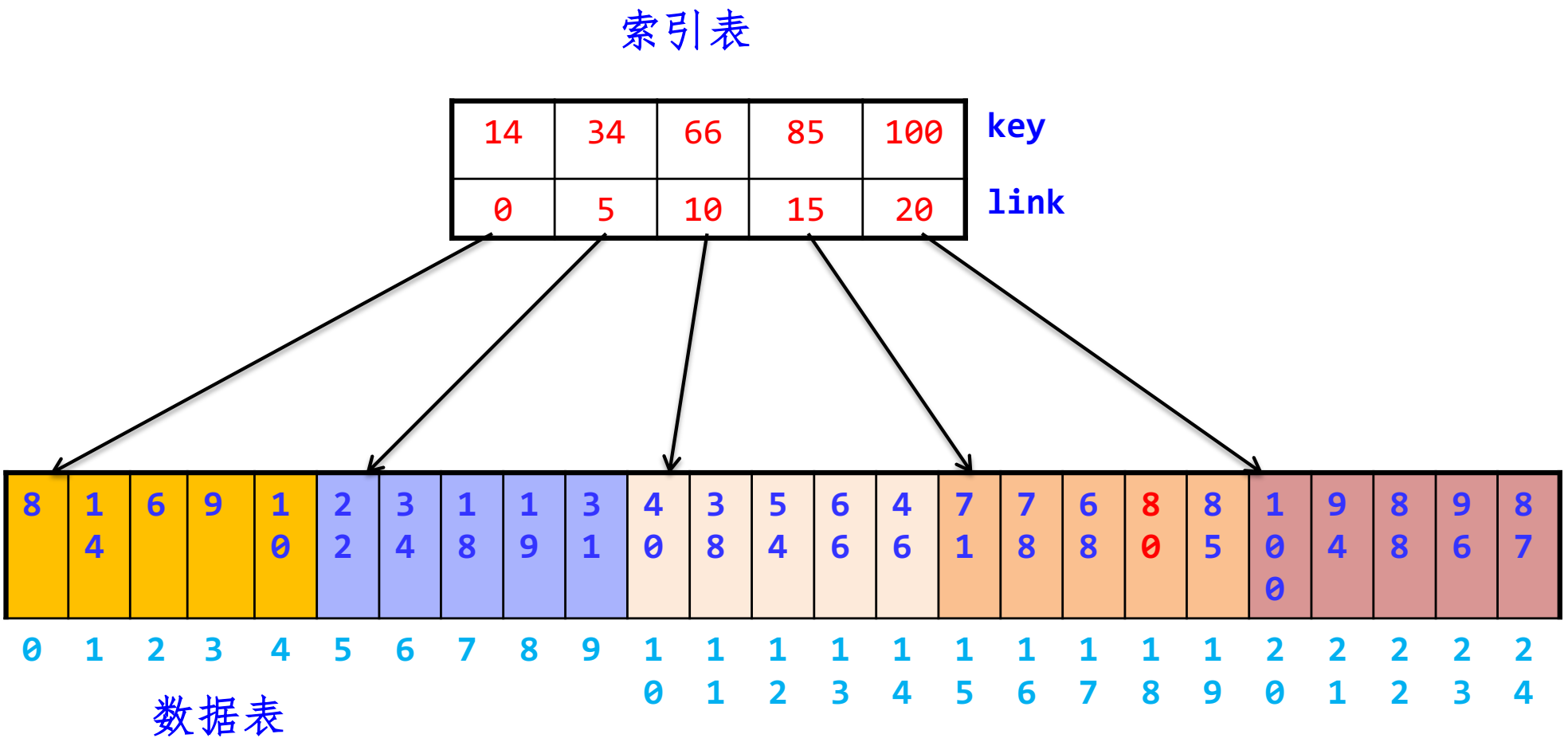
例如，设有一个线性表，其中包含25个元素，其关键字序列为：

8，14，6，9，10，22，34，18，19，31，40，38，54，66，46，71，78，68，80，85，100，94，88，96，87

分块：将n=25个记录分为b=5块，每块中有s=5个元素。

数据特性：



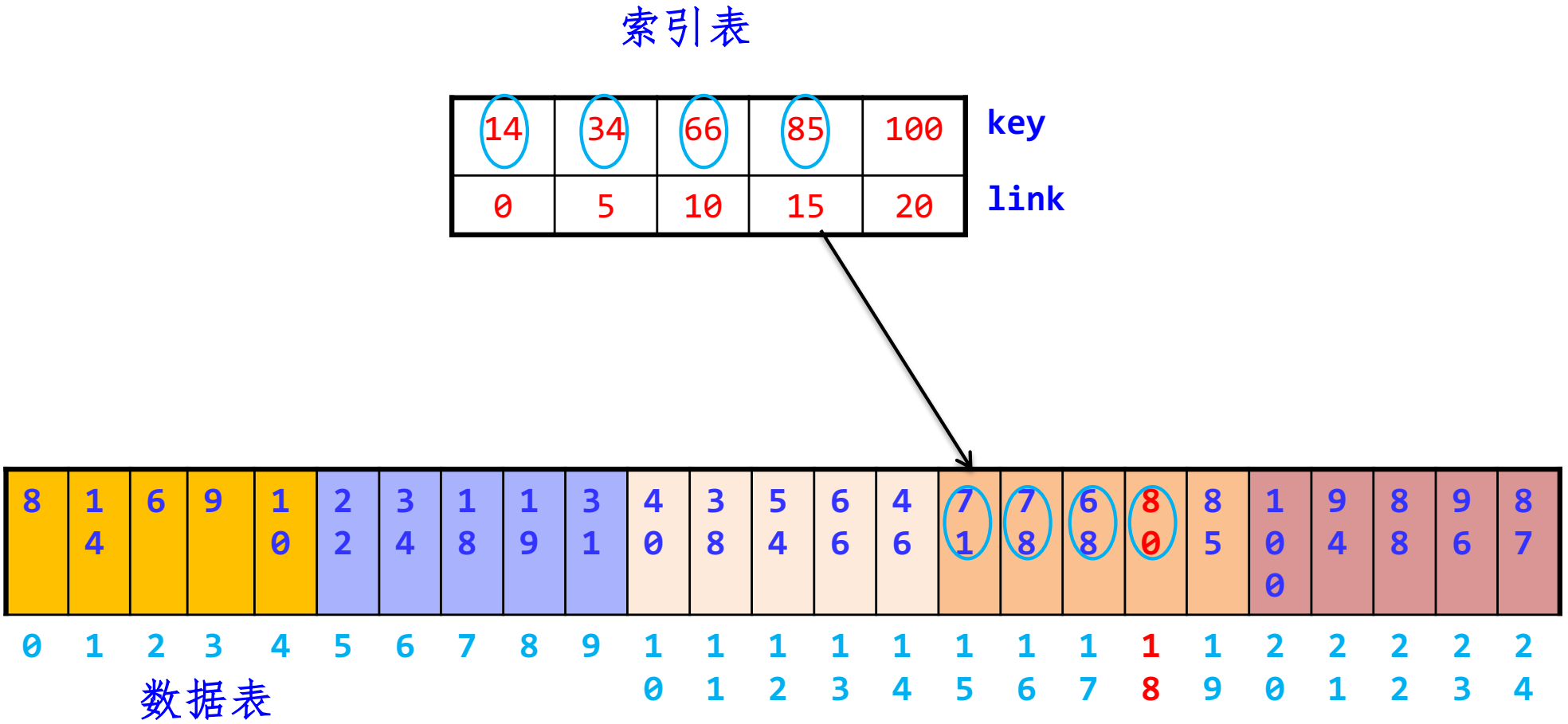


分块查找的索引存储结构

分块查找过程：

- 查找索引表（有序）：可以顺序查找块，也可以二分查找块。
- 查找数据块（无序）：只能顺序查找块中元素。

查找关键字为k=80的元素



（1）顺序查找索引表，比较4次

（2）在对应块中查找，比较4次，共比较8次。

索引表的元素类型定义如下：

```
struct IdxType          //索引表类型
{   int key;            //关键字（这里是对应块中的最大关键字）
    int link;           //该索引块在数据表中的起始下标
};
```

14	34	66	85	100
0	5	10	15	20

假设数据表长度为*n*，分为*b*个块，块长度为*s*。创建索引表I[0..*b*-1]

```
void CreateI(vector<int>& R,IdxType I[],int b) //构造索引表I[0..b-1]
{  int n=R.size();
   int s=(n+b-1)/b;                    //每块的元素个数
   int j=0;
   int jmax=R[j];
   for (int i=0;i<b;i++)               //构造b个块
   {  I[i].link=j;
      while (j<=(i+1)*s-1 && j<=n-1)//j遍历一个块,找其中最大关键字jmax
      {  if (R[j]>jmax) jmax=R[j];
         j++;
      }
      I[i].key=jmax;
      if (j<=n-1)                  //遍历完,jmax置为下一个块首元素关键字
         jmax=R[j];
   }
}
```

分块查找算法

```
int BlkSearch(vector<int>& R,IdxType I[],int b,int k)
//在R[0..n-1]和索引表I[0..b-1]中查找k
{  int n=R.size();
   int low=0,high=b-1;
   while (low<=high)              //在索引表中折半查找,找到块号为high+1
   {  int mid=(low+high)/2;
      if (k<=I[mid].key) high=mid-1;
      else low=mid+1;
   }
   if (high+1>=b) return -1; //块号超界,查找失败,返回-1
   int i=I[high+1].link;      //求所在块的起始位置
```

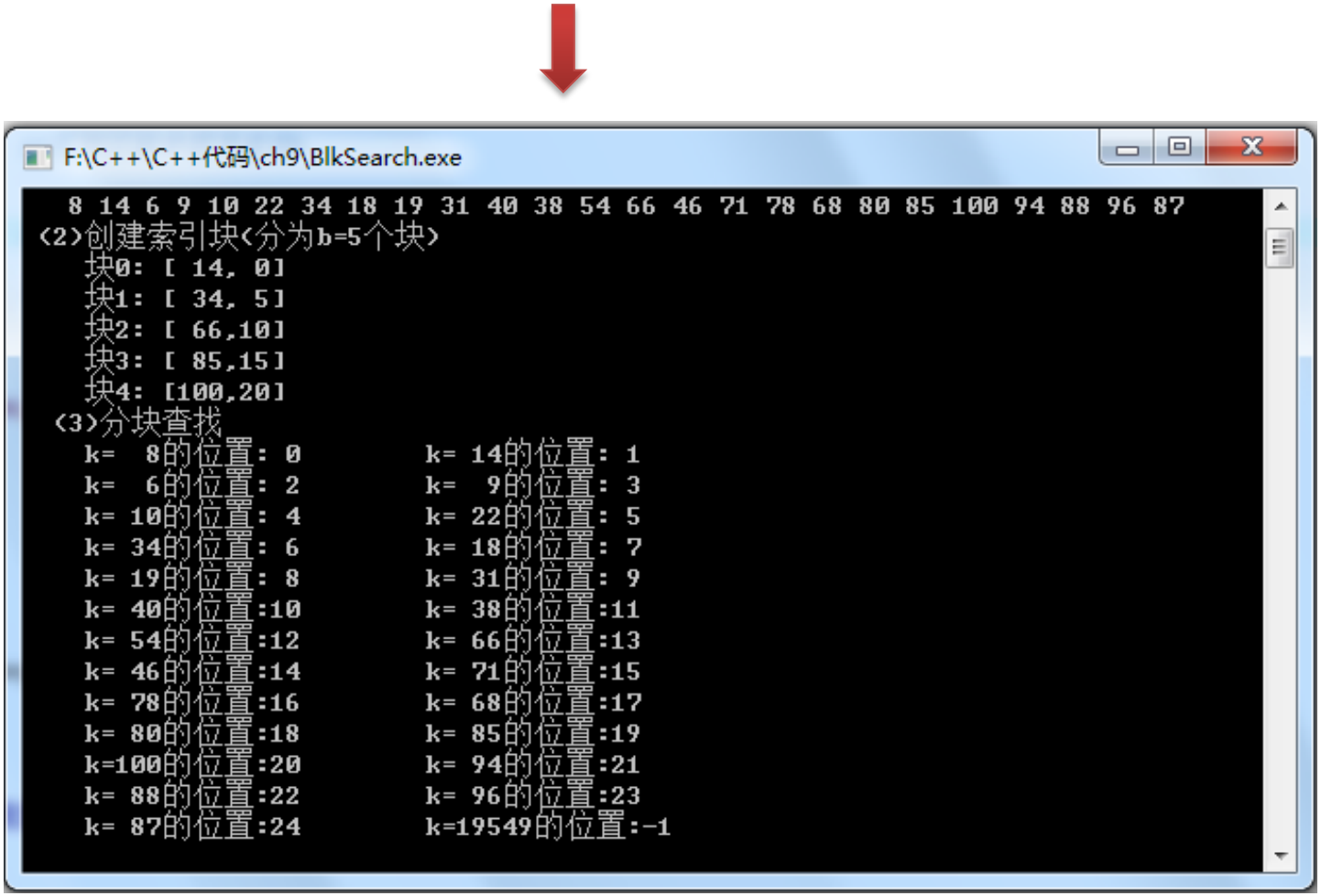
```
    int s=(n+b-1)/b;                          //求每块的元素个数s
    if (i==b-1)                               //第i块是最后块时
       s=n-s*(b-1);

    while (i<=I[high+1].link+s-1 && R[i]!=k)
       i++;                                   //在对应块中顺序查找k

    if (i<=I[high+1].link+s-1)
       return i;                              //查找成功,返回该元素的序号
    else
       return -1;                             //查找失败,返回-1
}
```

程序验证

```
int main()
{  vector<int> R={8,14,6,9,10,22,34,18,19,31,40,38,54,66,46,71,
       78,68,80,85,100,94,88,96,87};
   int b=5;
   IdxType *I=new IdxType[b];
   CreateI(R,I,b);
   printf("\n (1)初始数据\n   ");
   for (int i=0;i<R.size();i++)
      printf("%d ",R[i]);
   printf("\n (2)创建索引块(分为b=5个块)\n");
   for (int i=0;i<b;i++)
      printf("    块%d: [%3d,%2d]\n",i,I[i].key,I[i].link);
   printf("  (3)分块查找\n");
   for (int i=0;i<R.size();i+=2)
   {  int k1=R[i],k2=R[i+1];
      printf("    k=%3d的位置:%2d\t  k=%3d的位置:%2d\n",
             k1,BlkSearch(R,I,b,k1),k2,BlkSearch(R,I,b,k2));
   }
   return 0;
}
```


F:\C++\C++代码\ch9\BlkSearch.exe
8 14 6 9 10 22 34 18 19 31 40 38 54 66 46 71 78 68 80 85 100 94 88 96 87
(2)创建索引块(分为b=5个块)
块0: [14, 0]
块1: [34, 5]
块2: [66,10]
块3: [85,15]
块4: [100,20]
(3)分块查找
k= 8的位置: 0
k= 14的位置: 1
k= 6的位置: 2
k= 9的位置: 3
k= 10的位置: 4
k= 22的位置: 5
k= 34的位置: 6
k= 18的位置: 7
k= 19的位置: 8
k= 31的位置: 9
k= 40的位置:10
k= 38的位置:11
k= 54的位置:12
k= 66的位置:13
k= 46的位置:14
k= 71的位置:15
k= 78的位置:16
k= 68的位置:17
k= 80的位置:18
k= 85的位置:19
k=100的位置:20
k= 94的位置:21
k= 88的位置:22
k= 96的位置:23
k= 87的位置:24
k=19549的位置:-1

【例】 设数据序列中有100个元素，待查找元素的关键字k=47。如果在查找过程中，和k进行比较的元素依次是10，47，32，16，47，则所采用的查找方法可能是（ ）。

A.顺序查找　　B.折半查找　　C.分块查找　　D.都不是

- 只有分块查找需要进行两次查找。
- 第一次与47的比较是在索引表中查找，第2次与47的比较是在对应块中查找。
- C

分块查找性能分析

若有n个元素，每块中有s个元素（块数$b=\lceil n/s \rceil$）

- 用折半查找确定元素所在的块，则分块查找成功时的平均查找长度为：

$$ASL_{blk} = ASL_{bn} + ASL_{sq} = \log_2(b+1) - 1 + \frac{s+1}{2} \approx \log_2(n/s+1) + \frac{s}{2}$$

当s越小时，ASL_{blk}的值越小，即当采用折半查找确定块时，每块的长度越小越好。

若有*n*个元素，每块中有*s*个元素（块数*b*=⌈*n*/*s*⌉）

- 用顺序查找确定元素所在的块，则分块查找成功时的平均查找长度为：

$$ASL'_{blk} = ASL_{bn} + ASL_{sq} = \frac{b+1}{2} + \frac{s+1}{2} = \frac{1}{2}\left(\frac{n}{s} + s\right) + 1$$

当s=sqrt(*n*)时，ASL'$_{blk}$取极小值sqrt(n)+1，即当采用顺序查找确定块时，各块中的元素个数s选定为sqrt(*n*)时效果最佳。

第9章 查找

提纲

CONTENTS

9.1 查找的基本概念

9.2 线性表的查找

9.3 树表的查找

9.4 哈希表查找

9.3 树表的查找

- 几种特殊树形结构—统称为树表。
- 这里的树表采用链式存储结构，由于链式存储结构既适合查找，也适合数据修改，属于动态查找表。
- 对于动态查找表，不仅要讨论查找方法，还讨论修改方法。

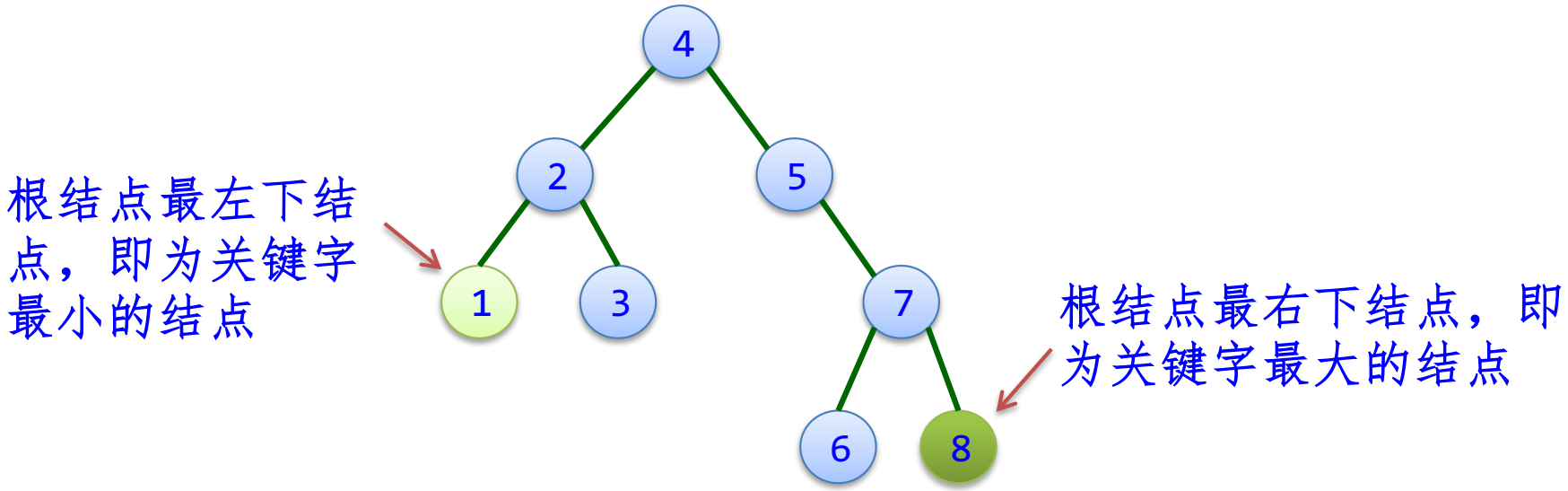
9.3.1 二叉排序树

1．二叉排序树的定义

二叉排序树（简称BST）又称二叉查找（搜索）树，其定义为：二叉排序树或者是空树，或者是满足如下性质的二叉树：

- 若它的左子树非空，则左子树上所有结点值（默认为结点关键字）均小于根结点值。
- 若它的右子树非空，则右子树上所有结点值均大于根结点值。
- 左、右子树本身又各是一棵二叉排序树。

一棵二叉排序树示例：



特点：

中序序列：1，2，3，4，5，6，7，8

中序序列是一个递增有序序列！

定义二叉排序树的结点类型如下：

```
template <typename T1,typename T2>
struct BSTNode                          //二叉排序树结点类
{   T1 key;                             //存放关键字,假设关键字为T1类型
    T2 data;                            //存放数据项,假设数据项为T2类型
    BSTNode* lchild;                    //存放左孩子指针
    BSTNode* rchild;                    //存放右孩子指针

    BSTNode(T1 k,T2 d)                  //构造函数
    {   key=k;
        data=d;
        lchild=rchild=NULL;             //新建结点默认为叶子结点
    }
};
```

二叉排序树的每个结点含**key**和**data**

设计二叉排序树类模板BSTClass<T1,T2>:

```
template <typename T1,typename T2>
class BSTClass                                  //二叉排序树类模板
{
public:
   BSTNode<T1,T2>* r;                           //二叉排序树根结点
   BSTNode<T1,T2>* f;                           //用于临时存放待删除结点的双亲
   BSTClass()                                   //构造函数
   {   r=NULL; f=NULL; }
   ~BSTClass()                                  //析构函数
   {  DestroyBTree(r);                          //调用DestroyBTree()函数
      r=NULL;
   }
   void DestroyBTree(BSTNode<T1,T2>* b)      //释放所有的结点空间
   {
      …
   }
   //二叉排序树的基本运算算法
};
```

2．二叉排序树的插入和生成

在根结点*p*的二叉排序树中插入关键字为*k*的结点的过程如下：

- 若*p*为空，创建一个key为*k*的结点，返回将它作为根结点。
- 若*k*<p->key，将*k*插入*p*结点的左子树中并且修改*p*的左指针。
- 若*k*>p->key，将*k*插入*p*结点的右子树中并且修改*p*的右指针。
- 其他情况是*k*=p->key，说明树中已有关键字*k*，修改data值并返回*p*。

```
void InsertBST(T1 k,T2 d)                          //插入一个(k,d)结点
{
   r=_InsertBST(r,k,d);
}

BSTNode<T1,T2>* _InsertBST(BSTNode<T1,T2>* p,T1 k,T2 d)
//在以p为根的BST中插入关键字为k的结点
{  if (p==NULL)                                    //原树为空,为根结点
      p=new BSTNode<T1,T2>(k,d);
   else if (k<p->key)
      p->lchild=_InsertBST(p->lchild,k,d);         //插入到p的左子树中
   else if (k>p->key)
      p->rchild=_InsertBST(p->rchild,k,d);         //插入到p的右子树中
   else                                            //相同关键字,修改data域
      p->data=d;
   return p;
}
```

创建二叉排序树r是从一个空树开始，先创建根结点，以后每插入一个关键字*k*，就调用一次InsertBST(k,d)算法将(k,d)插入到当前的二叉排序树中。

```
void CreateBST(vector<T1>& a,vector<T2>& b)
//由a和b向量创建一棵二叉排序树
{  r=new BSTNode<T1,T2>(a[0],b[0]);          //创建根结点
   for (int i=1;i<a.size();i++)              //创建其他结点
      InsertBST(a[i],b[i]);                  //插入(a[i],b[i])
}
```

3．二叉排序树的查找

查找算法：

```
BSTNode<T1,T2>* SearchBST(T1 k)  //在二叉排序树中查找关键字为k的结点
{
   return _SearchBST(r,k);            //r为二叉排序树的根结点
}

BSTNode<T1,T2>* _SearchBST(BSTNode<T1,T2>* p,T1 k)
//被SearchBST方法调用
{  if (p==NULL) return NULL;           //空树返回NULL
   if (p->key==k) return p;            //找到后返回p
   if (k<p->key)
      return _SearchBST(p->lchild,k); //在左子树中递归查找
   else
      return _SearchBST(p->rchild,k); //在右子树中递归查找
}
```

查找说明：

- 与折半查找的判定树类似，在二叉排序树中每个空指针处添加一个外部结点。
- 在二叉排序树中查找时，若查找成功，则是从根结点出发走了一条从根结点到查找到结点的路径。
- 若查找不成功，则是从根结点出发走了一条从根到某个外部结点的路径。
- 因此与折半查找类似，查找中关键字比较的次数不超过树的高度。

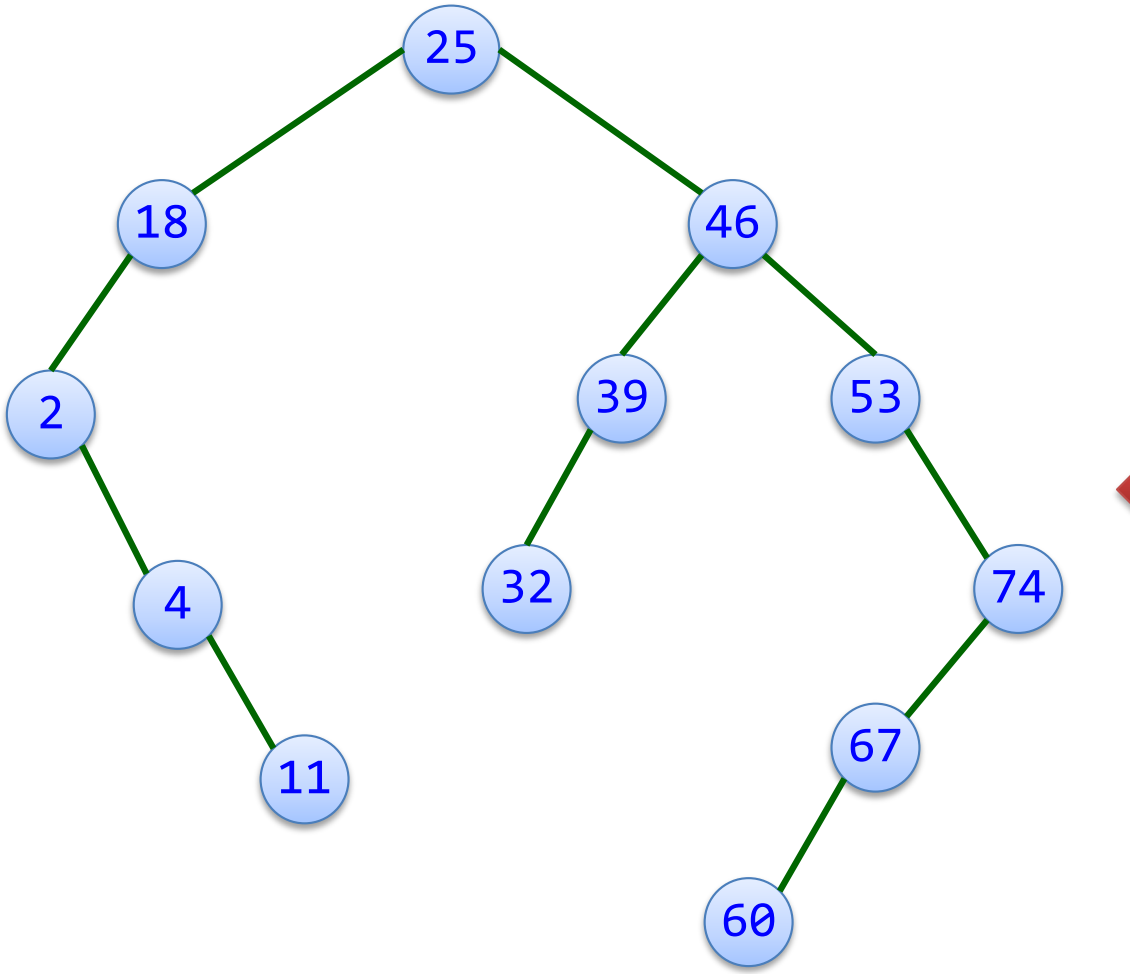
【例9.7】 已知一组关键字为

（25,18,46,2,53,39,32,4,74,67,60,11）

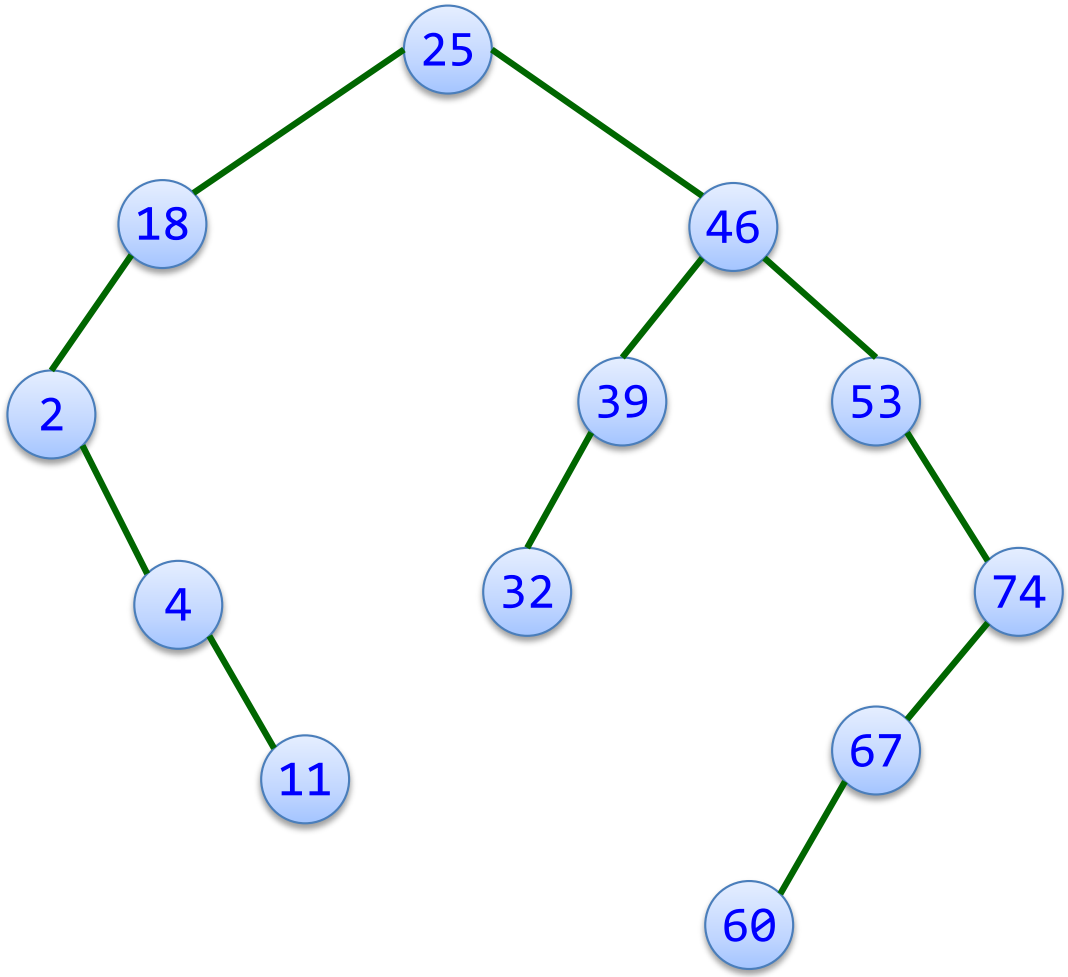
按表中的元素顺序依次插入到一棵初始为空的二叉排序树中，画出该二叉排序树，并求在等概率的情况下查找成功的平均查找长度和查找不成功的平均查找长度。

生成的二叉排序树如下。

25 18 46 2 53 39 32 4 74 67 60 11

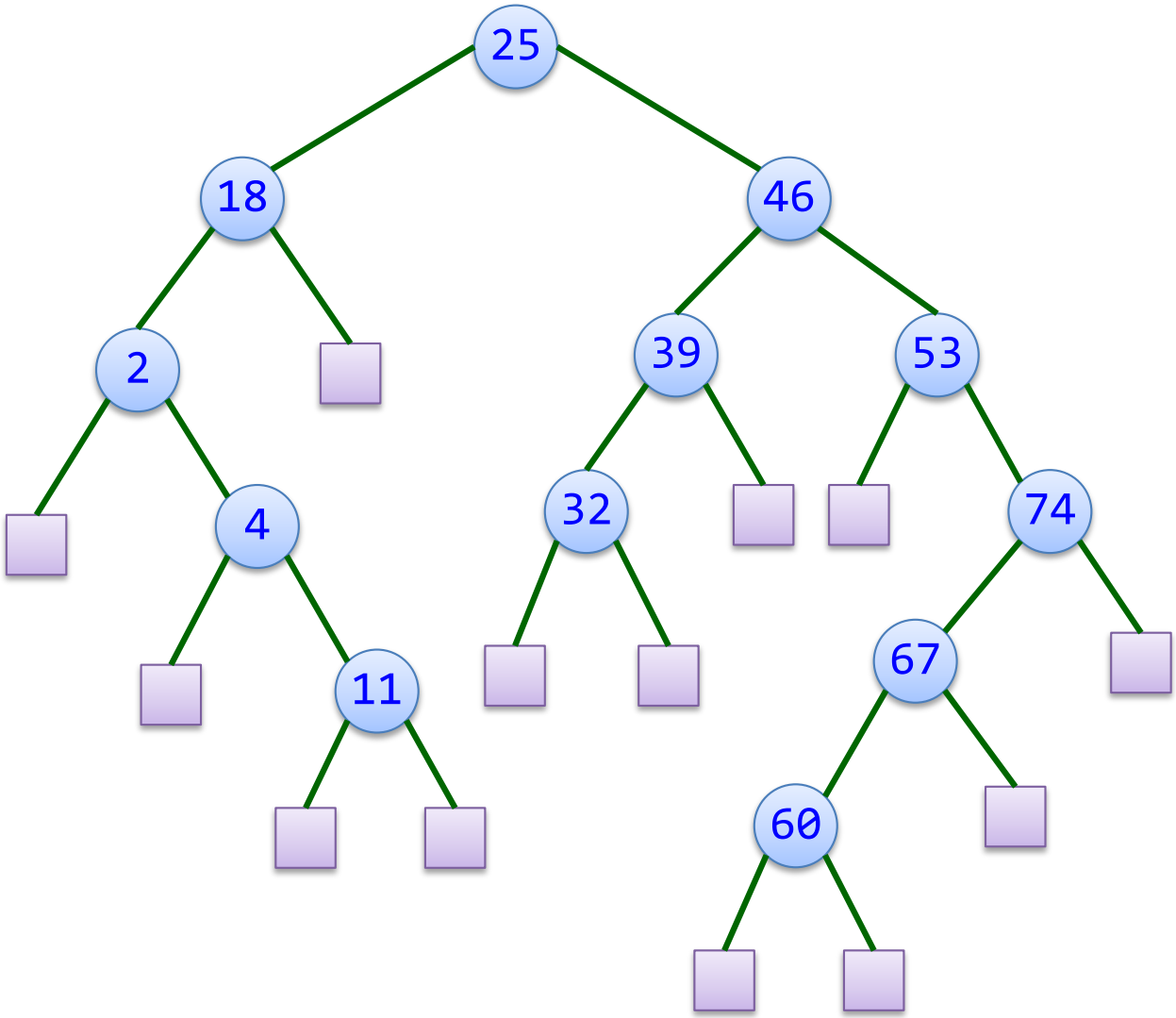


生成的二叉排序树



在等概率的情况下，查找成功的平均查找长度为：

$$ASL_{成功} = \frac{1\times1+2\times2+3\times3+3\times4+2\times5+1\times6}{12} = 3.5$$

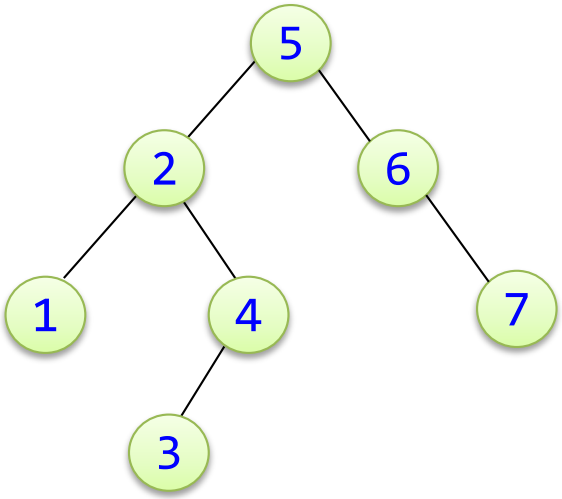


在等概率的情况下，查找不成功的平均查找长度为：

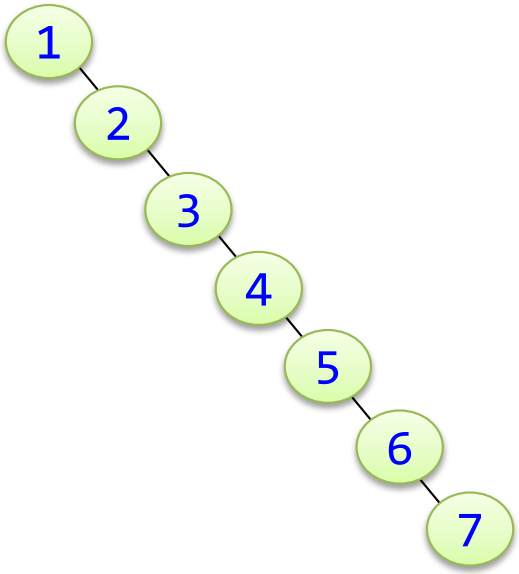
$$ASL_{不成功} = \frac{1\times 2 + 3\times 3 + 4\times 4 + 3\times 5 + 2\times 6}{13} = 4.15$$

提示

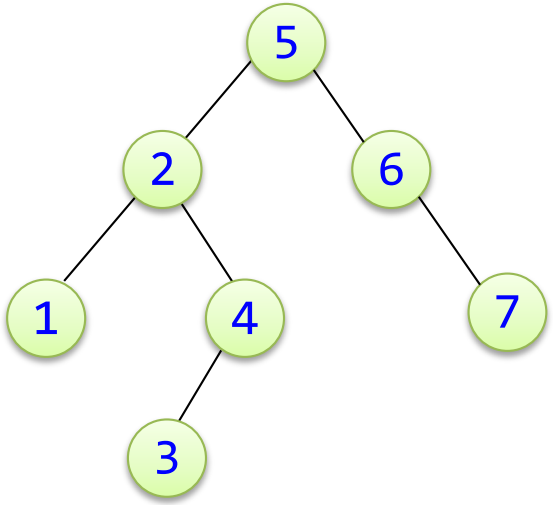
- 一个关键字集合可以有多个不同顺序的关键字序列，对于不同的关键字序列，CreateBST()算法创建的二叉排序树可能不同。
- 例如，关键字序列为（5，2，1，6，7，4，3），创建的二叉排序树如图(a)所示。若关键字序列为（1，2，3，4，5，6，7），创建的二叉排序树如图(b)所示。



（a）高度为4

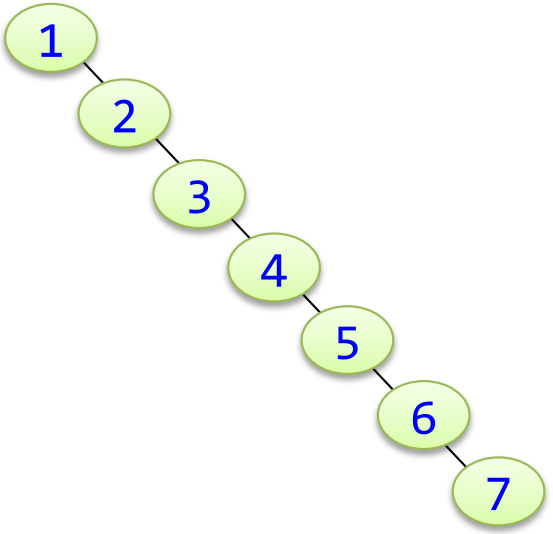


（b）高度为7



（a）高度为4

$$ASL_a = \frac{1\times 1 + 2\times 2 + 3\times 3 + 1\times 4}{7} = 2.57$$



（b）高度为7

$$ASL_b = \frac{1\times 1 + 1\times 2 + 1\times 3 + 1\times 4 + 1\times 5 + 1\times 6 + 1\times 7}{7} = 4$$

那么如何分析二叉排序树的查找性能呢？有如下两种分析方法。

- 给定含n个关键字的集合，假设所有关键字不相同，对应有$n!$个关键字序列，每个关键字序列构造一棵二叉排序树，所有这些二叉排序树中查找每个关键字的平均时间为$O(\log_2 n)$。
- 给定含n个关键字的关键字序列构造一棵二叉排序树。其中查找性能最好的是高度最小的二叉排序树，最好查找性能为$O(\log_2 n)$。查找性能最坏的是高度为n的二叉排序树（单支树），最坏查找性能为$O(n)$。平均情况由具体的关键字序列来确定。所以常说二叉排序树的时间复杂度在$O(\log_2 n)$和$O(n)$之间，就是指这种分析方法。

【例9.8】在含有27个结点的二叉排序树上，查找关键字为35的结点，以下4个选项中哪些是可能的关键字比较序列？

A.28，36，18，46，35　　B.18，36，28，46，35

C.46，28，18，36，35　　D.46，36，18，28，35

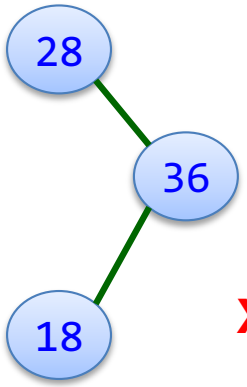
- 查找序列（k_1，k_2，…，k_n）的查找树画法是，每一层只有一个结点，首先k_1为根结点，再依次画出其他结点，若$k_{i+1}<k_i$，则k_{i+1}的结点作为k_i结点的左孩子，否则作为右孩子。
- 查找树是原来二叉排序树的一部分，也一定构成一棵二叉排序树。

A.28,36,18,46,35

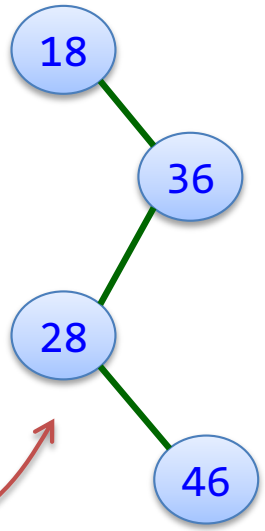
B.18,36,28,46,35

C.46,28,18,36,35

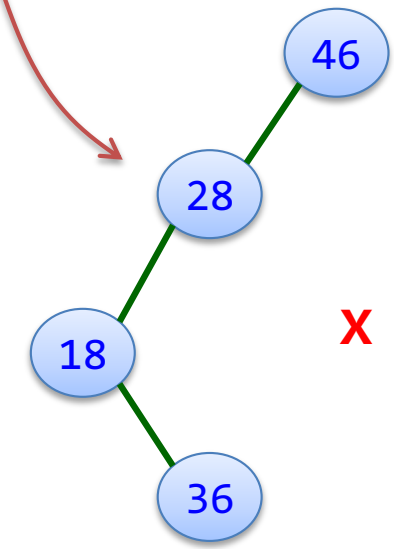
D.46,36,18,28,35



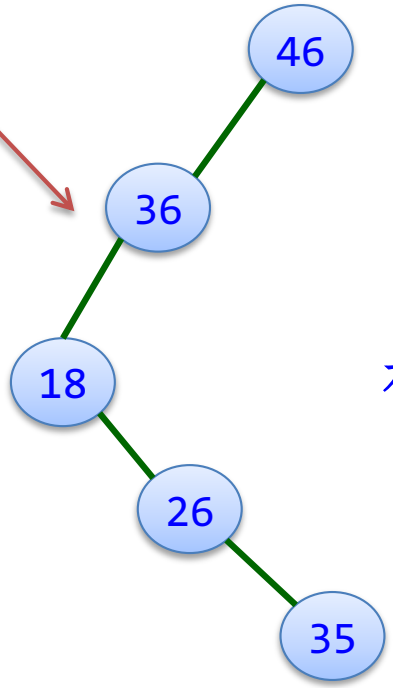
X



X



X



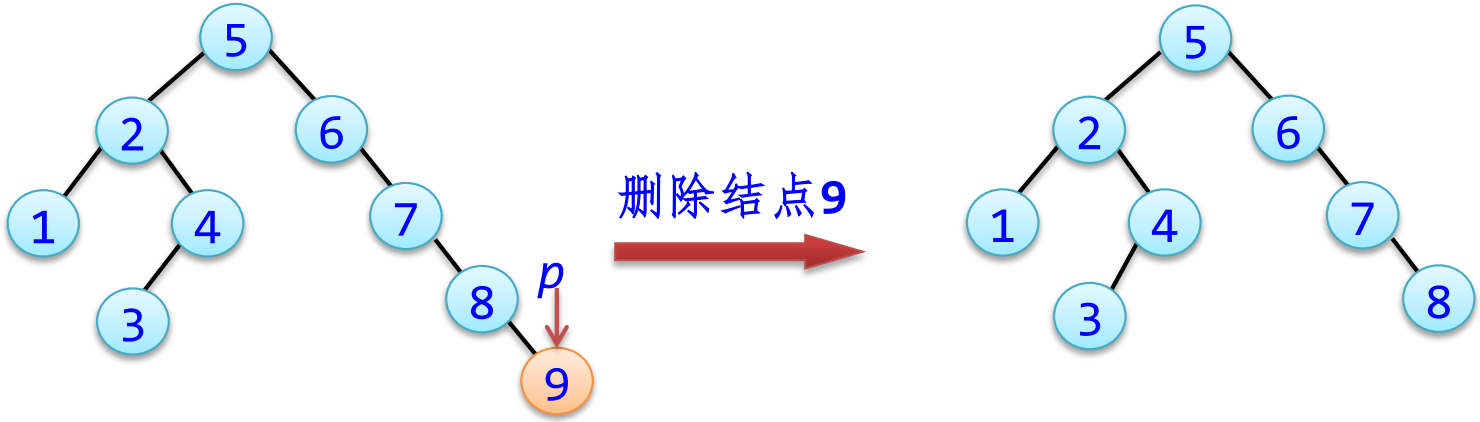
是一棵二叉排序√

4．二叉排序树的删除

- 删除关键字与删除结点是一回事（每个结点一个关键字）。
- 删除一个结点时不能简单地把以该结点为根的子树都删去，只能删除该结点本身，并且还要保证删除后的二叉树仍然满足BST性质。
- 也就是说，在二叉排序树中删除一个结点就相当于删除有序序列（即该树的中序序列）中的一个结点。

删除结点*p*分为以下几种情况

（1）若结点*p*是叶子结点（结点*p*的度为0），删除该结点等同于删除该结点的子树，所以可以直接删除该结点。



（a）结点*p*为叶子结点：直接删除

（2）若结点*p*只有左孩子没有右孩子（结点*p*的度为1），根据二叉排序树的特点，可以用结点*p*的左子树替代结点*p*的子树，也就是直接用其左孩子替代它（结点替代）。

（b）结点*p*仅有左孩子：用左孩子*q*结点替代结点*p*

（3）若结点*p*只有右孩子没有左孩子（结点*p*的度为1），根据二叉排序树的特点，可以用结点*p*的右子树替代结点*p*的子树，也就是直接用其右孩子替代它（结点替代）。

（c）结点*p*仅有右孩子：用右孩子*q*结点替代结点*p*

（4）若结点*p*既有左孩子又有右孩子（结点*p*的度为2） ⇨ 间接删除。

（d）结点*p*有左右孩子：找到其左孩子的最右下结点*q*，置*p*结点值为*q*结点值（值替代），再删除*q*结点（*q*结点没有右孩子，最多只有左孩子，采用(b)删除）

删除算法：

- 从二叉排序树中删除结点*p*是通过修改其双亲的相关指针实现的。
- 为此需要标识结点*p*的双亲结点*f*，并且用**flag**标识结点*p*是结点*f*的何种孩子，**flag=-1**表示结点*p*是根结点没有双亲，**flag=0**表示结点*p*是结点*f*的左孩子，**flag=1**表示结点*p*是结点*f*的右孩子。
- 所以，删除中的查找不能简单地采用前面的查找算法，而需要在查找中确定结点*p*对应的双亲结点*f*和左右孩子标记**flag**。

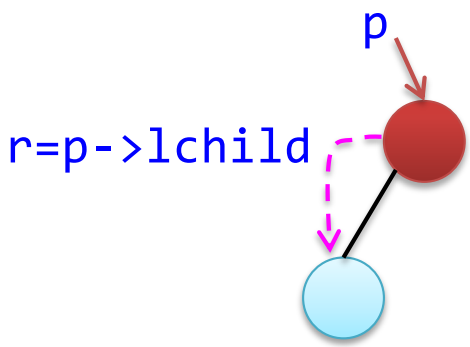
```
bool DeleteBST(T1 k)                                    //删除关键字为k的结点
{  f=NULL;
   return _DeleteBST(r,k,-1);                           //r为二叉排序树的根结点
}

bool _DeleteBST(BSTNode<T1,T2>* p,T1 k,int flag)
//被DeleteBST方法调用
{  if (p==NULL)
      return false;                                     //空树返回false
   if (p->key==k)
      return DeleteNode(p,f,flag);                      //找到后删除p结点
   if (k<p->key)
   {  f=p;
      return _DeleteBST(p->lchild,k,0);                 //在左子树中递归查找
   }
   else
   {  f=p;
      return _DeleteBST(p->rchild,k,1);                 //在右子树中递归查找
   }
}
```

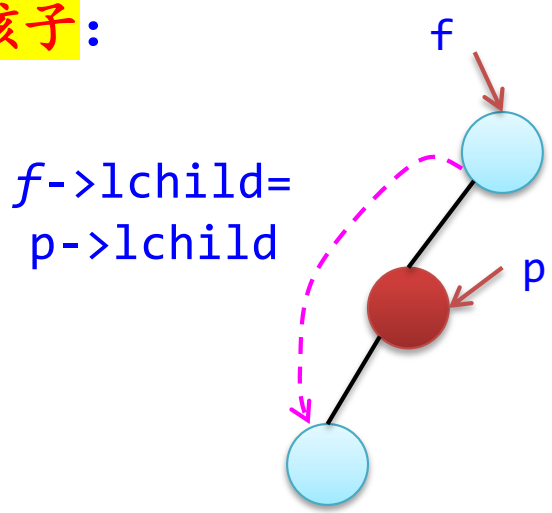
*f*记录找到的结点*p*的双亲结点

```
bool DeleteNode(BSTNode<T1,T2>* p,BSTNode<T1,T2>* f,int flag)
//删除结点p（其双亲为f）
{  if (p->rchild==NULL)                  //结点p只有左孩子(含p为叶子的情况)
   {  if (flag==-1)                      //结点p的双亲为空(p为根结点)
         r=p->lchild;                    //修改根结点r为p的左孩子
      else if (flag==0)                  //p为双亲f的左孩子
         f->lchild=p->lchild;            //将f的左孩子置为p的左孩子
      else                               //p为双亲f的右孩子
         f->rchild=p->lchild;            //将f的右孩子置为p的左孩子
   }
```

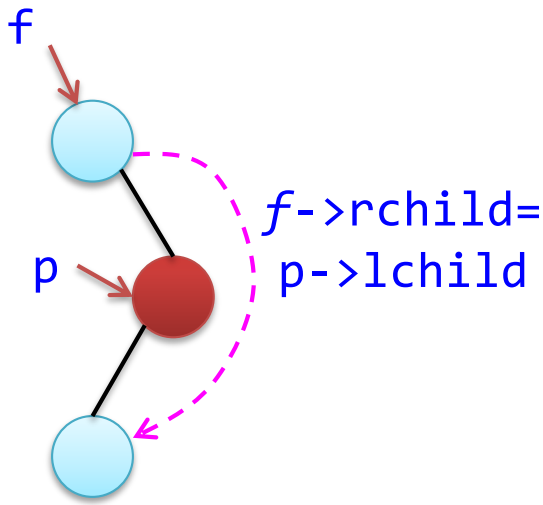
删除结点*p*：它仅有左孩子：



（a）flag=-1



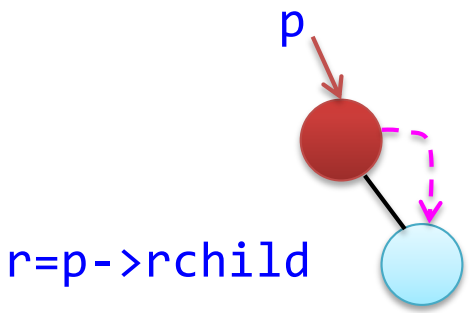
（b）flag=0



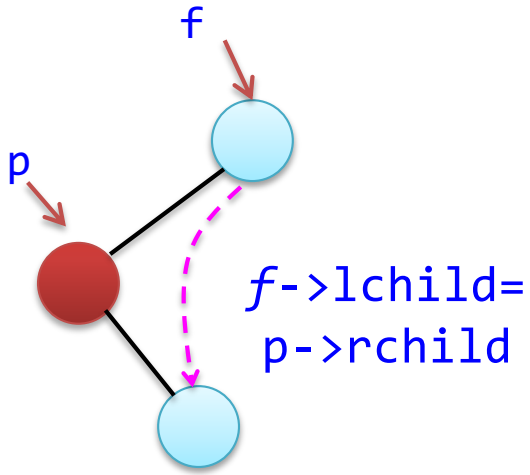
（c）flag=1

```
else if (p->lchild==NULL)          //结点p只有右孩子
{  if (flag==-1)                   //结点p的双亲为空(p为根结点)
      r=p->rchild;                 //修改根结点r为p的右孩子
   else if (flag==0)               //p为双亲f的左孩子
      f->lchild=p->rchild;         //将f的左孩子置为p的左孩子
   else                            //p为双亲f的右孩子
      f->rchild=p->rchild;         //将f的右孩子置为p的左孩子
}
```

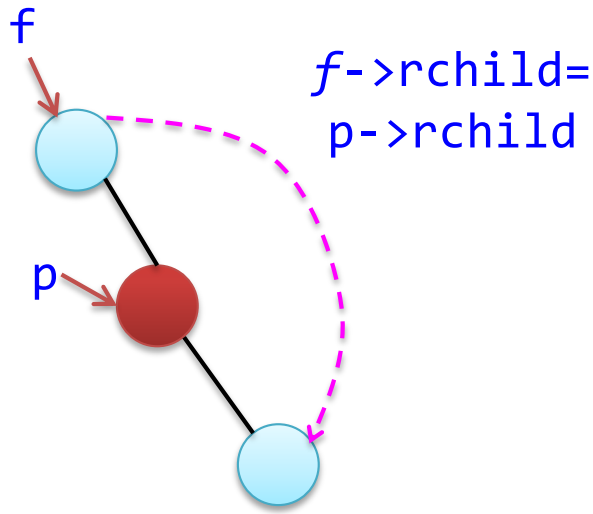
删除结点*p*：它仅有右孩子：与仅有左孩子类似！



（a）flag=-1



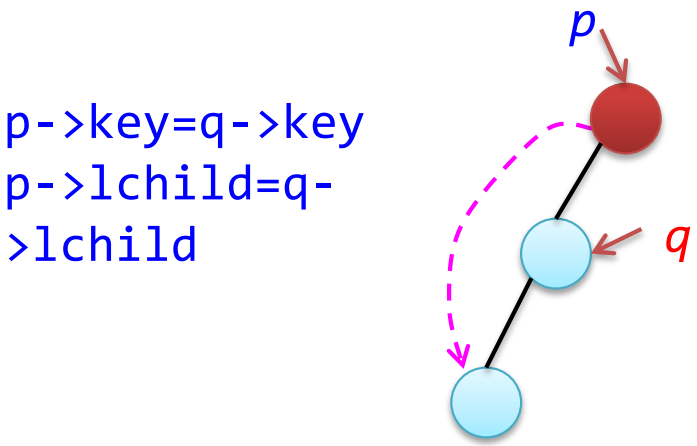
（b）flag=0



（c）flag=1

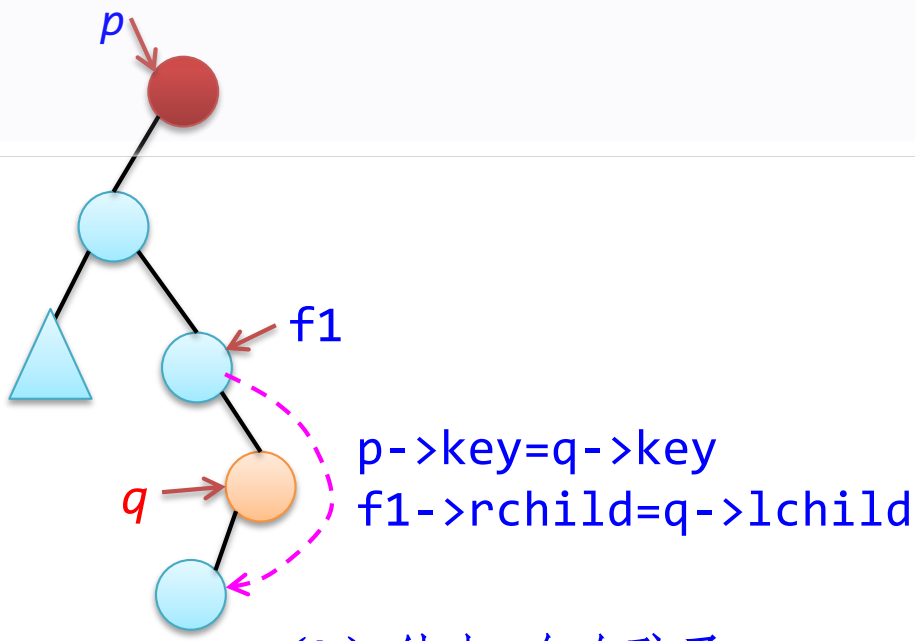
当被删结点*p*有左、右孩子时，采用前面的删除方法，先让*q*指向其左孩子结点，分为以下两种情况：

```
else                                        //结点p有左右孩子
{  BSTNode<T1,T2>* f1=p;                    //f1为结点q的双亲结点
   BSTNode<T1,T2>* q=p->lchild;             //q转向结点p的左孩子
   if (q->rchild==NULL)                     //若结点q没有右孩子
   {  p->key=q->key;                        //将被删结点p的值用q的值替代
      p->data=q->data;
      p->lchild=q->lchild;                  //删除结点q
   }
```



（a）结点q没有右孩子

```
        else                                    //若结点q有右孩子
        {  while (q->rchild!=NULL)              //找到最右下结点q,其双亲结点为f1
           {  f1=q;
              q=q->rchild;
           }
           p->key=q->key;                       //将被删结点p的值用q的值替代
           p->data=q->data;
           f1->rchild=q->lchild;                //删除结点q
        }
    }
    return true;
}
```

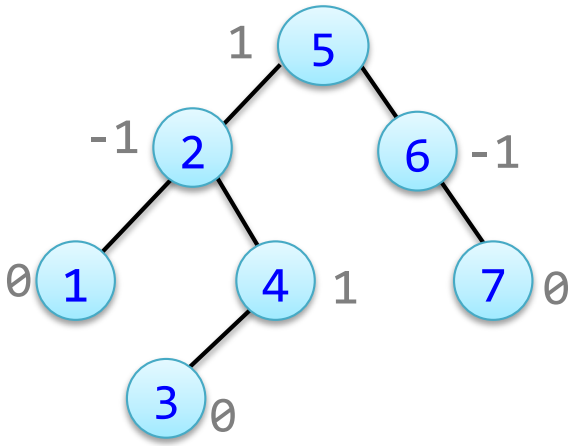


（b）结点q有右孩子

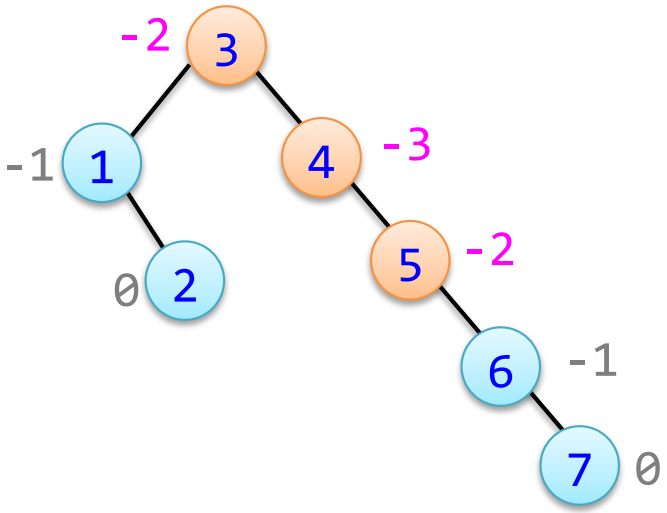
9.3.2 平衡二叉树

- 既保持BST性质又保证树的高度较小，通过这样的平衡规则和操作来维护$O(\log_2 n)$高度的二叉排序树称为平衡二叉树，平衡二叉树有多种。
- 较为著名的有AVL树。

- AVL树的高度平衡性质：树中每个结点的左、右子树的高度至多相差1。
- 也就是说，如果树T中结点*v*有孩子结点*x*和*y*，则$|h(x)-h(y)|\leqslant 1$，$h(x)$表示以结点*x*为根的子树高度。



（a）一棵AVL树



（b）一棵非AVL树

AVL树中结点类型设计为AVLNode<T1，T2>类模板，每个结点存放[key，data]，其中关键字key为T1类型，数据项data为T2类型。

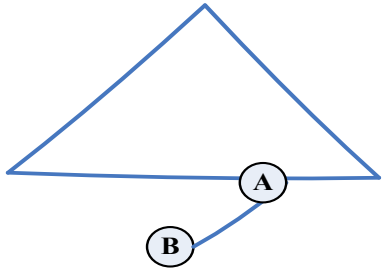
```
template <typename T1,typename T2>
struct AVLNode                      //AVL树结点类模板
{  T1 key;                          //关键字k
   T2 data;                         //关键字对应的值d
   int ht;                          //当前结点的子树高度
   AVLNode* lchild,*rchild;         //左右指针

   AVLNode(T1 k,T2 d)               //构造函数，新建结点均为叶子，高度为1
   {  key=k;
      data=d;
      ht=1;                         //当前结点的子树高度
      lchild=rchild=NULL;
   }
};
```

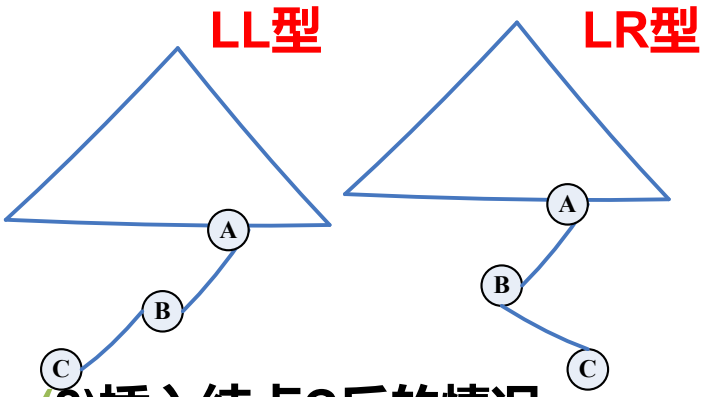
设计对应的AVL树类模板为AVLTree<T1，T2>，其中省略的析构函数与前面BSTClass<T1，T2>中析构函数完全相同。

```
template <typename T1,typename T2>
class AVLTree                          //AVL树类模板
{  AVLNode* r;                         //AVL的根结点
public:
   AVLTree():r(NULL) {}                //构造函数
   int getht(AVLNode* p)               //返回结点p的子树高度
   {  if (p==NULL) return 0;
      return p->ht;
   }
   //AVL树的其他基本运算算法
};
```

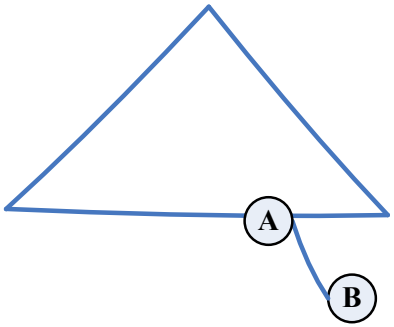
- 如何使构造的二叉排序树是一棵AVL树呢？
- 关键是每次向树中插入新结点时使所有结点的平衡因子满足高度平衡性质，这就要求插入后一旦哪些结点失衡就要进行调整。



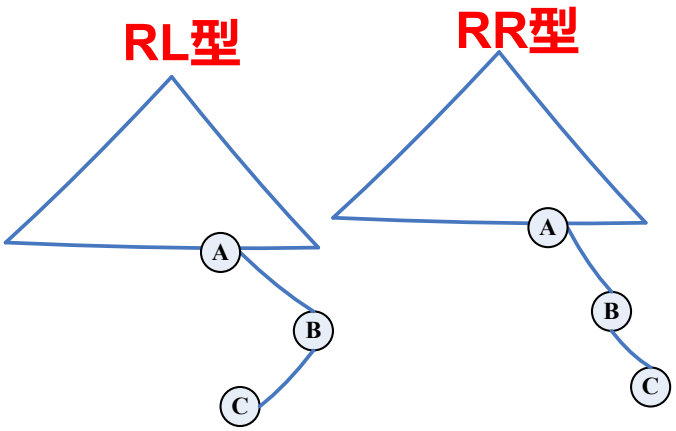
(1)一颗平衡的二叉树



(2)插入结点C后的情况



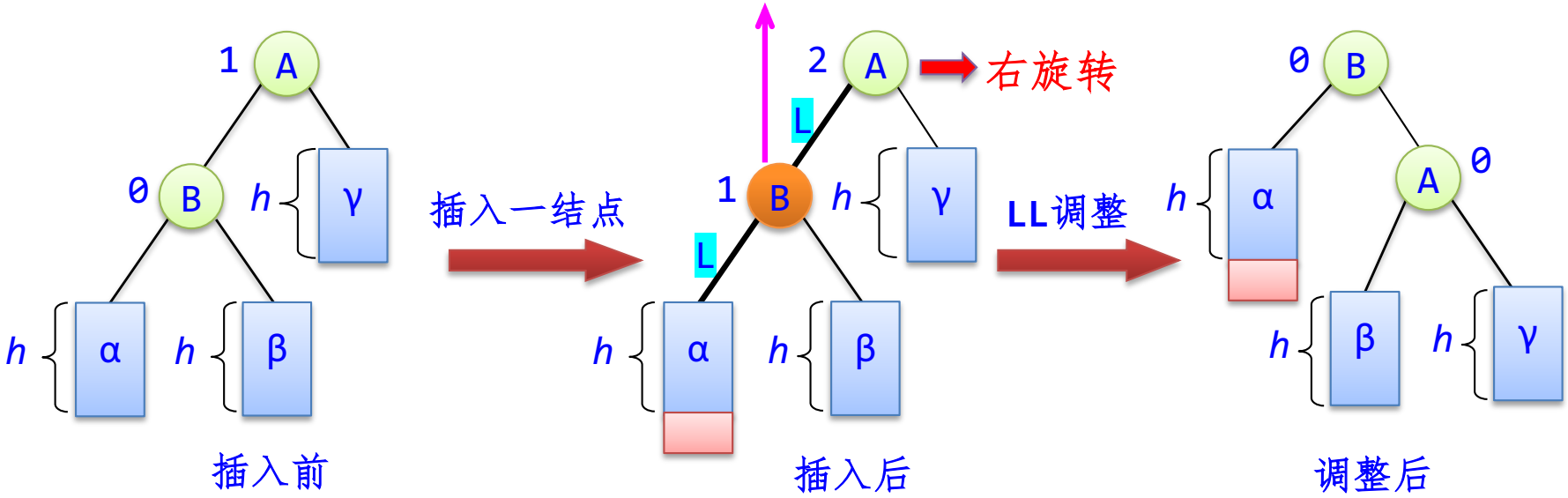
(1)一颗平衡的二叉树



(2)插入结点C后的情况

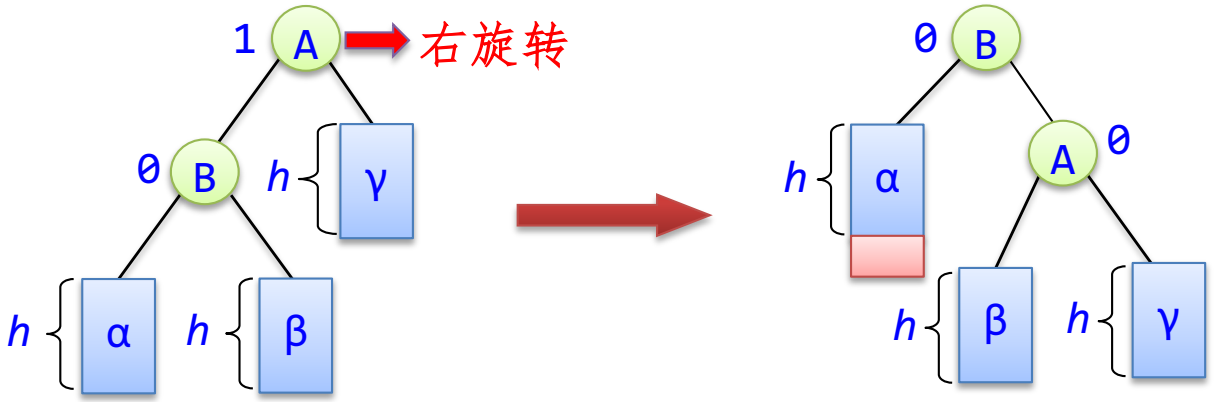
1．AVL树插入结点的调整方法

1）LL型调整



右旋转算法

```
AVLNode* right_rotate(AVLNode* a)              //以结点a为根做右旋转
{   AVLNode* b=a->lchild;
    a->lchild=b->rchild;
    b->rchild=a;
    a->ht=max(getht(a->rchild),getht(a->lchild))+1; //更新A结点的高度
    b->ht=max(getht(b->rchild),getht(b->lchild))+1; //更新B结点的高度
    return b;
}
```

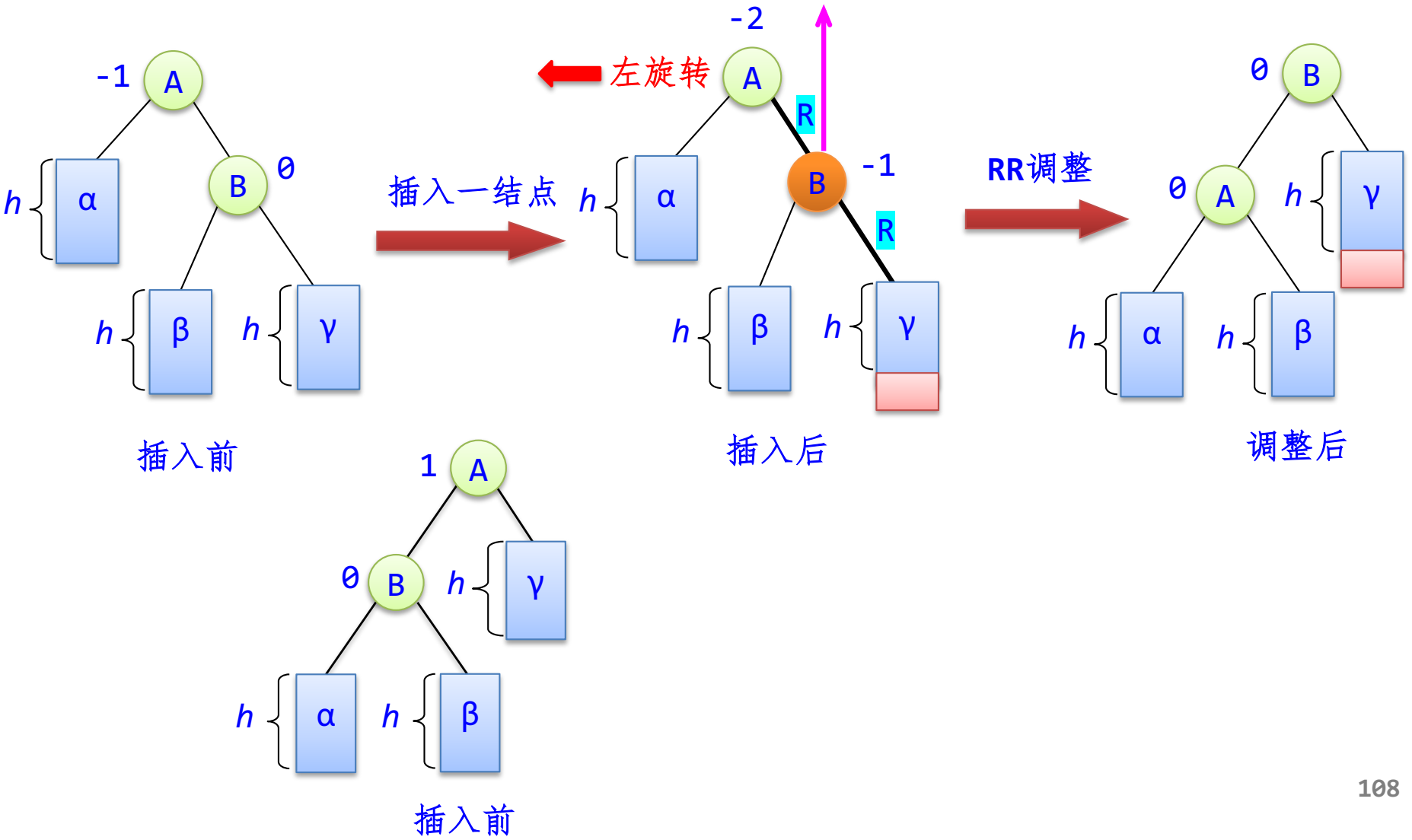


旋转前　　旋转后

直接利用右旋转实现LL调整

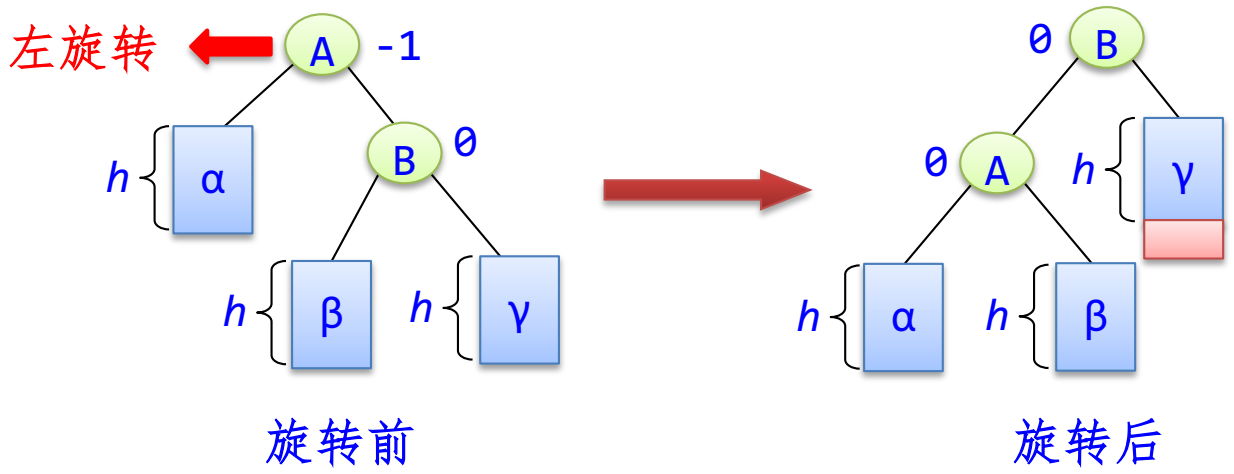
```
AVLNode* LL(AVLNode* a)                    //LL型调整
{
    return right_rotate(a);
}
```

2）RR型调整



左旋转算法

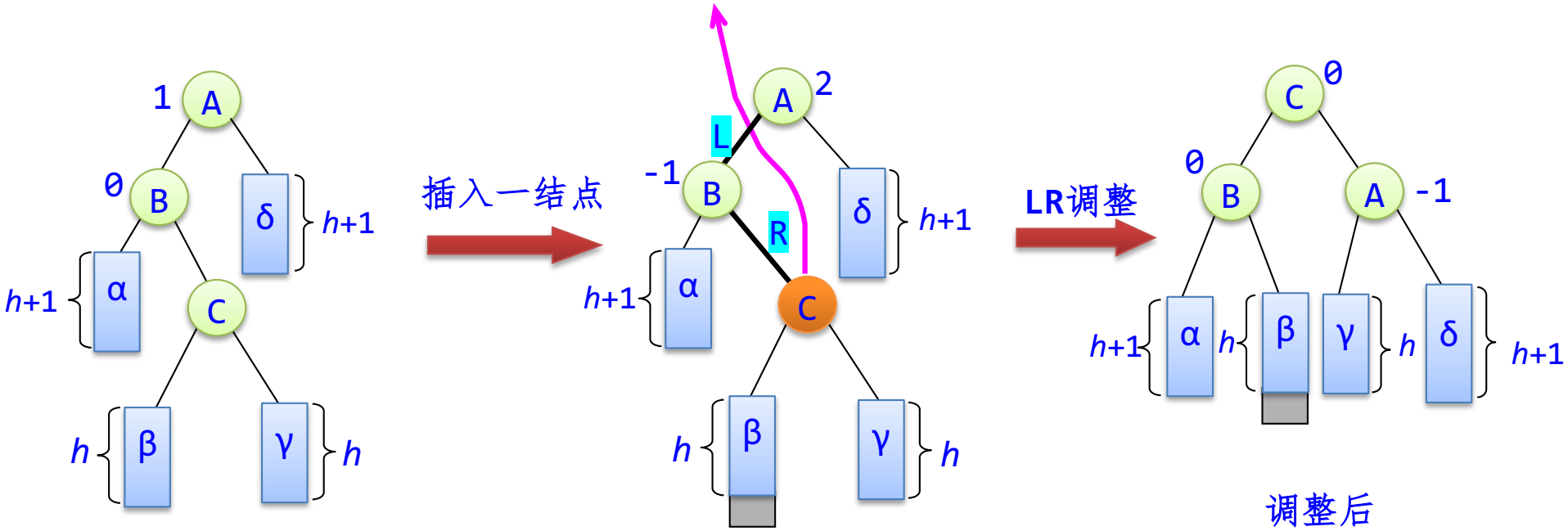
```
AVLNode* left_rotate(AVLNode* a)      //以结点a为根做左旋转
{   AVLNode* b=a->rchild;
    a->rchild=b->lchild;
    b->lchild=a;
    a->ht=max(getht(a->rchild),getht(a->lchild))+1; //更新A结点的高度
    b->ht=max(getht(b->rchild),getht(b->lchild))+1; //更新B结点的高度
    return b;
}
```



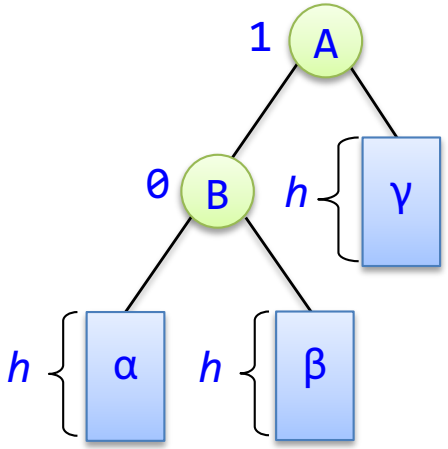
直接利用左旋转实现RR调整

```
AVLNode* RR(AVLNode* a)                    //RR型调整
{
    return left_rotate(a);
}
```

3）LR型调整



插入前　　插入后　　调整后

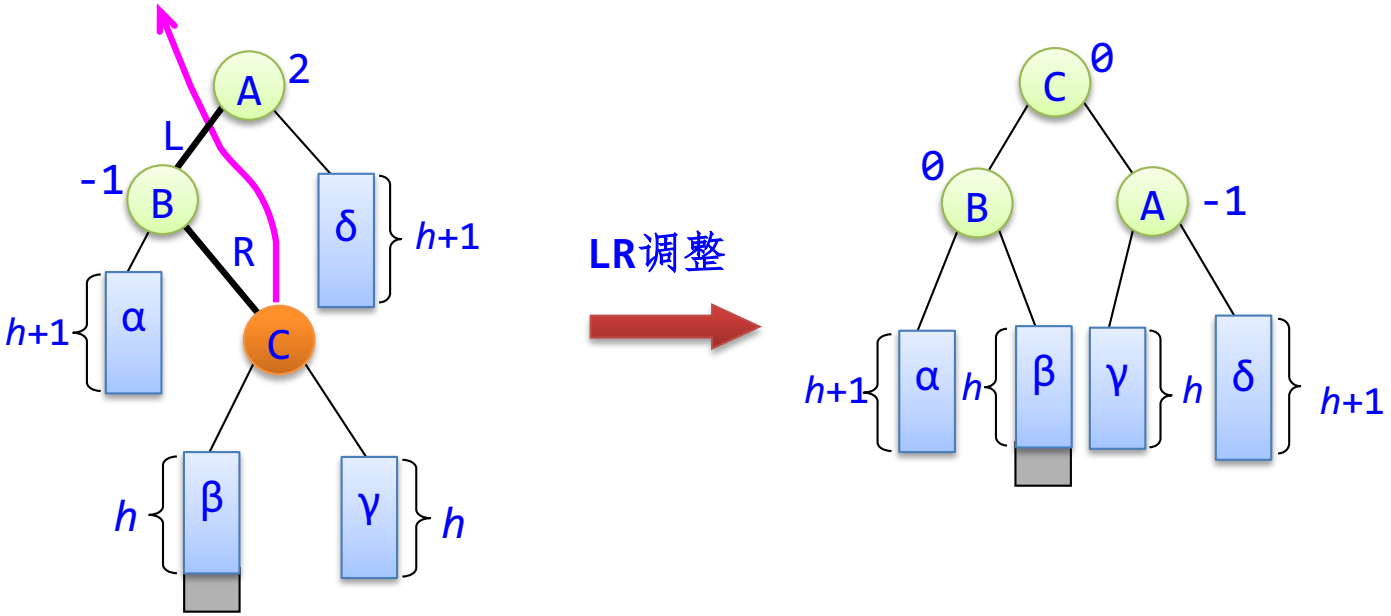


插入前

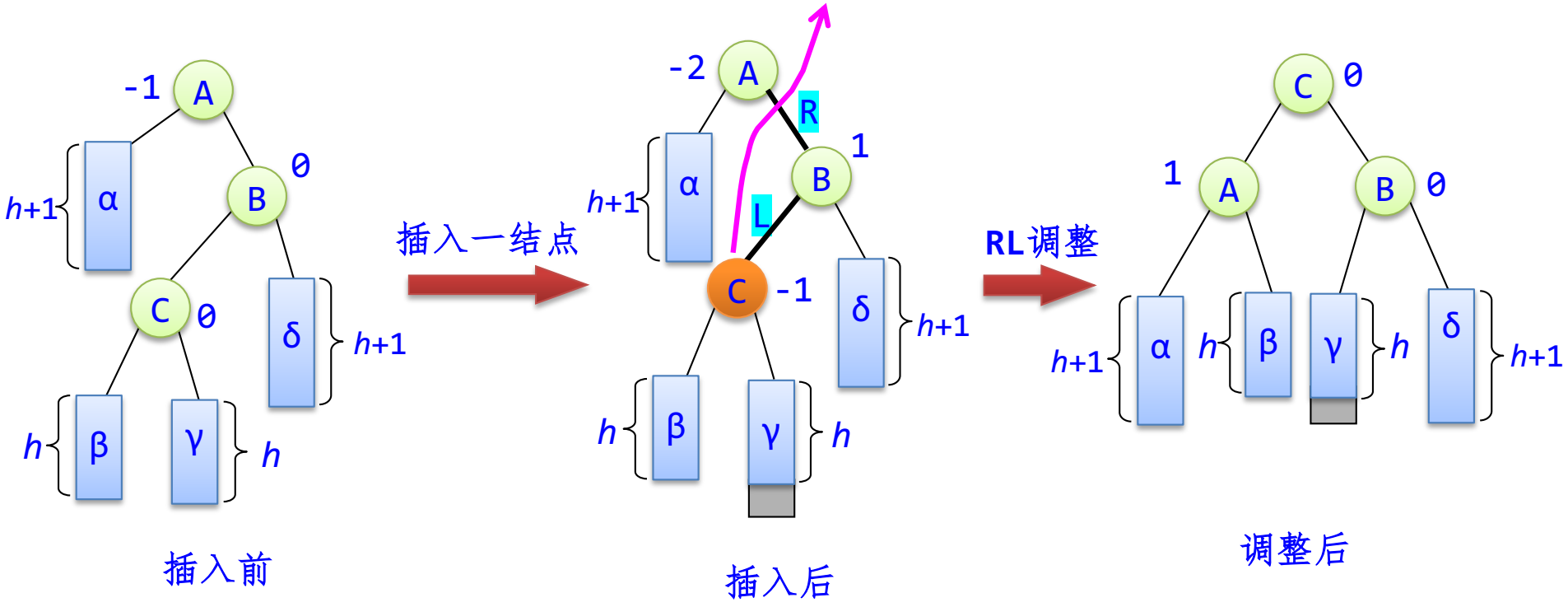
LR双旋转：A的左子树B先左旋转，再按根结点A右旋转！

对应的LR型调整算法

```
AVLNode* LR(AVLNode* a)                          //LR型调整
{  AVLNode* b=a->lchild;
   a->lchild=left_rotate(b);                     //结点b左旋
   return right_rotate(a);                       //结点a右旋
}
```



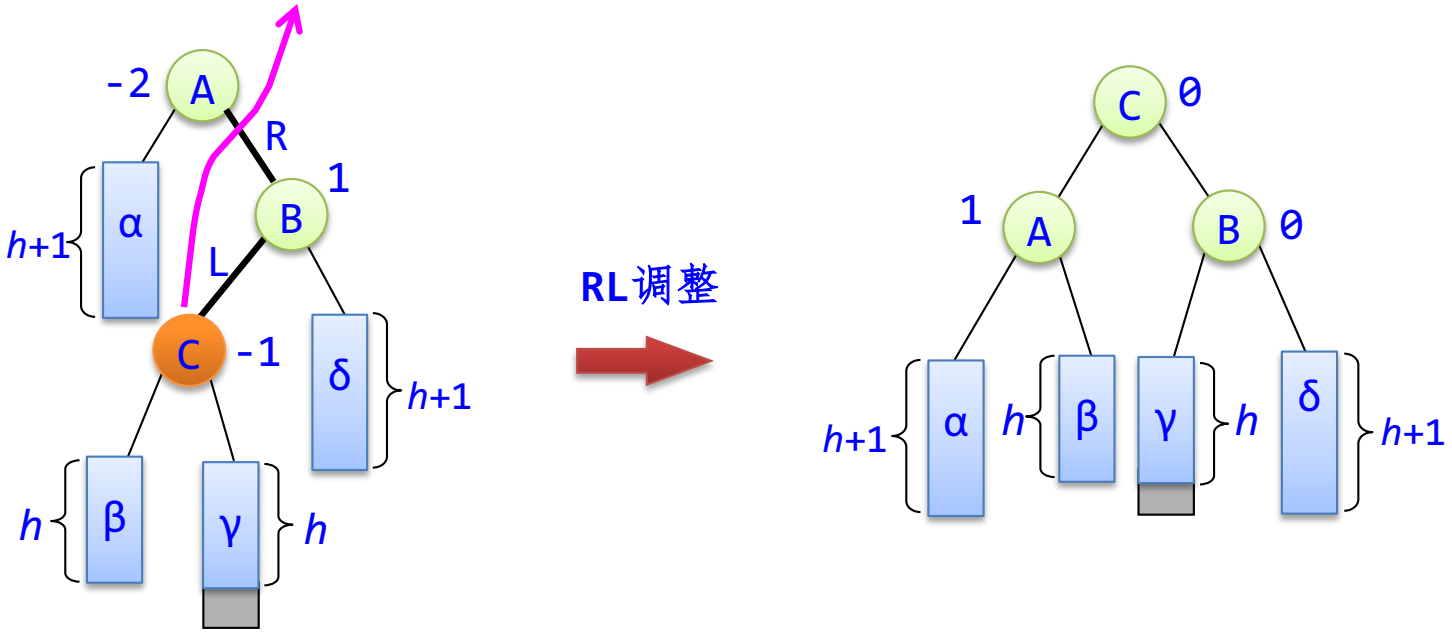
4）RL型调整



RL双旋转：A的右子树B先右旋转，再按根结点A左旋转！

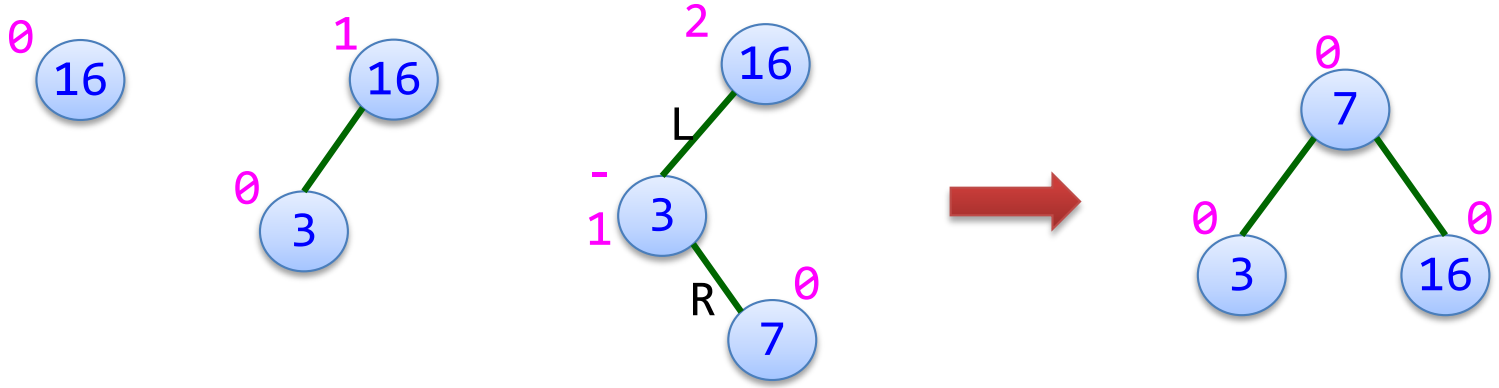
对应的RL型调整算法

```
AVLNode* RL(AVLNode* a)                    //RL型调整
{   AVLNode* b=a->rchild;
    a->rchild=right_rotate(b);             //结点b右旋
    return left_rotate(a);                 //结点a左旋
}
```

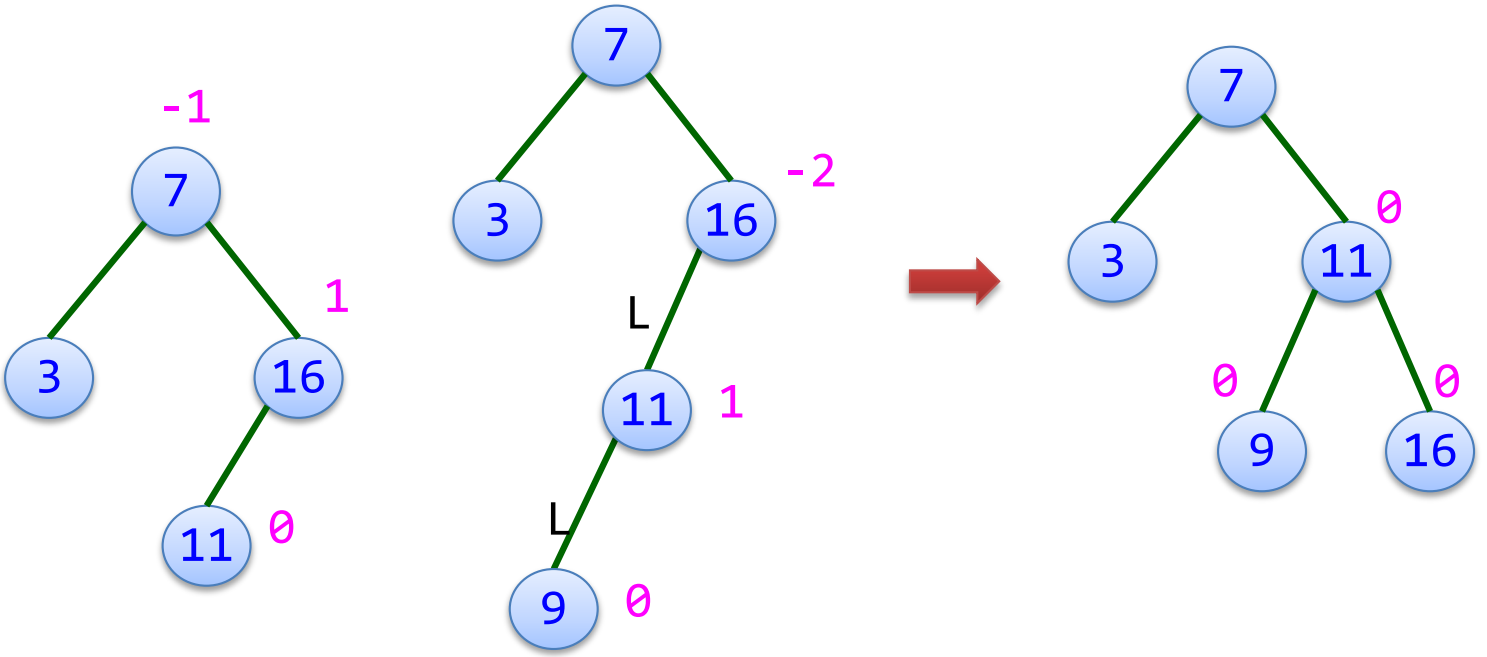


【例9.9】输入关键字序列（16，3，7，11，9，26，18，14，15），给出构造一棵AVL树的步骤。

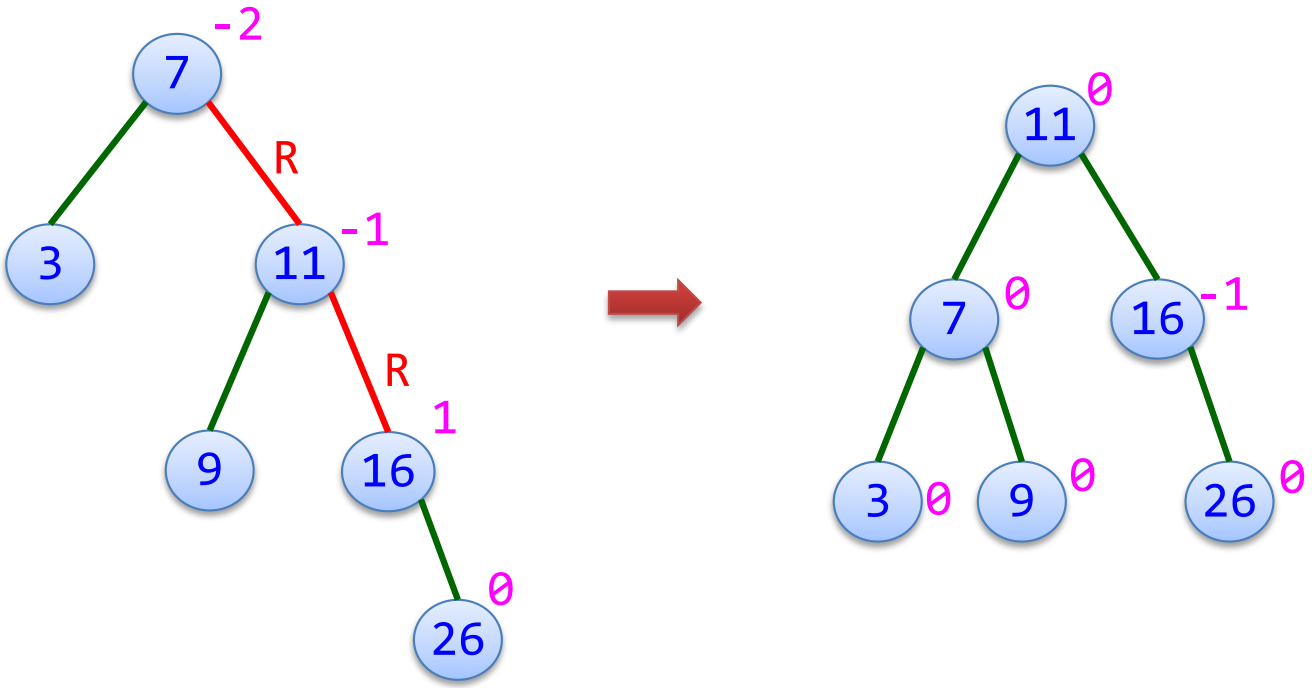
16　3　7



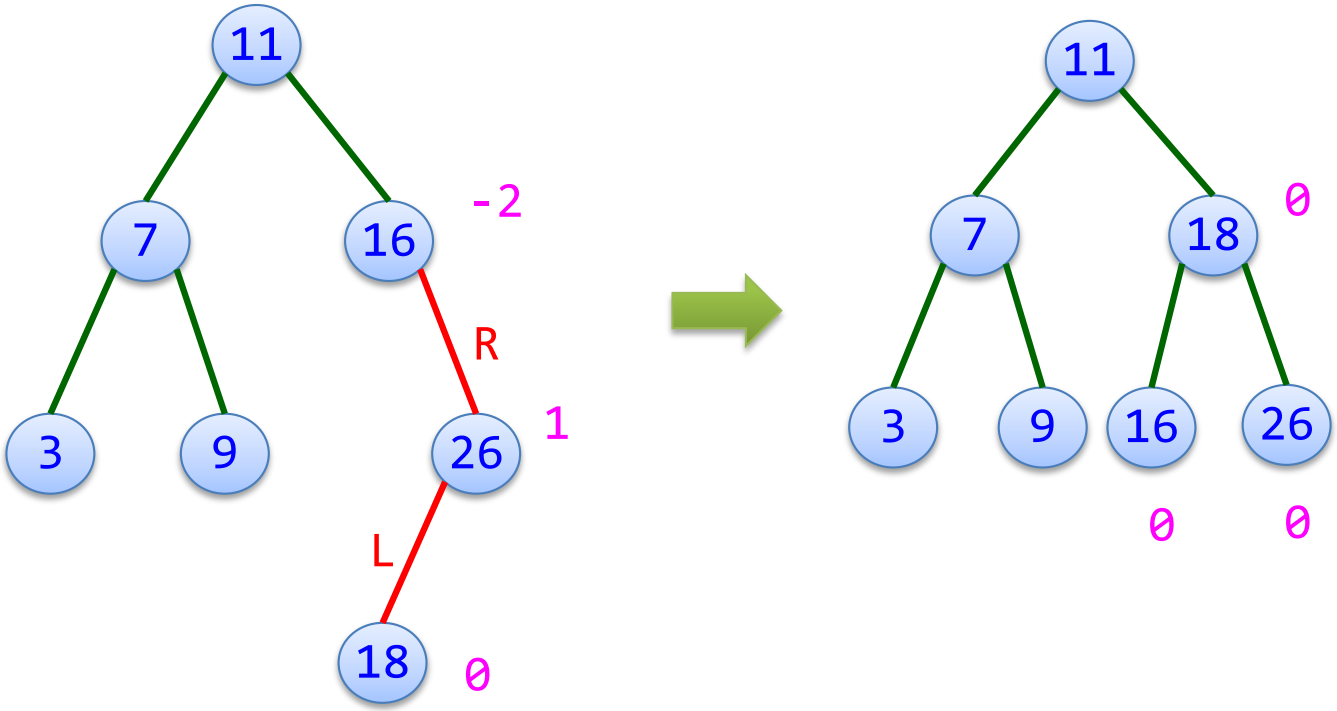
11 9



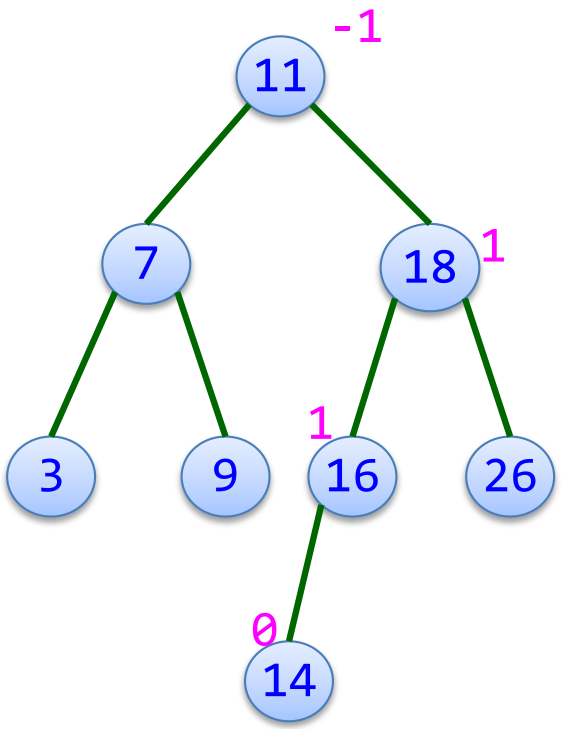
26



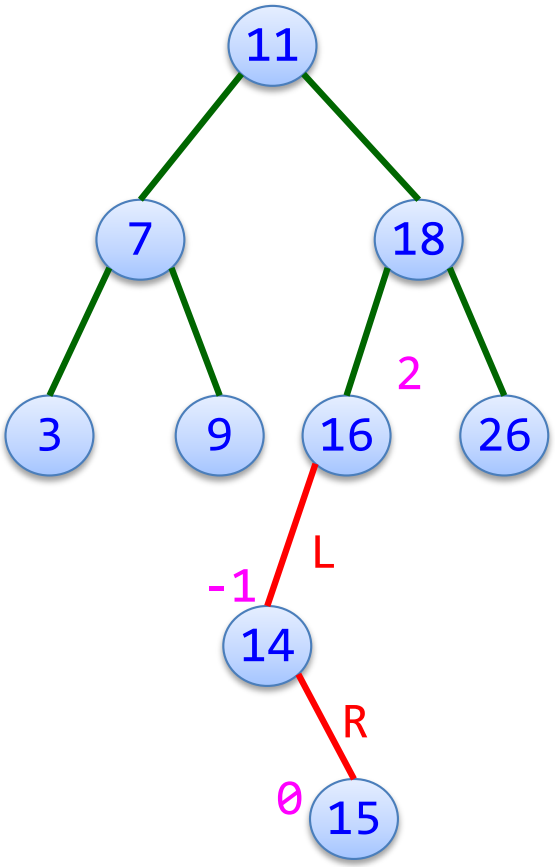
18

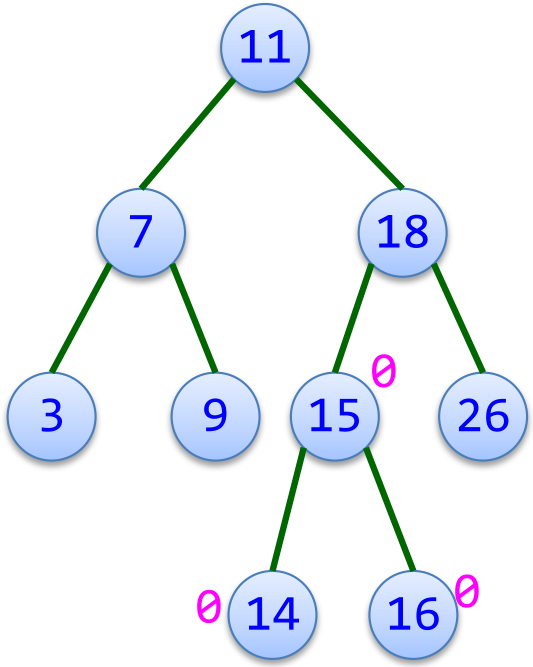


14



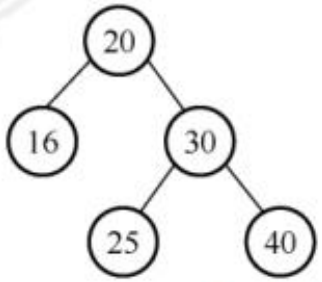
15





构造的结果AVL树

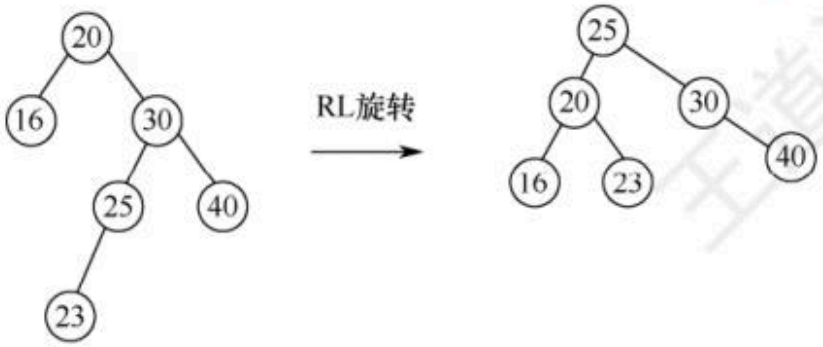
【2021 统考真题】给定平衡二叉树如下图所示，插入关键字 23 后，根中的关键字是（ ）。



A. 16　　B. 20　　C. 23　　D. 25

33. D

关键字 23 的插入位置为 25 的左孩子，此时破坏了平衡的性质，需要对平衡二叉树进行调整。最小不平衡子树就是该树本身，插入位置在根结点的右子树的左子树上，因此需要进行 RL 旋转，RL 旋转过程如下图所示，旋转完成后根结点的关键字为 25，所以选 D。



2．AVL树删除结点的调整方法

首先在AVL树中查找关键字为*k*的结点*x*（假定存在这样的结点并且唯一），删除结点*x*的过程如下：

（1）如果结点*x*左子树为空，用其右孩子结点替换它，即直接删除结点*x*。

（2）如果结点*x*右子树为空，用其左孩子结点替换它，即直接删除结点*x*。

自学

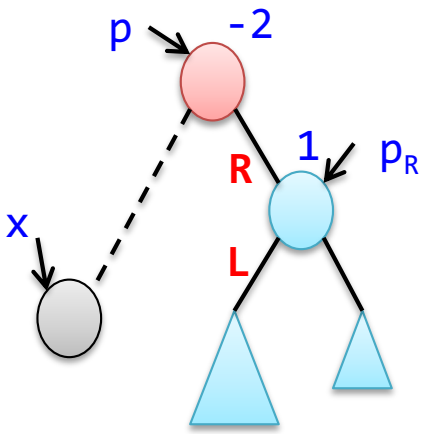
（3）如果结点*x*同时有左右子树（这种情况下，结点*x*是通过值替换间接删除的，称为间接删除结点），分为两种情况：

① 若结点*x*的左子树较高，在其左子树中找到最大结点*q*，直接删除结点*q*，用结点*q*的值替换结点*x*的值。

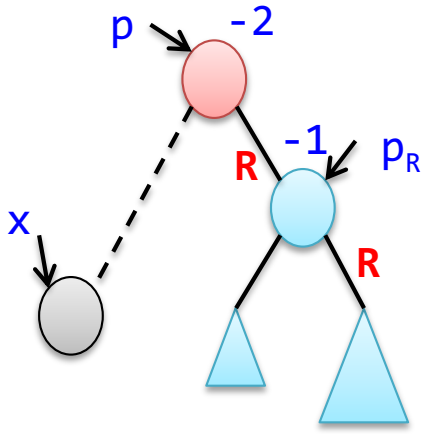
② 若结点*x*的右子树较高，在其右子树中找到最小结点*q*，直接删除结点*q*，用结点*q*的值替换结点*x*的值。

（4）当直接删除结点***x***时，沿着其双亲到根结点方向逐层向上求结点的平衡因子，若一直找到根结点时路径上的所有结点均平衡，说明删除后的树仍然是一棵平衡二叉树，不需要调整，删除结束。若找到路径上的第一个失衡结点***p***，就要进行调整。

① 若直接删除的结点在结点***p***的左子树中。



（a）p_R的左子树高：RL

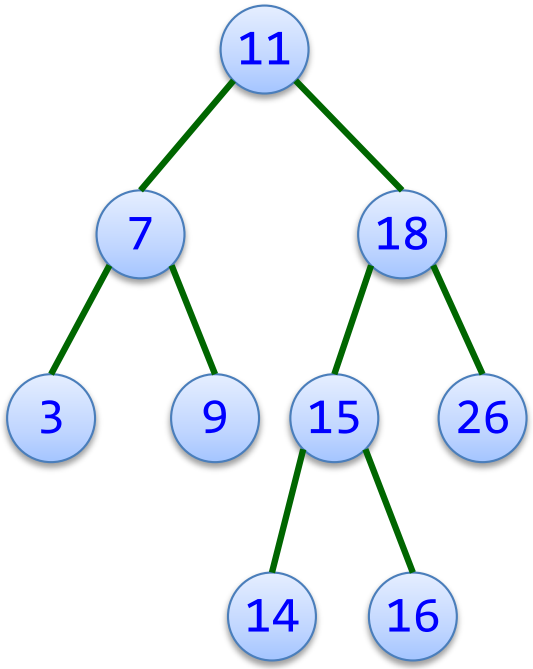


（b）p_R的右子树高：RR

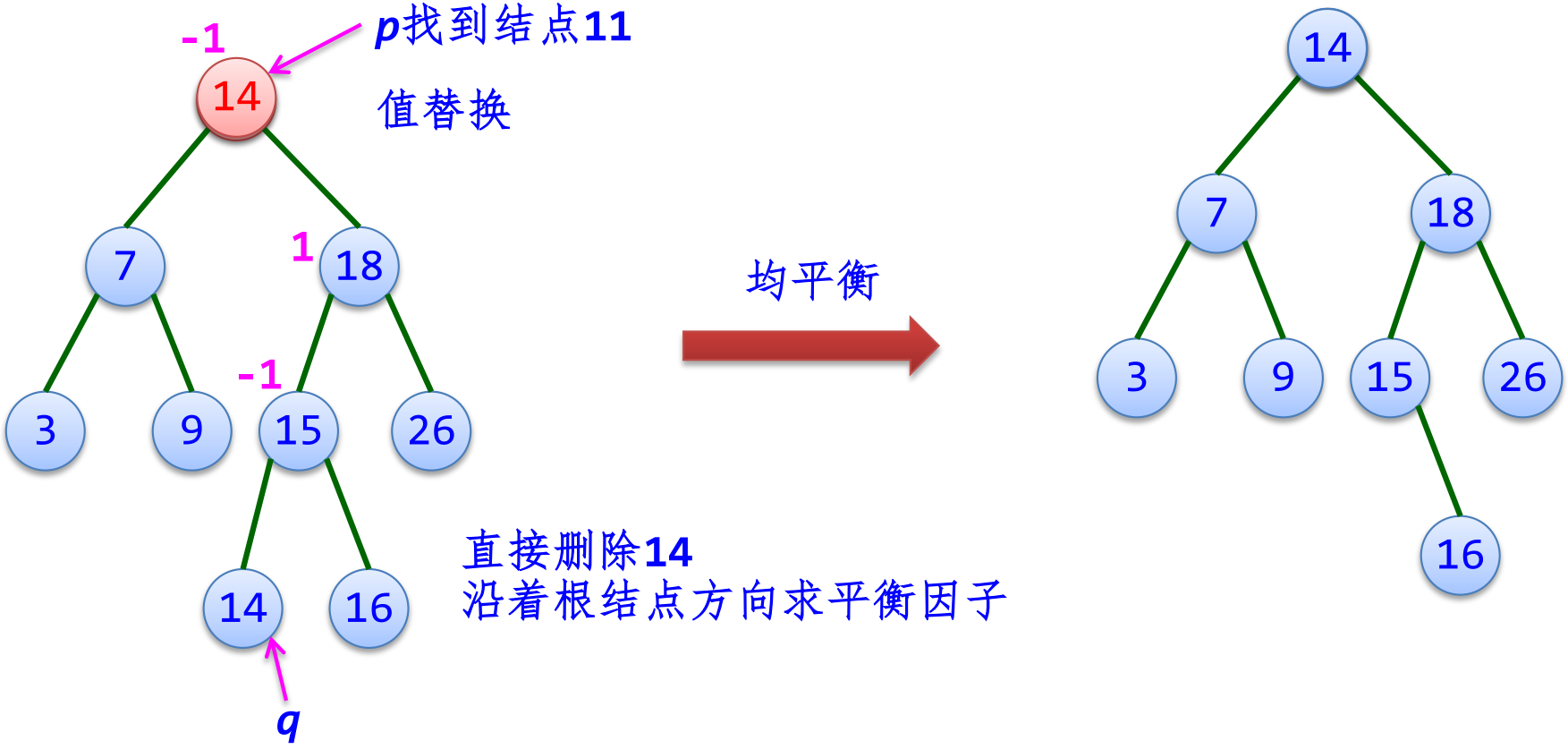
(c)若p_R的左右子树高度相同，则做RL或RR调整均可

② 若直接删除的结点在结点***p***的右子树中，调整过程类似。

【例9.10】对例9.9生成的AVL树，给出删除结点11、9和3的过程。

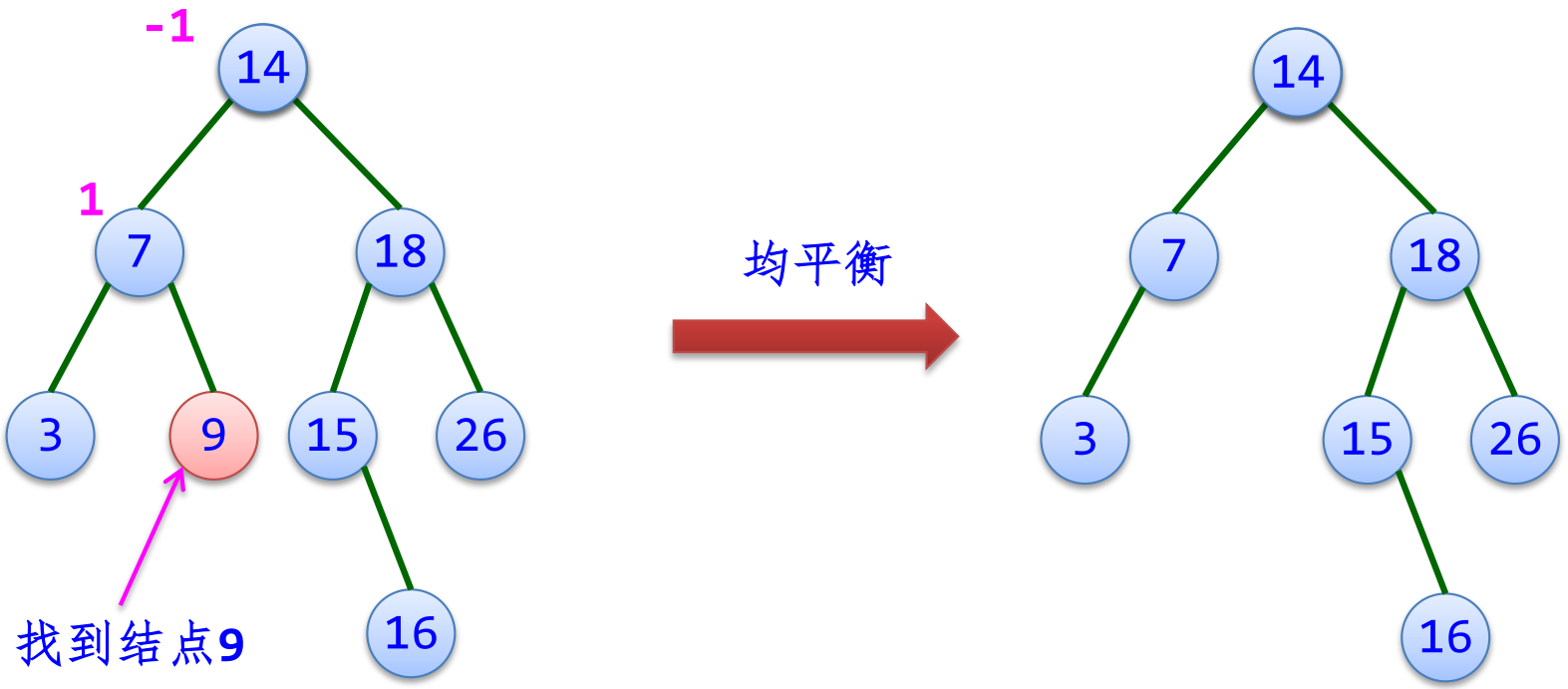


删除结点11



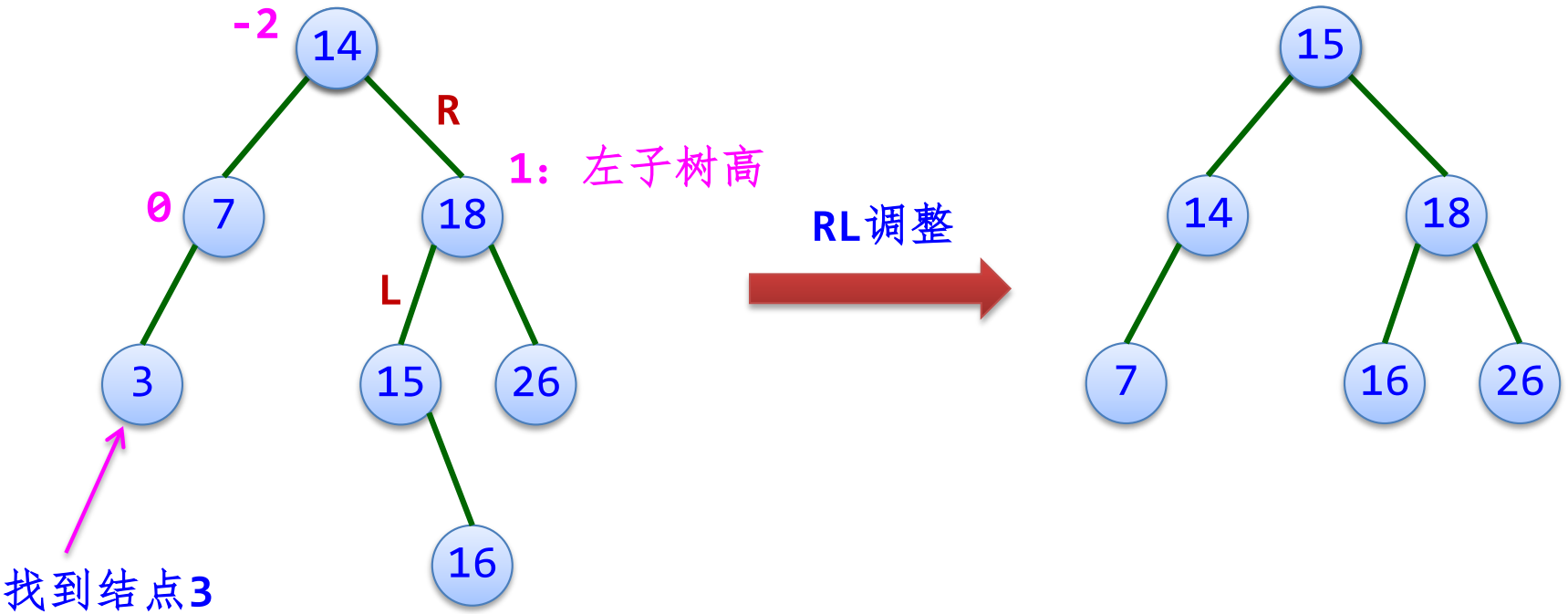
结点1有左右孩子
右子树高，找到右孩子的最左小结点q

删除结点9



直接删除9
沿着根结点方向求平衡因子

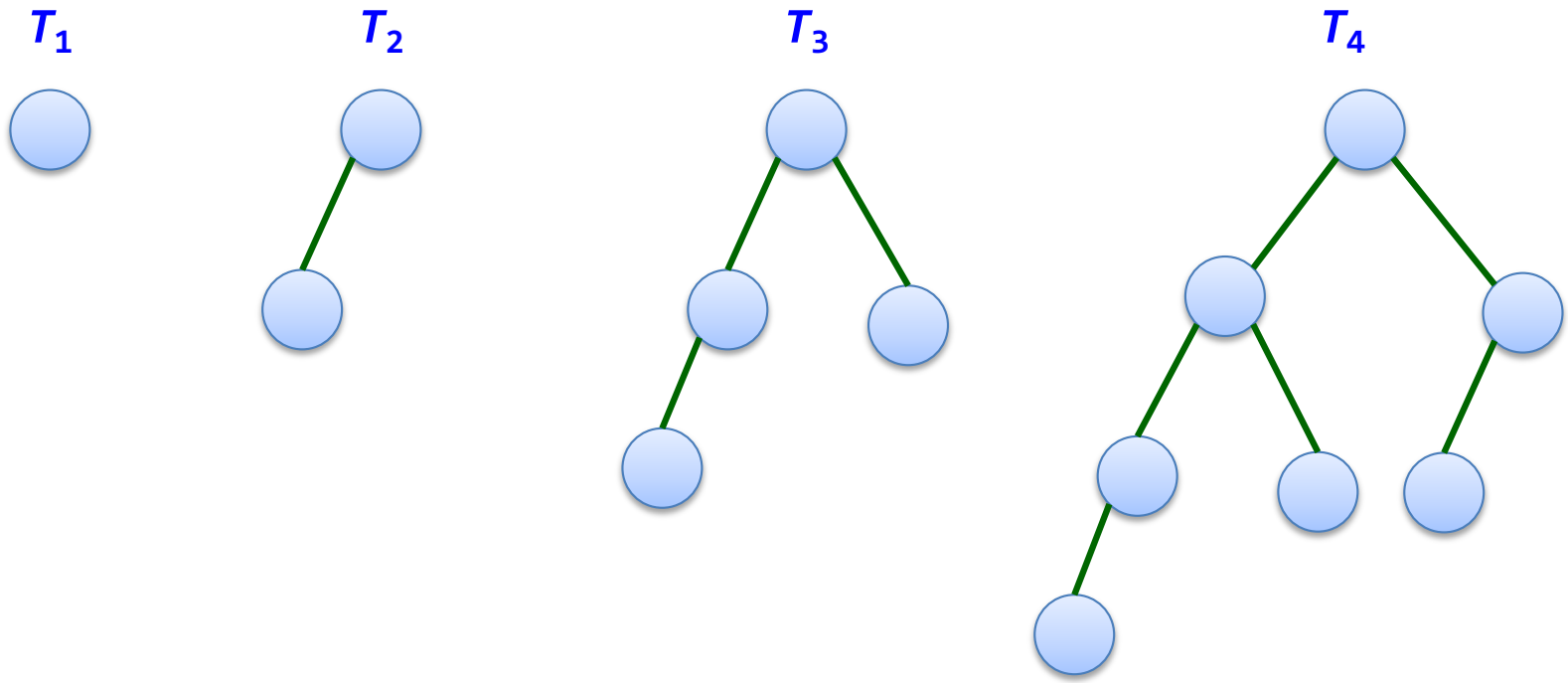
删除结点3



直接删除3
沿着根结点方向求平衡因子

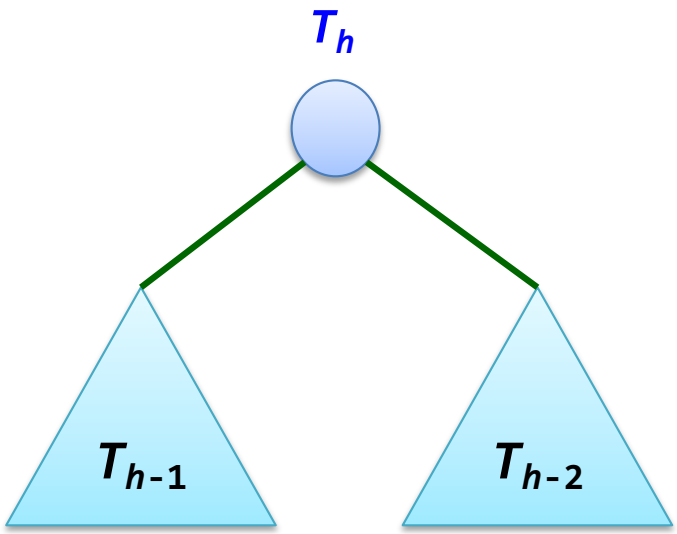
3．AVL树的查找

构造一系列的AVL树T_1、T_2、T_3、…，其中，T_h（h=1、2、3、…）是高度为h且结点数尽可能少的AVL树（总是让左子树较高）。



高度固定结点个数n最少的平衡二叉树

构造T_h，先分别构造T_{h-1}和T_{h-2}，使T_h以T_{h-1}和T_{h-2}作为其根结点的左、右子树。



设$N(h)$（高度h是正整数）为T_h的结点数：

$$N(1)=1$$
$$N(2)=2$$
$$N(h)=N(h-1)+N(h-2)+1$$

Fibonacci数的关系：

$F(1)=1$

$F(2)=1$

$F(h)=F(h-1)+F(h-2)$

通过检查两个序列的前几项就可发现两者之间的对应关系：

$N(h)=F(h+2)-1$

$$F(h)=\frac{1}{\sqrt{5}}\varphi^{h},\ \varphi=(1+\sqrt{5})/2$$

$$N(h)=\frac{1}{\sqrt{5}}\varphi^{h+2}-1$$

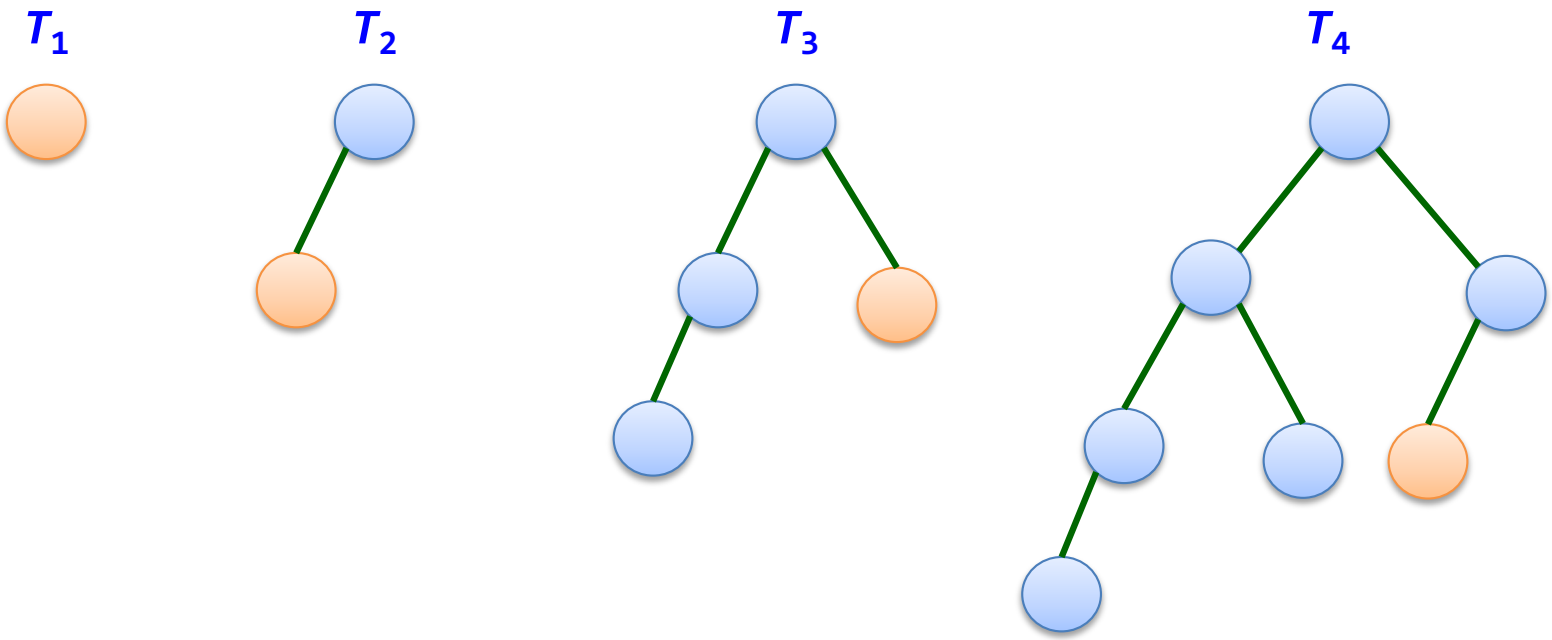
如果树中有***n***个结点，那么树的最大高度为：

$$\log_{\varphi}\left(\sqrt{5}(n+1)\right)-2\approx 1.44\log_2(n+2)=O(\log_2 n)$$

含有*n*个结点的AVL树对应的查找时间复杂度为$O(\log_2 n)$。

另外，设$M(h)$（高度h是正整数）为T_h中最小叶子结点的层次，可以看出有下列关系成立：

$$M(1)=1$$
$$M(2)=2$$
$$M(h)=\min(M(h-1), M(h-2))+1$$

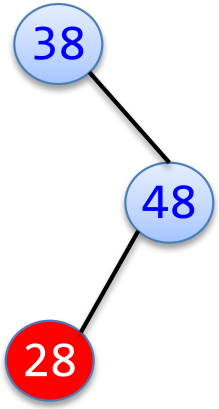


【例】在含有15个结点的AVL树中查找关键字为28的结点，以下哪些是可能的关键字比较序列？

A.30，36　　B.38，48，28

C.48，18，38，28　　D.60，30，50，40，38，36

画出4个查找序列对应的查找树都是二叉排序树，B序列对应的查找树不是一棵二叉排序树，排除选项B。



k=28

A.30, 36

B.38, 48, 28

C.48, 18, 38, 28

D.60, 30, 50, 40, 38, 36

- 设N_h表示高度为*h*的AVL树中含有的最少结点数，按照前面*N*(*h*)的公式求得： *N*(3)=4，*N*(4)=7，*N*(5)=12，*N*(6)=20>15
- 也就是说，15个结点的AVL树的最大高度为5。
- 而D序列比较了6次还查找失败，显然是错误的，排除选项D。

k=28

A.30, 36　　B.38, 48, 28

C.48, 18, 38, 28　　D.60, 30, 50, 40, 38, 36

- *n*=15的AVL树的最小高度为4，即$\log_2(n+1)=4$，此时为一棵满二叉树。
- 即*h*只能是4和5。
- *h*=4时为一棵满二叉树，所有叶子结点的层次为4，此时失败的查找需要4次比较，所以选项A不可能

k=28

A.30, 36　　B.38, 48, 28

C.48, 18, 38, 28　　D.60, 30, 50, 40, 38, 36

- *h*=5时，加上外部结点后树高为6，此时将外部结点看成叶子结点，一定也是一棵AVL树。
- 设M(*h*)表示高度为*h*的AVL树中最小叶子结点的层次，按照前面M(*h*)的公式求得M(3)=2，M(4)=3，M(5)=3，M(6)=4。这样说明15个结点的AVL树中最小外部结点层次为4，查找失败至少需要3次比较，排除选项A。

答：C

补充：红黑树

1、什么是红黑树

红黑树是一棵二叉排序树，每个结点有一个表示颜色的标志，增加外部结点（通常用nil表示），同时满足以下性质：

① 每个结点的颜色为红色或者黑色。

② 根结点的颜色为黑色。

③ 所有外部结点（空节点）的颜色为黑色。

④ 如果一个结点是红色，则它的所有孩子结点为黑色。

⑤ 对于每个结点，从该结点出发的所有路径上包含相同个数的黑色结点。

路径特指从一个结点到其子孙结点中某个外部结点的路径。

补充：红黑树

1、什么是红黑树

着色性质

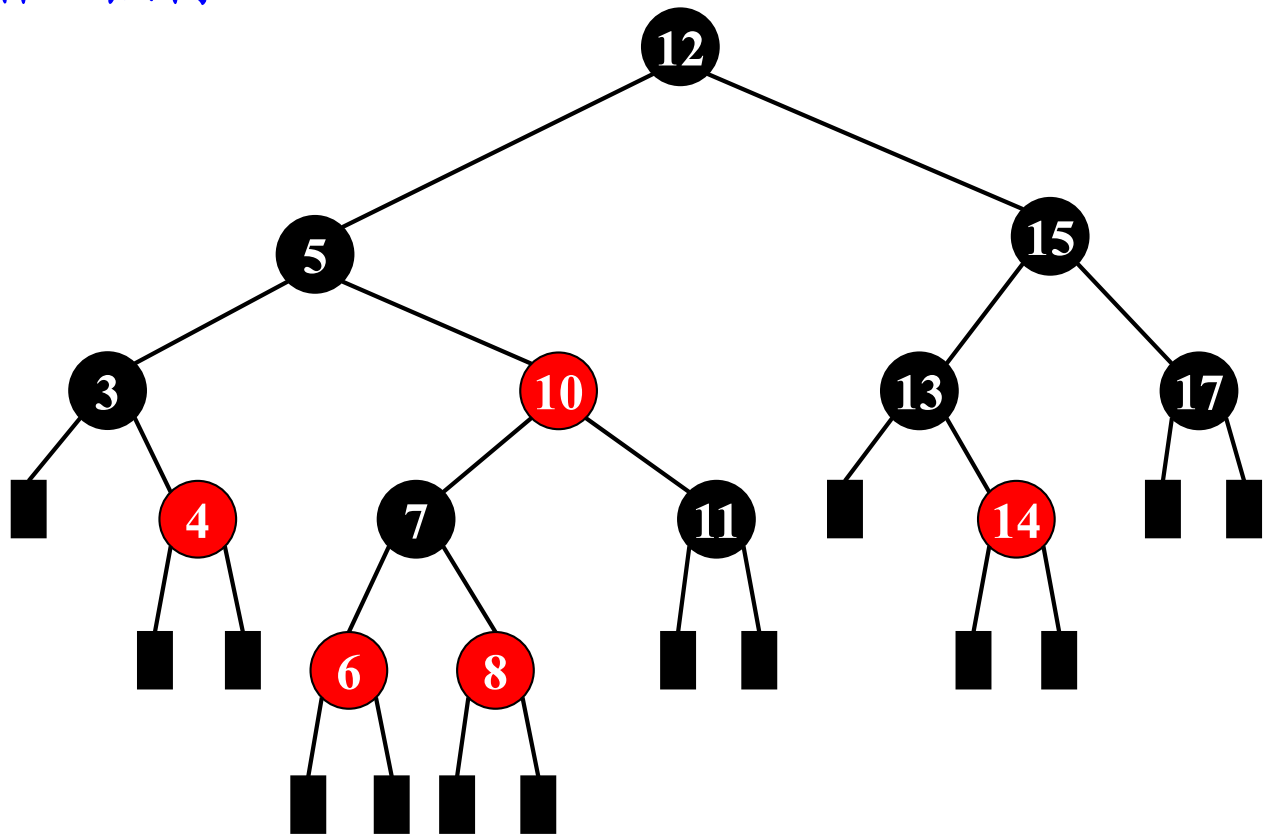
二叉查找树中每个结点着色为红黑两色中的一种颜色；根结点为黑色；任一红结点的孩子只能为黑色。

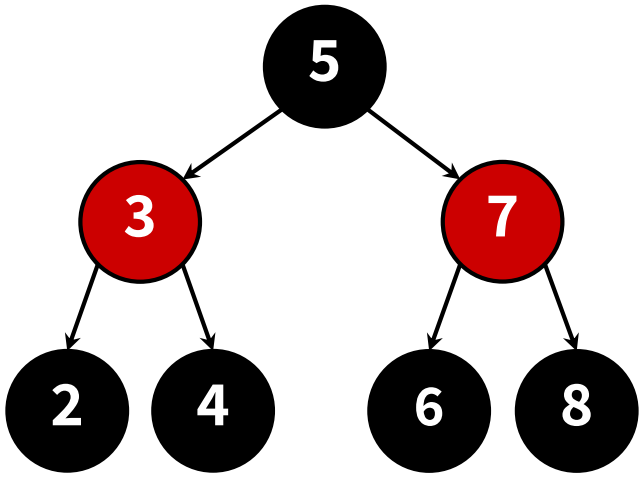
黑高度相等性质

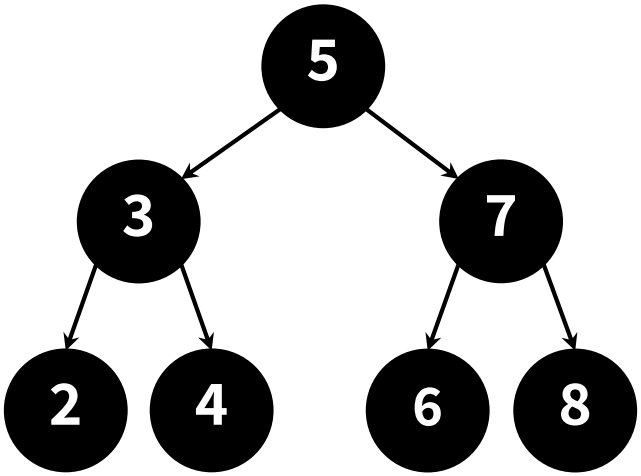
每个结点x到其所有子孙叶结点的路径中所包含的黑结点个数（不包含结点x）必须相等，并称该个数为结点x的黑高度，记为bh(x)，规定空结点的黑高度为0，则叶子结点的黑高度是1，根结点的黑高度为红黑树的黑高度。

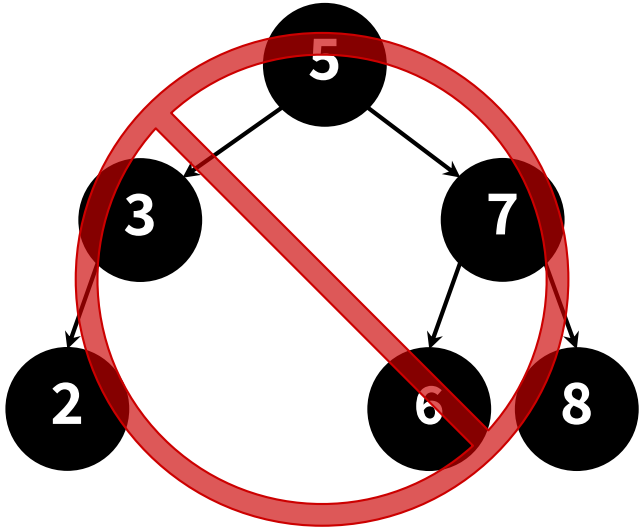
红黑树是一颗自平衡二叉查找树，在插入删除操作时通过特定操作保持二叉树的平衡，从而获得较高的查找性能。最坏O(logn)时间内完成查找，插入和删除。

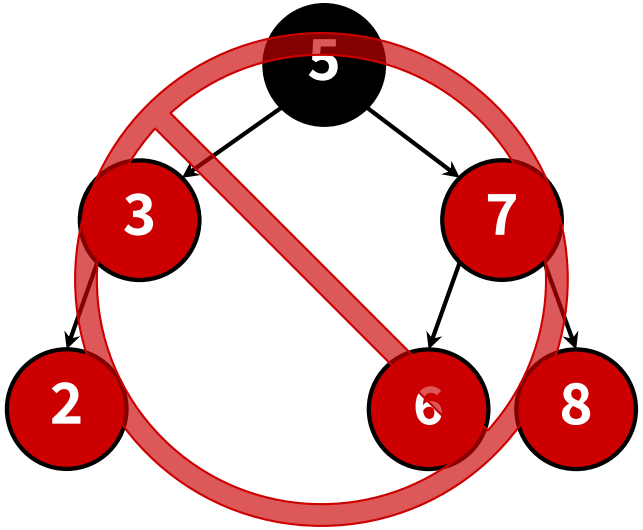
一棵红黑树




5
3
7
2
4
6
8

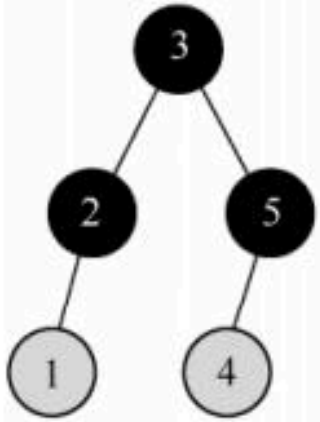

5
3
7
2
4
6
8


5
3
7
2
6
8

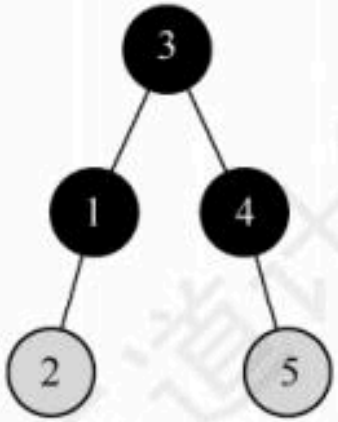

5
3
7
2
6
8

19. 下列四个选项中，满足红黑树定义的是（ ）。

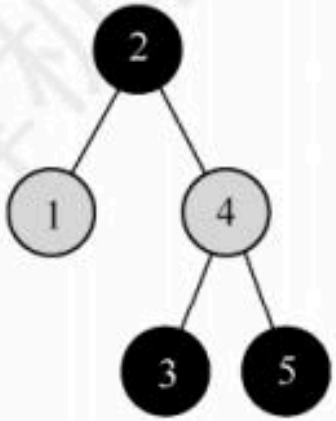
A.



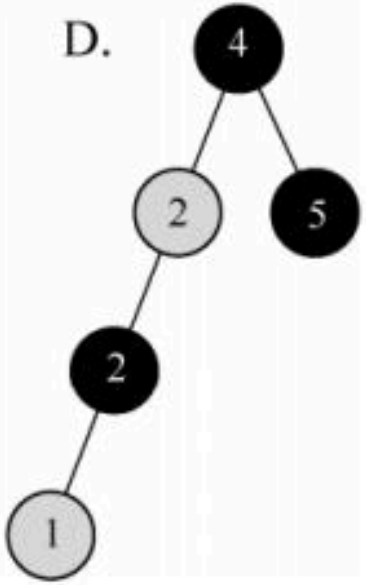
B.



C.



D.

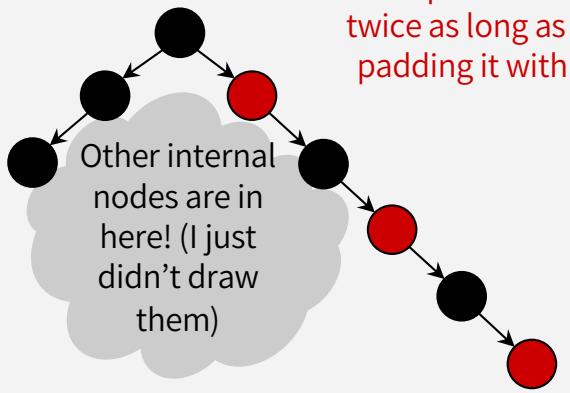


WHAT'S THE POINT OF THESE RULES?

1. Every node is either **red** or **black**
2. The root is a **black** node
3. No **red** node has a **red** child
4. Every root-NIL path has the same number of **black** nodes on them

Intuitively, these rules are a *proxy* for balance:
The **black** nodes are ~balanced across the tree.
And the **red** nodes might elongate paths but not by much!

Rules 3 & 4 guarantee that one path can be at most twice as long as another by padding it with red nodes



THEOREM: Any Red-Black Tree with n nodes has height O(log n)

红黑树是一种弱平衡二叉树，查找性能为$O(\log_2 n)$

第4条性质（如果一个结点是红色，则它的所有孩子结点为黑色） ⇨
任何一条路径上不能有两个相邻的红色结点，这样最短的可能路径均由黑色结点构成，最长的可能路径由交替的红色和黑色结点构成。

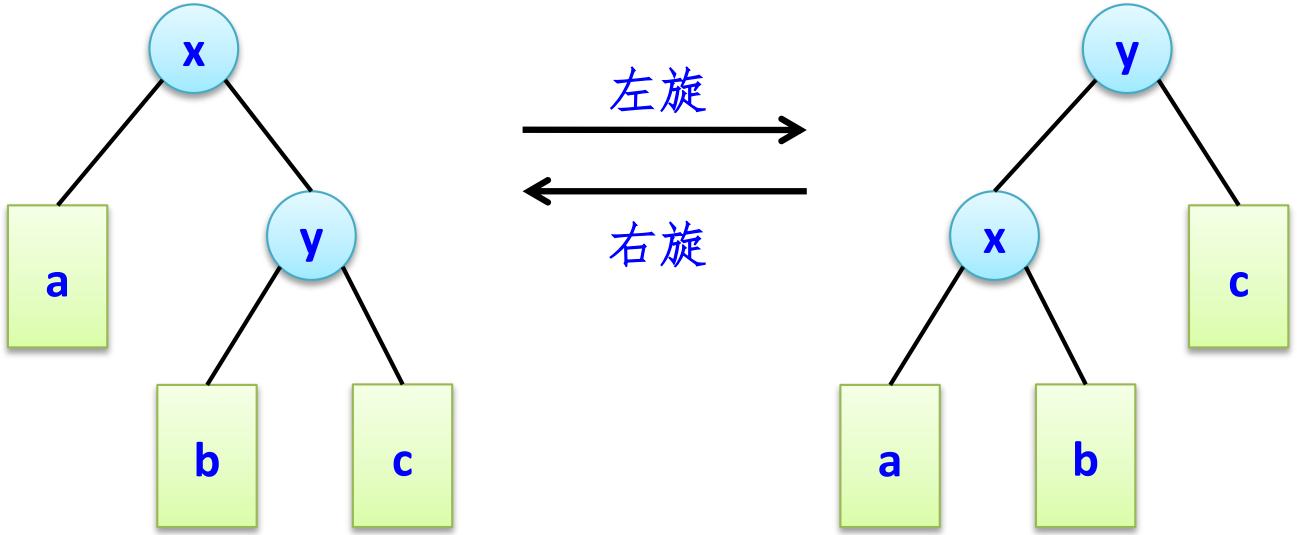
+

第5条性质（从每个结点出发的所有路径上包含相同个数的黑色结点）
⇨ 所有最长的路径都有相同个数的黑色结点。

- 从根结点到叶子结点的最长路径的长度不大于最短路径长度的两倍。
- 含n个结点的红黑树的高度至多为$2\log_2(n+1)$，因此红黑树查找算法的平均时间复杂度为$O(\log_2 n)$。

2、旋转操作

自学



实际上AVL树中的各种调整可以通过上述旋转操作实现：LL型调整就是对结点A做右旋操作，RR型调整就是对结点A做左旋操作，LR型调整就是先对结点B做左旋操作再对结点A做右旋操作，而RL型调整就是先对结点B做右旋操作再对结点A做左旋操作。

3、红黑树中插入新结点

插入过程：

（1）红黑树是一棵二叉排序树，按照二叉排序树插入结点的方法找到新结点插入的位置。

提问：红黑树插入操作的时候，插入位置已经找到，把插入结点放到正确的位置就可以啦，请问插入结点是应该是什么颜色呢？为什么？

答：插入红色结点。因为红色在父结点（如果存在）为黑色结点时，红黑树的黑色平衡没被破坏，不需要做自平衡操作。但如果插入结点是黑色，那么插入位置所在的子树黑色结点总是多1，必须做自平衡。

注意：插入位置始终是叶子结点（查找失败的位置）

3、红黑树中插入新结点

插入过程：

（1）红黑树是一棵二叉排序树，按照二叉排序树插入结点的方法找到新结点插入的位置。

（2）将新结点的颜色设置为红色后插入到指定位置（插入红色结点不会破坏红黑树的第4条性质）。以新插入结点作为当前结点N执行步骤（3）。

（3）对当前结点N进行判断和调整。

- 若结点N为整棵树的根结点，只需要将结点N的颜色改为黑色即可。
- 若结点N的双亲结点P的颜色为黑色，此时仍然满足红黑树性质，不需要做任何调整。
- 若结点N的双亲结点P的颜色为红色，由于结点N和P都是红色，破坏了红黑树的第4条性质，此时需要调整，假设其祖父为结点G，祖父结点的另一个孩子即叔叔为结点U，将调整分为3种情况。

3、红黑树中插入新结点

插入过程：

（1）红黑树是一棵二叉排序树，按照二叉排序树插入结点的方法找到新结点插入的位置。

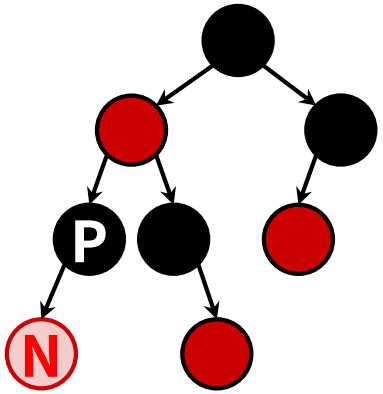
（2）将新结点的颜色设置为红色后插入到指定位置（插入红色结点不会破坏红黑树的第4条性质）。以新插入结点作为当前结点N执行步骤（3）。

（3）对当前结点N进行判断和调整。

3、红黑树中插入新结点

（3）当前对结点N进行判断和调整。

- 若结点N为整棵树的根结点，只需要将结点N的颜色改为黑色即可。
- 若结点N的双亲结点P的颜色为黑色，此时仍然满足红黑树性质，不需要做任何调整。

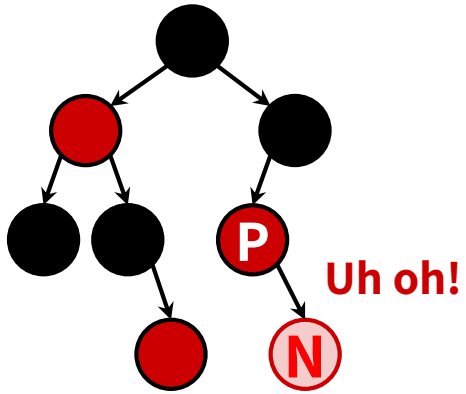


All good ☺

3、红黑树中插入新结点

（3）当前对结点N进行判断和调整。

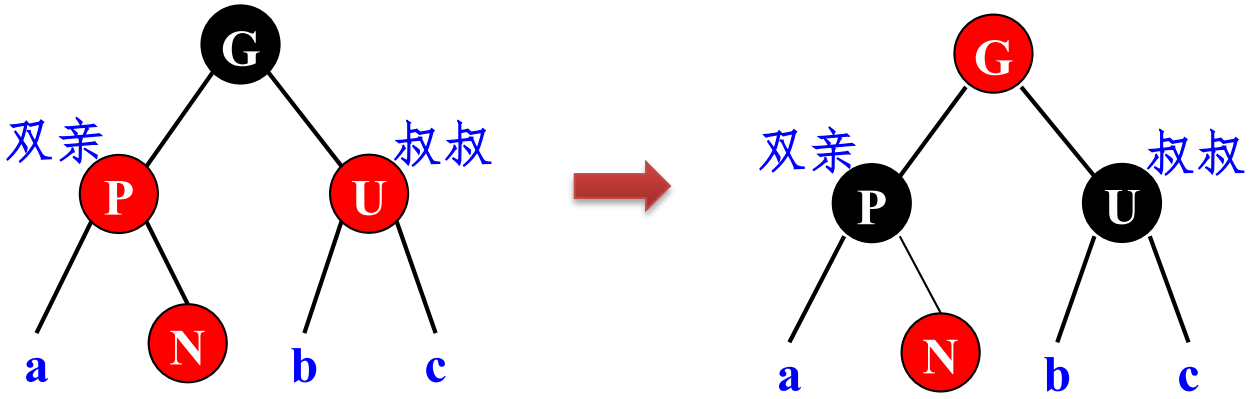
- 若结点N为整棵树的根结点，只需要将结点N的颜色改为黑色即可。
- 若结点N的双亲结点P的颜色为黑色，此时仍然满足红黑树性质，不需要做任何调整。
- 若结点N的双亲结点P的颜色为红色，由于结点N和P都是红色，破坏了红黑树的第4条性质，此时需要调整。
  - 假设其祖父为结点G，祖父结点的另一个孩子即叔叔为结点U，将调整分为3种情况。



① 当前结点N的叔叔结点U是红色。

调整方式是采用改变结点颜色的操作：

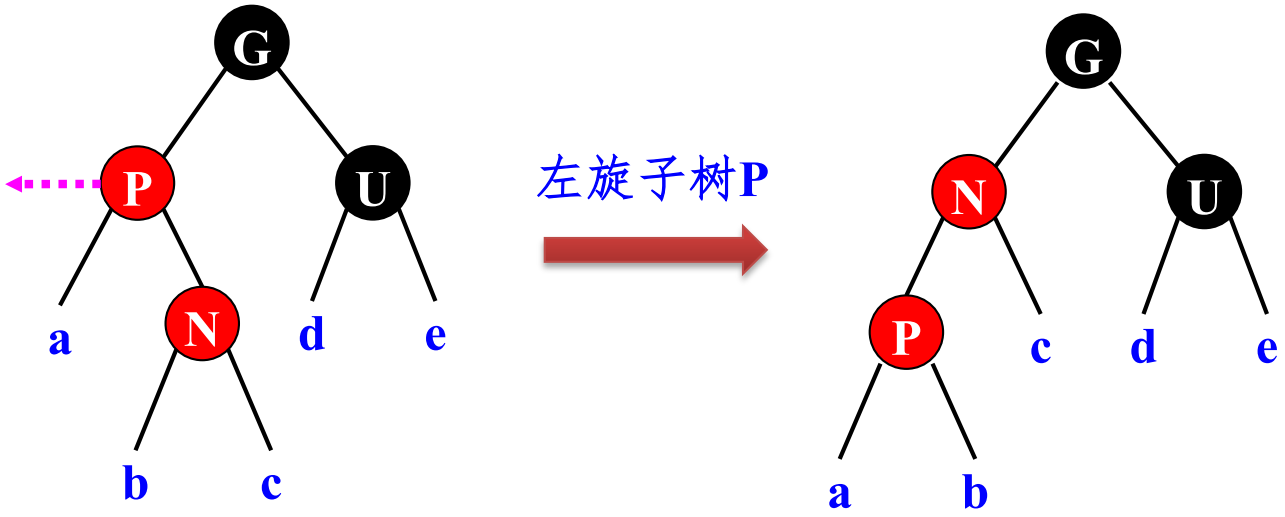
- 将双亲结点P和叔叔结点U的颜色改为黑色，祖父结点G的颜色改为红色（用来维护第4条性质）。
- 以祖父结点G为当前结点继续进行调整。



② 当前结点N的叔叔结点U是黑色，并且结点N是右孩子。

调整方式：

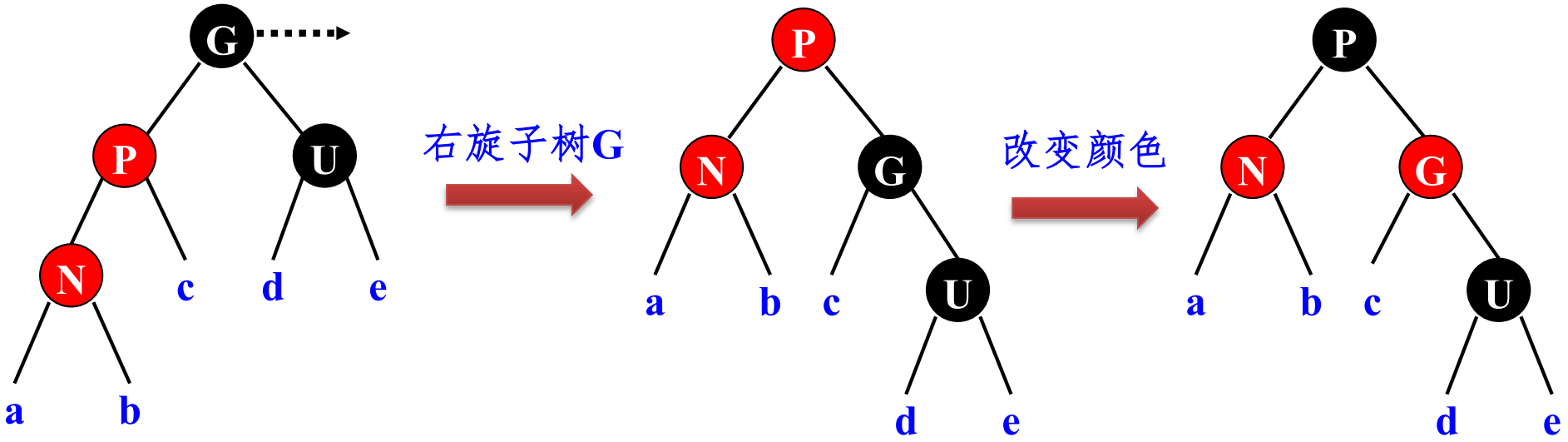
- 左旋以双亲结点P为根的子树。
- 再以结点P为当前结点按情况③继续进行调整。

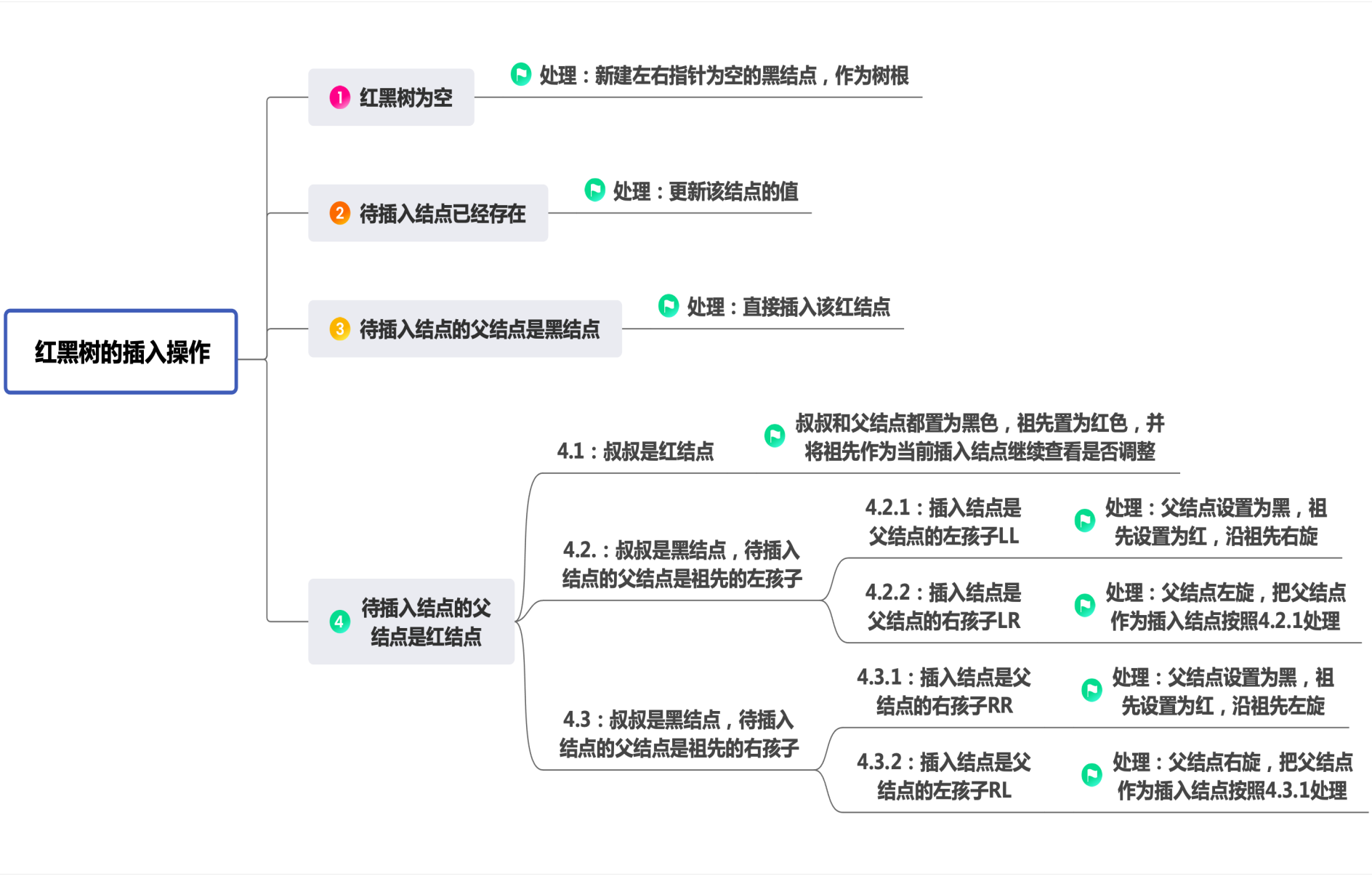


③ 当前结点N的叔叔结点U是黑色，并且结点N是左孩子。

调整方式：

- 先右旋以祖父结点G为根的子树。
- 然后交换以前的双亲结点P和祖父结点G的颜色。




红黑树的插入操作
1 红黑树为空
处理：新建左右指针为空的黑结点，作为树根
2 待插入结点已经存在
处理：更新该结点的值
3 待插入结点的父结点是黑结点
处理：直接插入该红结点
4 待插入结点的父结点是红结点
4.1：叔叔是红结点
叔叔和父结点都置为黑色，祖先置为红色，并将祖先作为当前插入结点继续查看是否调整
4.2.：叔叔是黑结点，待插入结点的父结点是祖先的左孩子
4.2.1：插入结点是父结点的左孩子LL
处理：父结点设置为黑，祖先设置为红，沿祖先右旋
4.2.2：插入结点是父结点的右孩子LR
处理：父结点左旋，把父结点作为插入结点按照4.2.1处理
4.3：叔叔是黑结点，待插入结点的父结点是祖先的右孩子
4.3.1：插入结点是父结点的右孩子RR
处理：父结点设置为黑，祖先设置为红，沿祖先左旋
4.3.2：插入结点是父结点的左孩子RL
处理：父结点右旋，把父结点作为插入结点按照4.3.1处理

- 上述插入过程的时间复杂度为$O(\log_2 n)$，而且插入中的旋转操作不超过两次。
- 红黑树中删除结点的操作也是先按二叉排序树删除方法删除对应的结点，然后进行判断和调整使其满足红黑树的性质，这里不再详述。同样删除过程的时间复杂度为$O(\log_2 n)$。

讨论

提问：请观察红黑树的红结点多还是黑结点多？最坏情况是怎样的？因此插入红结点除了因为父结点是黑色不用调整，还有什么优点？

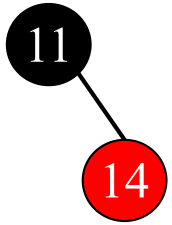
答：通常黑结点多。最坏情况是根结点黑色，然后每一个结点的一个分支全部是黑结点，一个分支是一层红结点，一层黑结点，使得黑高度相同，但高度差1倍。

因为大部分情况下黑结点多，插入红结点不用调整，使得插入操作比AVL树减少大量旋转，功能更优。

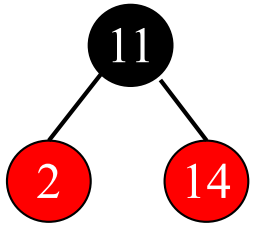
示例

练习题9. 有一个关键字序列为（11，14，2，7，1，15，5，8，4），从一棵空红黑树开始依次插入各个关键字创建一棵红黑树，给出创建红黑树的这个过程。

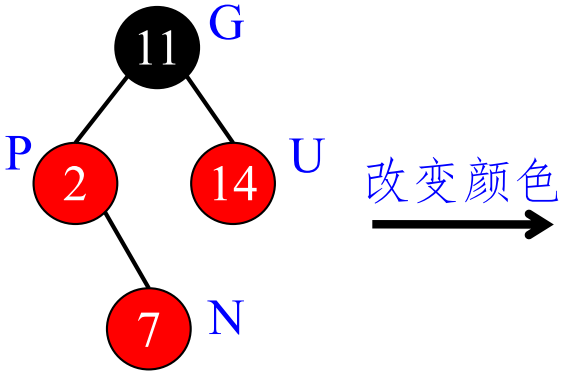
(a)插入11

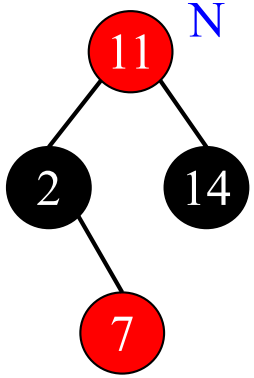


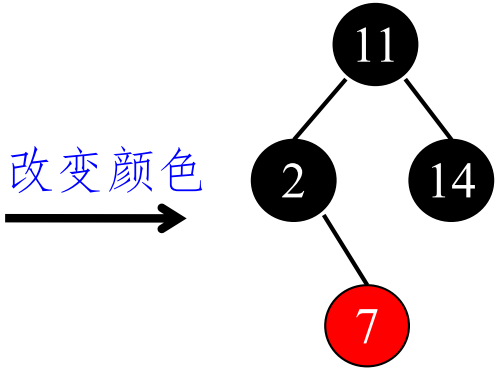
(b)插入14



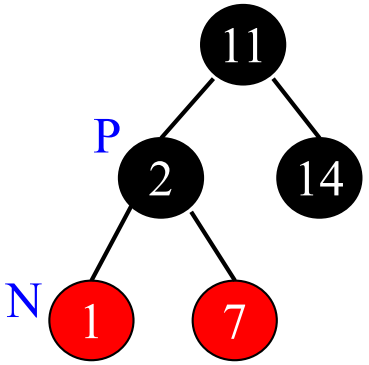
(c)插入2



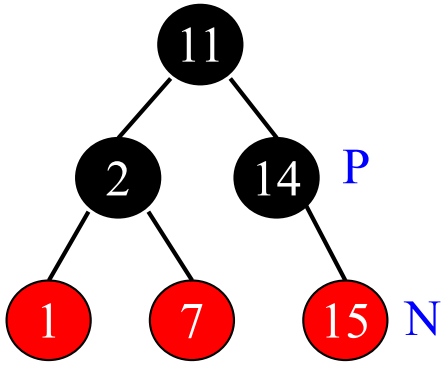




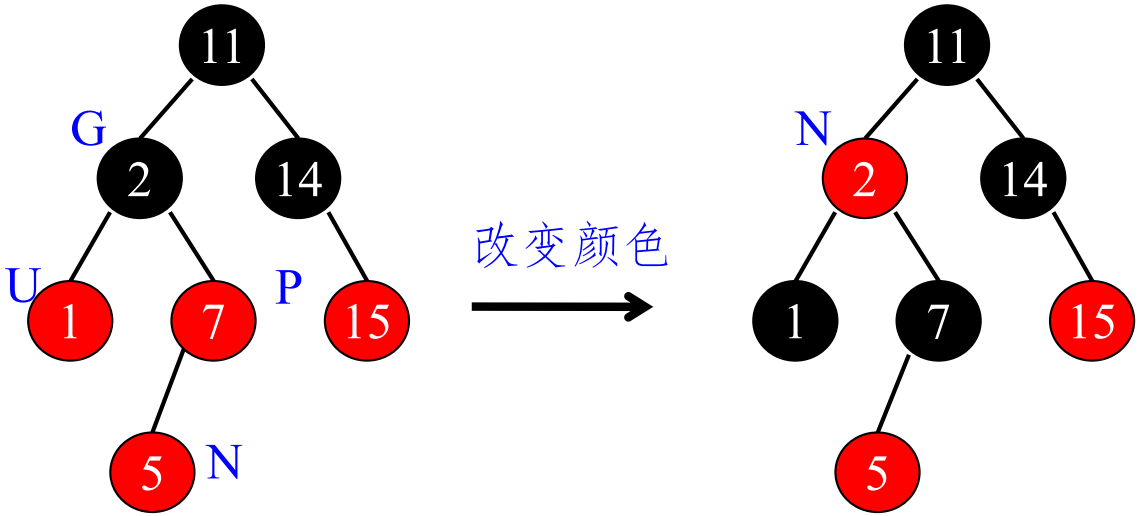
(d)插入7



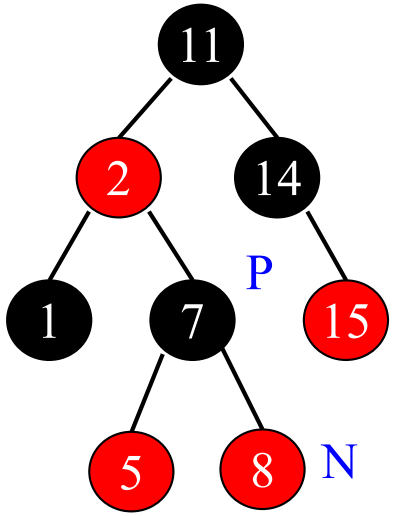
(e)插入1



(f)插入15

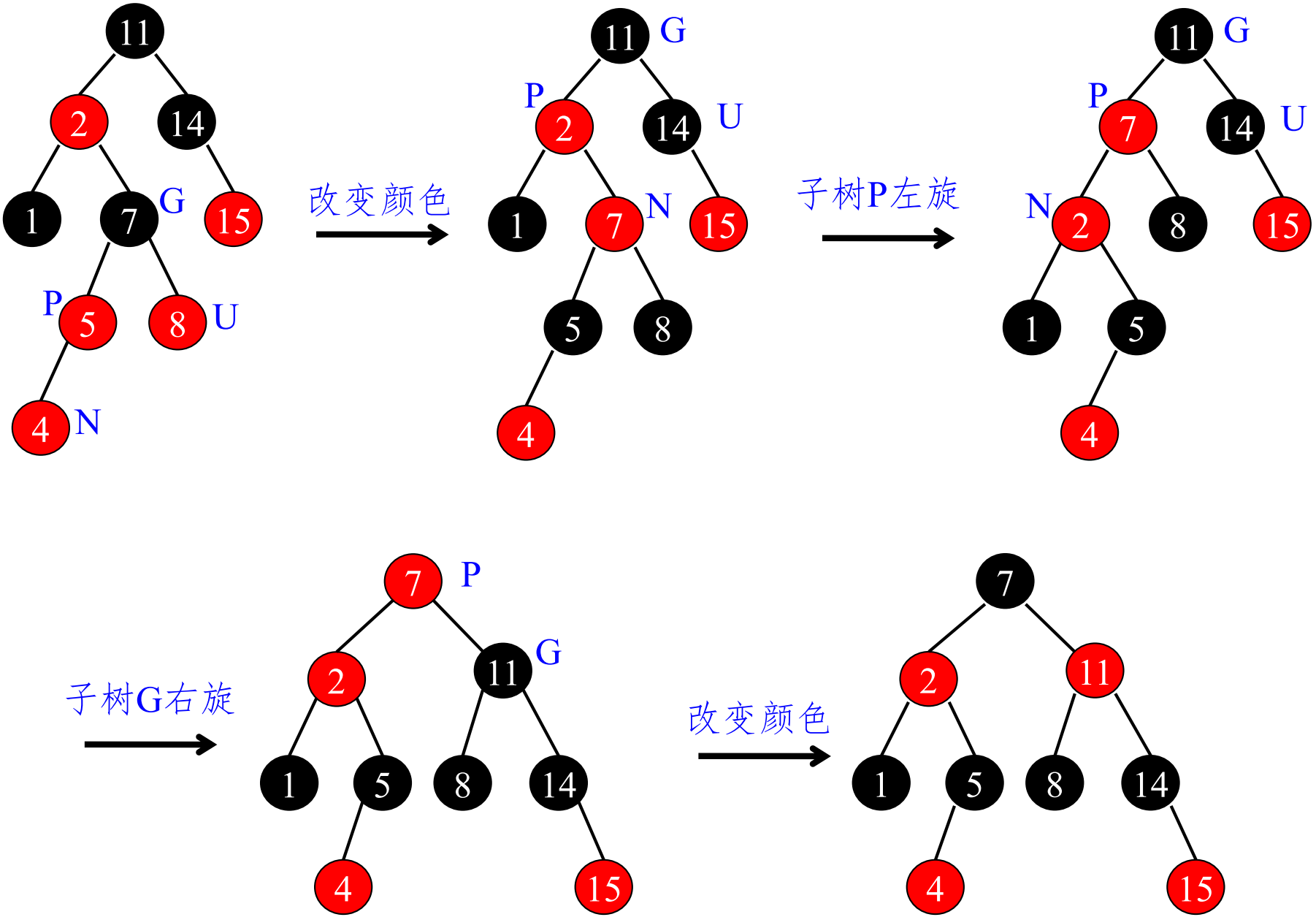


(g)插入5



(h)插入8

插入关键字4

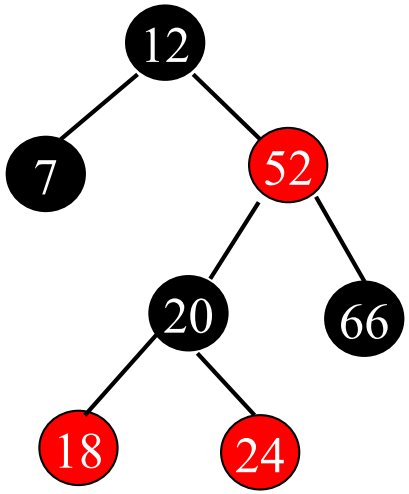

11
2
14
1
7
G
15
P
5
8
U
4
N
改变颜色
11
G
P
2
14
U
1
7
N
15
5
8
4
子树P左旋
11
G
P
7
14
U
N
2
8
15
1
5
4
子树G右旋
7
P
2
11
G
1
5
8
14
4
15
改变颜色
7
2
11
1
5
8
14
4
15

练习

练习题9. 有一个关键字序列为（12，7, 24, 66, 52, 18, 20），从一棵空红黑树开始依次插入各个关键字创建一棵红黑树，给出创建红黑树的这个过程。

练习

练习题9. 有一个关键字序列为（12，7, 24, 66, 52, 18, 20），从一棵空红黑树开始依次插入各个关键字创建一棵红黑树，给出创建红黑树的这个过程。



9.3.3* STL中的关联容器

自学

- 所谓关联容器就是容器中每个元素有一个key（关键字），通过key来存储和读取元素。
- STL中的关联容器有集合（set）和映射（map）两类，均采用红黑树组织数据。
- 红黑树是一种弱平衡二叉树，在维护平衡的成本上比AVL树低，插入和删除等操作都比较稳定。
- 由于树结构中没有位置的概念，所以关联容器没有提供顺序容器中的[]，front()、push_front()、back()、push_back()以及pop_back()操作。

1. set（集合容器）/multiset（多重集容器）

- set和multiset都是集合类模板，其元素值称为关键字。set中元素的关键字是唯一的，multiset中元素的关键字可以不唯一，而且默认情况下会对元素按关键字自动进行升序排列。
- 查找速度比较快（时间复杂度为$O(\log_2 n)$），同时支持集合的交、差和并等一些集合上的运算。

set/multiset的主要成员函数

成员函数	说明
empty()	判断容器是否为空
size()	返回容器中实际元素个数
insert(k)	插入元素k
erase(k)	从容器删除元素k
erase(it)	从容器删除迭代器it指向的元素
clear()	删除所有元素
count(k)	返回容器中关键字k出现的次数
find(k)	如果容器中存在关键字为k的元素，返回该元素的迭代器，否则返回end()值
begin()	用于正向迭代，返回容器中第一个元素的位置
end()	用于正向迭代，返回容器中最后一个元素后面一个位置
rbegin()	用于反向迭代，返回容器中最后一个元素的位置
rend()	用于反向迭代，返回容器中第一个元素前面一个位置

- STL为set/multiset提供了通用算法lower_bound(beg，end，k)和upper_bound(beg，end，k)等。
- 前者返回一个迭代器指向第一个关键字大于等于*k*的元素，后者返回一个迭代器指向第一个关键字大于*k*的元素。

```
#include<iostream>
#include<set>
using namespace std;
int main()
{  set<int> s;                          //定义set容器s
   set<int>::iterator it;               //定义set容器迭代器it
   s.insert(1);
   s.insert(3);
   s.insert(2);
   s.insert(2);                         //2重复，不会插入
   printf(" s: ");
   for (it=s.begin();it!=s.end();it++)
      printf("%d ",*it);                //输出:1 2 3
   printf("\n");
```

```
    multiset<int> ms;                       //定义multiset容器ms
    multiset<int>::iterator mit;            //定义multiset容器迭代器mit
    ms.insert(1);
    ms.insert(3);
    ms.insert(2);
    ms.insert(2);                           //重复的2会插入
    printf("ms: ");
    for (mit=ms.begin();mit!=ms.end();mit++)
       printf("%d ",*mit);                  //输出:1 2 2 3
    printf("\n");
    return 0;
}
```

2. map（映射容器）/multimap（多重映射容器）

- map和multimap都是映射类模板。
- 映射是指元素类型为（key，value），其中key为关键字，value是对应的值，可以使用关键字key来访问相应的值value。
- map/multimap中的key和value是一个pair结构类型。

```
struct pair
{   T1 first;           //关键字
    T2 second;          //值
}
```

map/multimap的主要成员函数

成员函数	说明
empty()	判断容器是否为空
size()	返回容器中实际元素个数
map[k]	返回关键字为k的元素的引用，如果不存在这样的关键字，则以k作为关键字插入一个元素（不适合multimap）
insert(e)	插入一个元素e并返回该元素的位置
erase(k)	从容器删除元素k
erase(it)	从容器删除迭代器it指向的元素
erase(beg,end)	从容器删除[beg,end)迭代器范围的元素
clear()	删除所有元素
find(k)	在容器中查找关键字为k的元素
count(k)	返回容器中关键字为k的元素个数（map中只有1或者0）
begin()	用于正向迭代，返回容器中第一个元素的位置
end()	用于正向迭代，返回容器中最后一个元素后面一个位置
rbegin()	用于反向迭代，返回容器中最后一个元素的位置
rend()	用于反向迭代，返回容器中第一个元素前面一个位置

- STL为map/multimap提供了通用算法lower_bound(beg，end，k)和upper_bound(beg，end，k)等。
- 前者返回一个迭代器指向第一个关键字大于等于*k*的元素，后者返回一个迭代器指向第一个关键字大于*k*的元素。

1）map的构造函数

map共提供了6个构造函数，有的涉及到内存分配器。最简单的方式就是仅仅给出key和value的类型，例如：

```
map<int,string> stmap;  //int关键字为学号，string值为姓名
```

2）插入元素

向map中插入元素主要有3种方法，分别是用insert函数插入pair数据，用insert函数插入value_type数据，用数组方式插入数据。例如以下3个语句对应3种插入方法：

```
stmap.insert(pair<int, string>(1, "Mary"));
stmap.insert(map<int, string>::value_type (2, "Smith"));
stmap[3] = "John";
```

3）获取一个关键字对应的值

直接使用map[关键字]获取该关键字对应的值。例如：

```
string xm=stmap[4];
```

执行该语句时，只有当stmap中有这个关键字（4）时才会成功返回学号4的姓名，否则会自动插入一个元素，其关键字为4，对应值为string类型的默认值空串。

4）按关键字最小和最大元素

假设stmap容器按关键字递增有序，则：

- stmap.begin()地址为存放最小关键字的结点。
- --stmap.end()地址为存放最大关键字的结点。

5）it所指结点的前驱和后继结点

若迭代器it指向stmap容器中的某个非空结点，则：

- --it指向其前驱结点，++it指向其后继结点。
- 迭代器不能执行±i运算，所以it-1和it+1是错误的。

6）按关键字查找元素

（1）用count函数来判定关键字是否出现，其返回值要么是0，要么是1。其缺点是无法定位关键字对应元素出现位置。例如：

```
if (stmap.count(1)!=0)    //判断stmap中是否存在关键字为1的元素
   cout << "查找成功" << endl;
else
   cout << "查找失败" << endl;
```

（2）用find函数来定位元素出现位置，它返回的一个迭代器，当查找成功时返回元素所在位置的迭代器，查找失败时返回end()函数值。例如：

```
map<int,string>::iterator it;
it=stmap.find(2);
if (it!=stmap.end())          //在stmap查找关键字为2的元素
   cout << "查找成功，其姓名是" << it->second << endl;
else
   cout << "查找失败" << endl;
```

（3）使用lower_bound或者upper_bound函数查找关键字，与有序vector向量的折半查找类似。

7）删除元素

```
map<int,string>::iterator it=stmap.begin();
stmap.erase(it);                    //删除第一个元素
stmap.erase(3);                     //删除关键字为3的元素
```

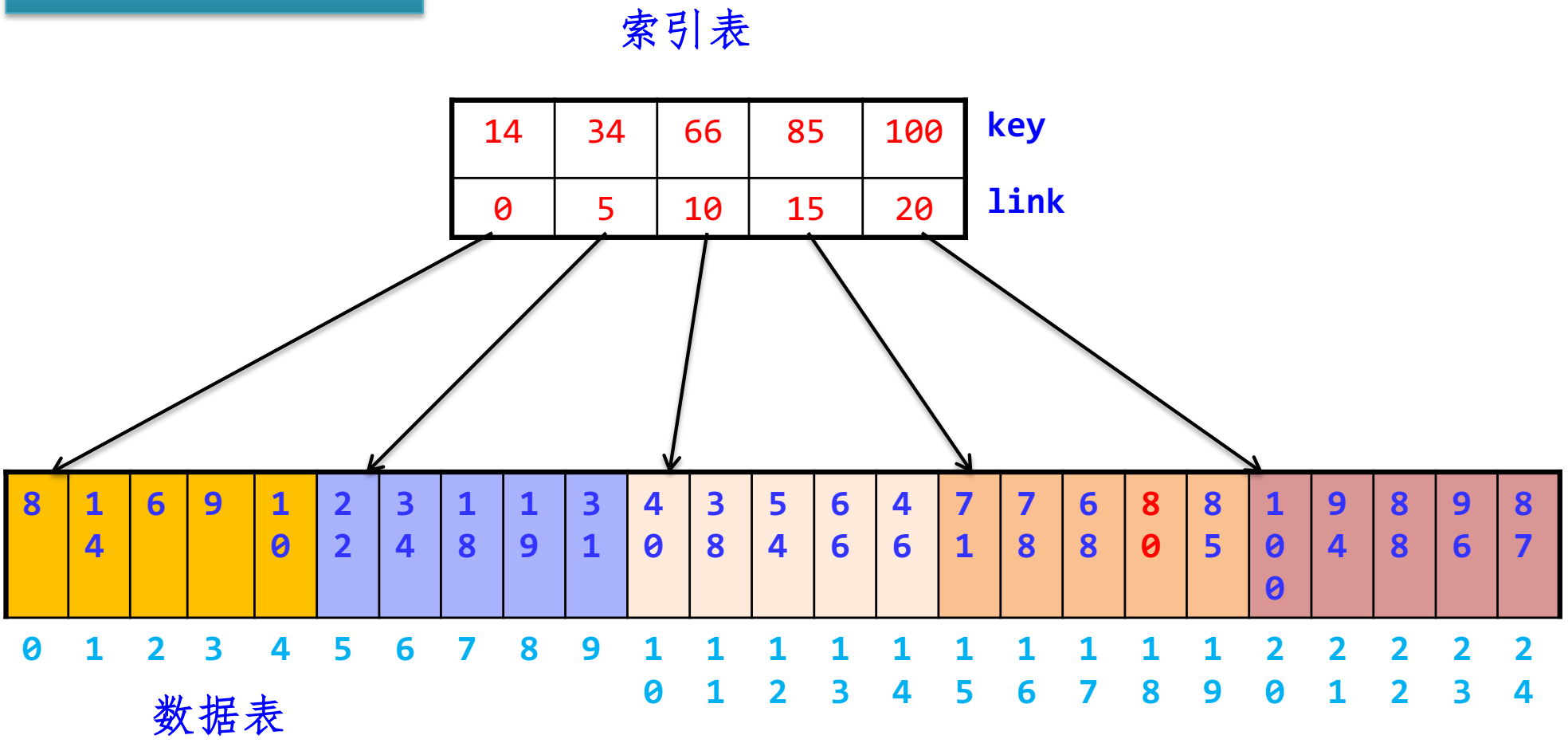
8）修改元素

```
stmap[1]="June";                    //直接将学号1的姓名改为"June"
```

9）map中元素排序

- map默认利用pair的“<”运算符将所有元素按key的升序排列，不能对map使用sort通用算法排序。
- 可以像使用sort通用算法一样定制自己的关系比较函数以确定排序顺序。

回顾 索引结构



分块查找的索引存储结构

线性索引的问题

- 线性索引太大，存储在磁盘块中
  - ✓ 在一次检索过程中可能多次访问几个磁盘块，从而影响检索的效率
  - ✓ 使用二级线性索引
- 更新线性索引
  - ✓ 在数据库中插入或者删除记录时

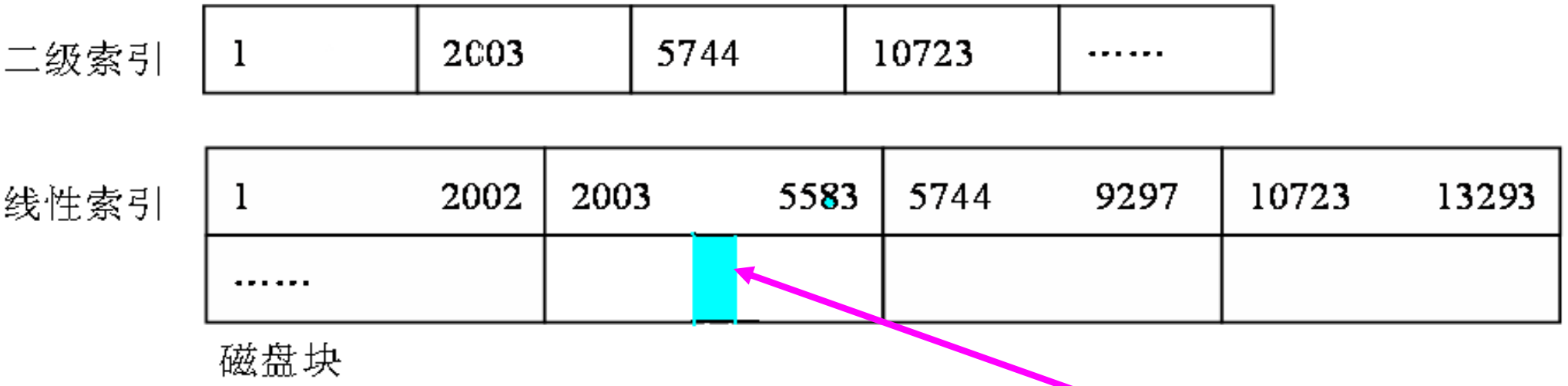
二级线性索引

- 例如，磁盘块的大小是 1024 字节，每对(关键码，指针)索引对需要8个字节
  - 1024 / 8 = 128
  - 每磁盘块可以存储 128 条这样的索引对
- 假设数据文件包含 10000 条记录
  - 稠密一级线性索引中包含 10000 条记录
    - 10000/128 = 78.1
    - 那么一级线性索引占用 79 个磁盘块
  - 相应地，二级线性索引文件中有 79 项索引对
  - 这个二级线性索引文件可以在一个磁盘块

二级线性索引的例子

关键字2555的记录指针

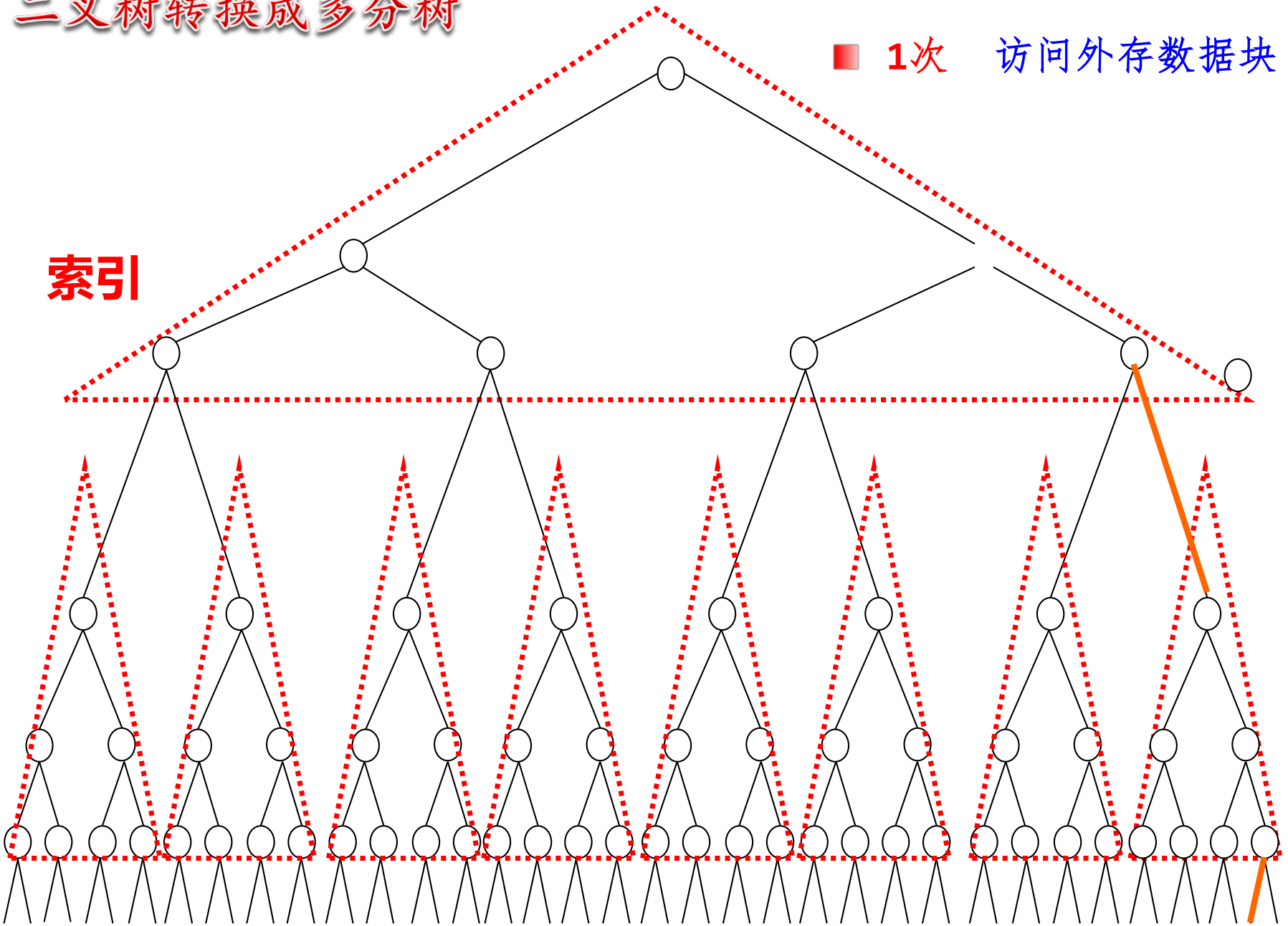
例如：检索关键码为2555的记录



1. 二级线性索引文件读入内存
2. 二分法找关键码的值小于等于2555的最大关键码所在一级索引磁盘块地址—— 关键码为2003的记录
3. 根据记录2003中的地址指针找到其对应的一级线性索引文件的磁盘块，并把该块读入内存
4. 按照二分法对该块进行检索，找到所需要的记录在磁盘上的位置
5. 最后把所需记录读入，完成检索操作

二叉树转换成多分树

- 2次 访问索引块
- 1次 访问外存数据块



9.3.4 B树的特点

假设有10亿条数据：

- 用平衡二叉排序树存储，树高约30层。
- 最坏情况下查找需要约30次内存比较，但如果数据在磁盘上，可能需要30次磁盘I/O（每次I/O只读一个键），这在实际中是无法接受的。
- 用一棵阶数为500的B树存储，树高可能仅为3到4层。最坏情况下查找只需要3-4次磁盘I/O（每次I/O读一个包含500个键的节点），性能差异巨大。

B树的特性

1. 节点大小 = 磁盘块大小
2. 降低树高
3. 局部性原理与预读

9.3.4 B树 vs 平衡二叉树

对比维度	平衡二叉排序树 (AVL/红黑树)	B树 / B+树
设计目标	**最小化内存中的比较次数**	**最小化磁盘I/O次数**
数据组织	**二叉。**	**多路**
树的高度	约为$O(\log_2 N)$	约为O(log_m N)　m为阶数
一次I/O的收益	从磁盘读取一个节点（一个键），仅在内存中排除一半数据。	从磁盘读取一个节点（包含上百个键），能在内存中排除海量数据，效率极高。
适合场景	**内存中的数据**（如标准库的std::map, Java的TreeMap）	**存储在磁盘上的海量数据**
访问模式	随机内存访问快速，适合动态内存操作。	顺序读取磁盘块高效，利用**局部性原理**预读。

9.3.4 B树

一种外查找的数据组织结构（一种平衡的多分树 Balanced Tree）

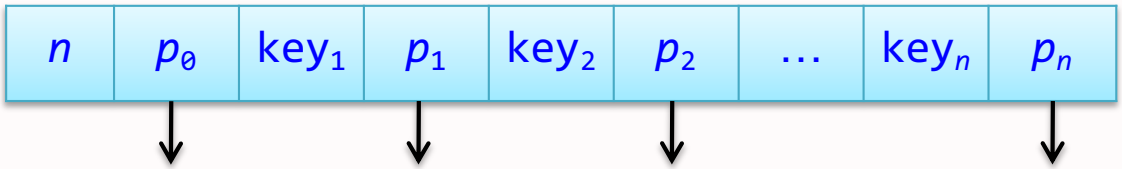
B树中所有结点的最大子树个数称为B树的阶，通常用*m*表示，从查找效率考虑，要求*m*≥3。一棵*m*阶B树或者是一棵空树，或者是满足下列要求的*m*叉树：

（1）树中每个结点至多有*m*棵子树（即至多含有*m*-1个关键字，设Max=*m*-1）。

（2）若根结点不是叶子结点，则根结点至少有两棵子树。

（3）除根结点外，所有结点至少有⌈*m*/2⌉棵子树（即至少含有⌈*m*/2⌉-1个关键字，设Min=⌈*m*/2⌉-1）。

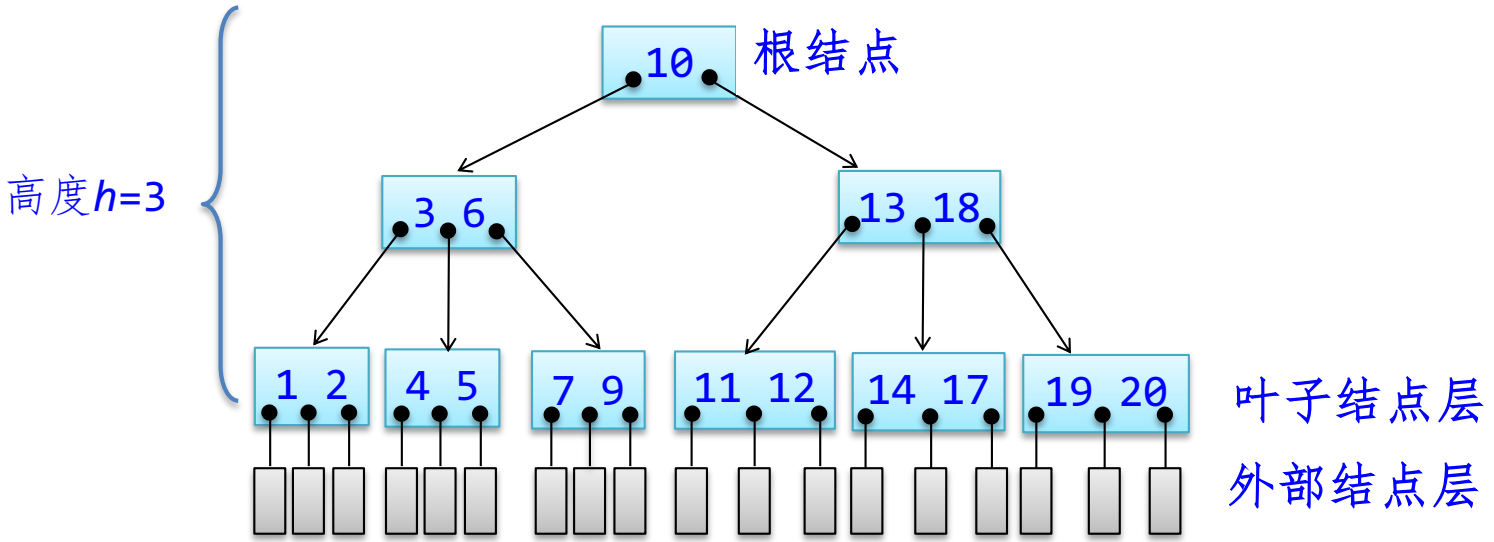
（4）每个结点的结构如下：



$\lceil m/2 \rceil - 1 \leqslant n \leqslant m-1$，并且满足有序性

（5）所有的叶子结点在同一层。

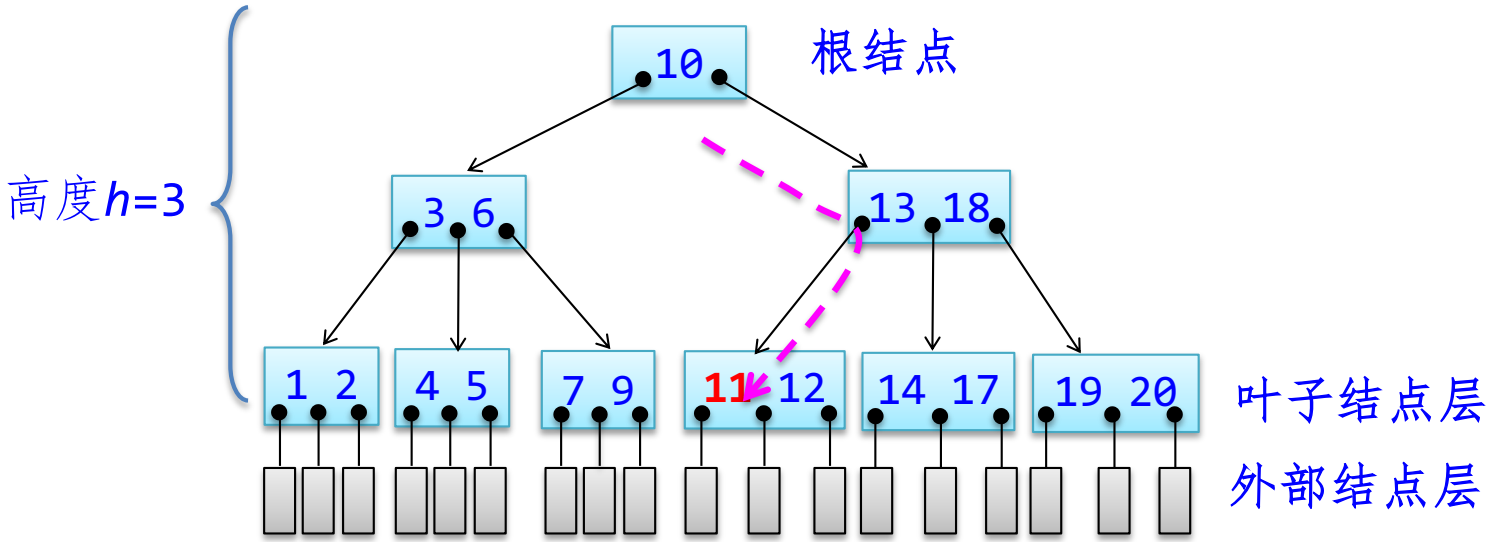
一棵3阶B树



- $m=3$。
- 非叶子结点至少有$\lceil m/2 \rceil=2$个孩子，至多有$m=3$个孩子（这类结点的关键字个数为1～2个）。
- 根结点有两个孩子结点。
- 所有叶子结点都在同一层上。

1．B树的查找

在B树中查找给定关键字的方法类似于二叉排序树上的查找，不同的是在每个结点上确定向下查找的路径不一定是二路的，而是*n*+1路的（*n*为该结点的关键字个数）。



交替的两步过程

1. 把根结点读出来，在根结点所包含的关键码 Key_1，…，Key_j 中查找给定的关键码值
   - 找到则检索成功
2. 否则，确定要查的关键码值是在某个Key_i和 Key_{i+1} 之间，于是取 p_i 所指向的结点继续查找
   如果pi指向外部空结点，表示检索失败

分析查找性能　$\lceil m/2\rceil-1\leqslant n\leqslant m-1$，$\lceil m/2\rceil\leqslant$结点子树数$\leqslant m$

- 假设m阶B树的高度为h（h中不含外部结点层，外部结点层看成是第$h+1$层），访问的结点个数不超过$O(h)$。
- 那么，含有N个关键字的m阶B树可能达到的最大高度h是多少呢？显然在关键字个数固定时，每一层关键字个数越少树的高度越高。

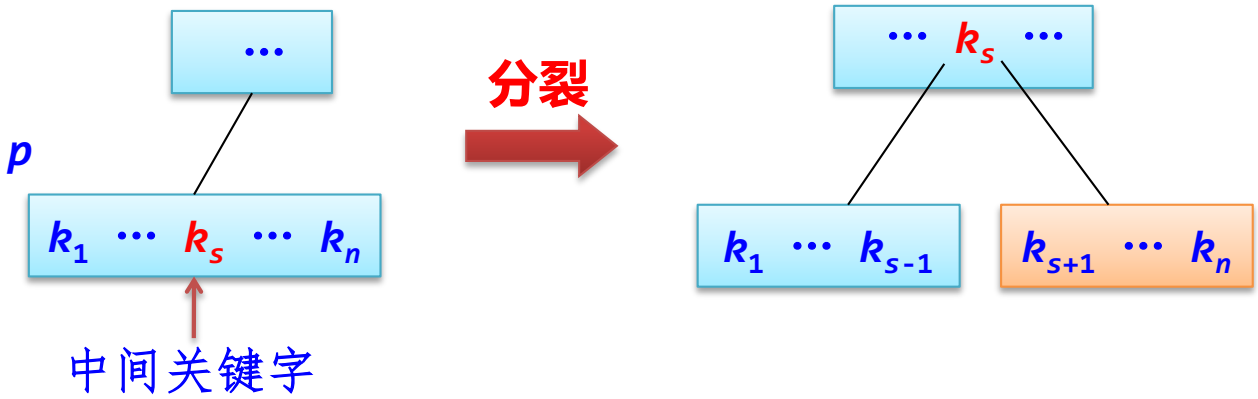
- 第1层最少结点数为1。
- 第2层最少结点数为2。
- 第3层最少结点数为$2\lceil m/2\rceil$。
- 第4层最少结点数为$2\lceil m/2\rceil^2$个。
- …
- 第h层最少结点数为$2\lceil m/2\rceil^{h-2}$个。
- 第$h+1$层（外部结点层）最少结点数为$2\lceil m/2\rceil^{h-1}$个。
- m阶B树中共含有N个关键字，则外部结点必为$N+1$个，即$N+1\geqslant 2\lceil m/2\rceil^{h-1}$，有$h-1\leqslant \log_{\lceil m/2\rceil}(N+1)/2$，则$h\leqslant \log_{\lceil m/2\rceil}(N+1)/2+1=O(\log_m N)$。

2．B树的插入

- 利用前述的查找过程找到关键字*k*的插入结点*p*（注意*m*阶B树的插入结点一定是某个叶子结点）。
- 判断结点*p*是否还有空位置，即其关键字个数*n*是否满足*n*<Max（Max=*m*-1）：

① 若*n*<Max成立，说明结点*p*有空位置，直接把关键字*k*有序插入到结点*p*中（插入关键字*k*后结点*p*的所有关键字仍有序）。

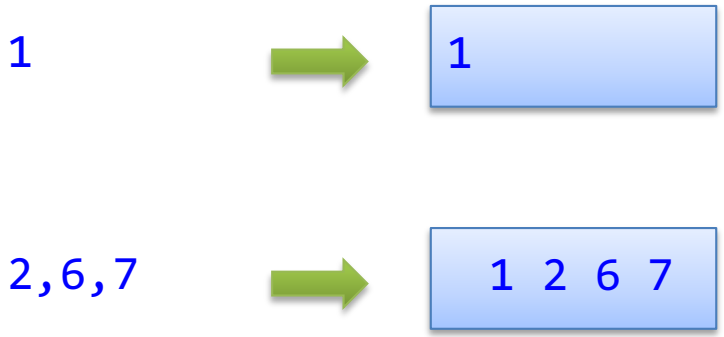
② 若n=Max，说明结点p没有空位置，需要把结点p分裂成两个。

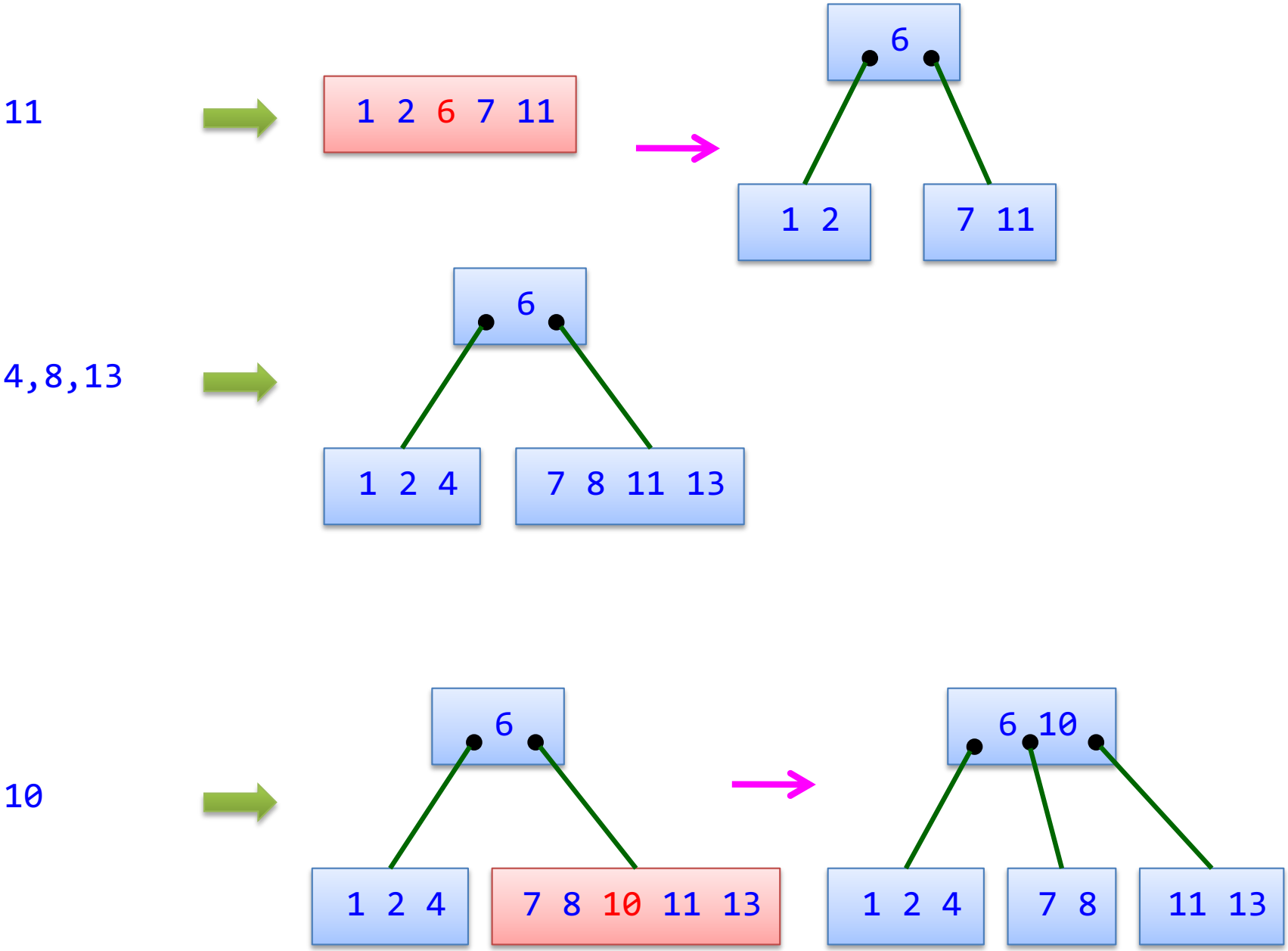


如果此时双亲结点的关键字个数也超过Max，则要再分裂，再往上插，直至这个过程传递到根结点为止。如果根结点也需要分裂，则整个m阶B树增高一层。

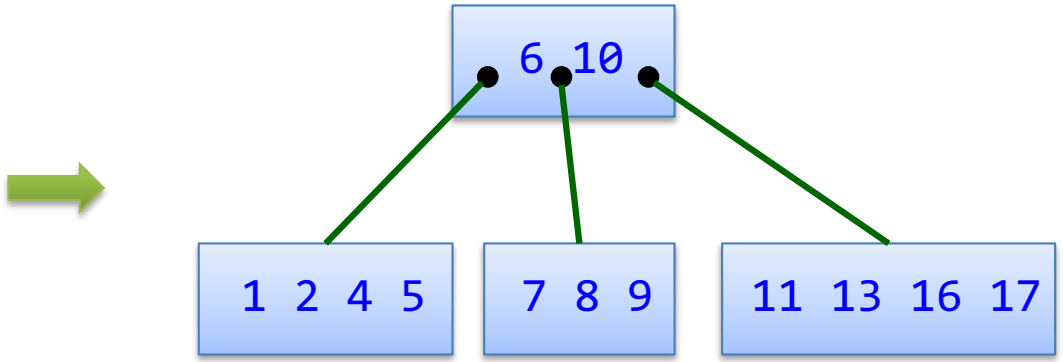
【例9.12】关键字序列为（1，2，6，7，11，4，8，13，10，5，17，9，16，20，3，12，14，18，19，15），创建一棵5阶B树。

这里m=5，结点中最大关键字个数Max=m-1=4。

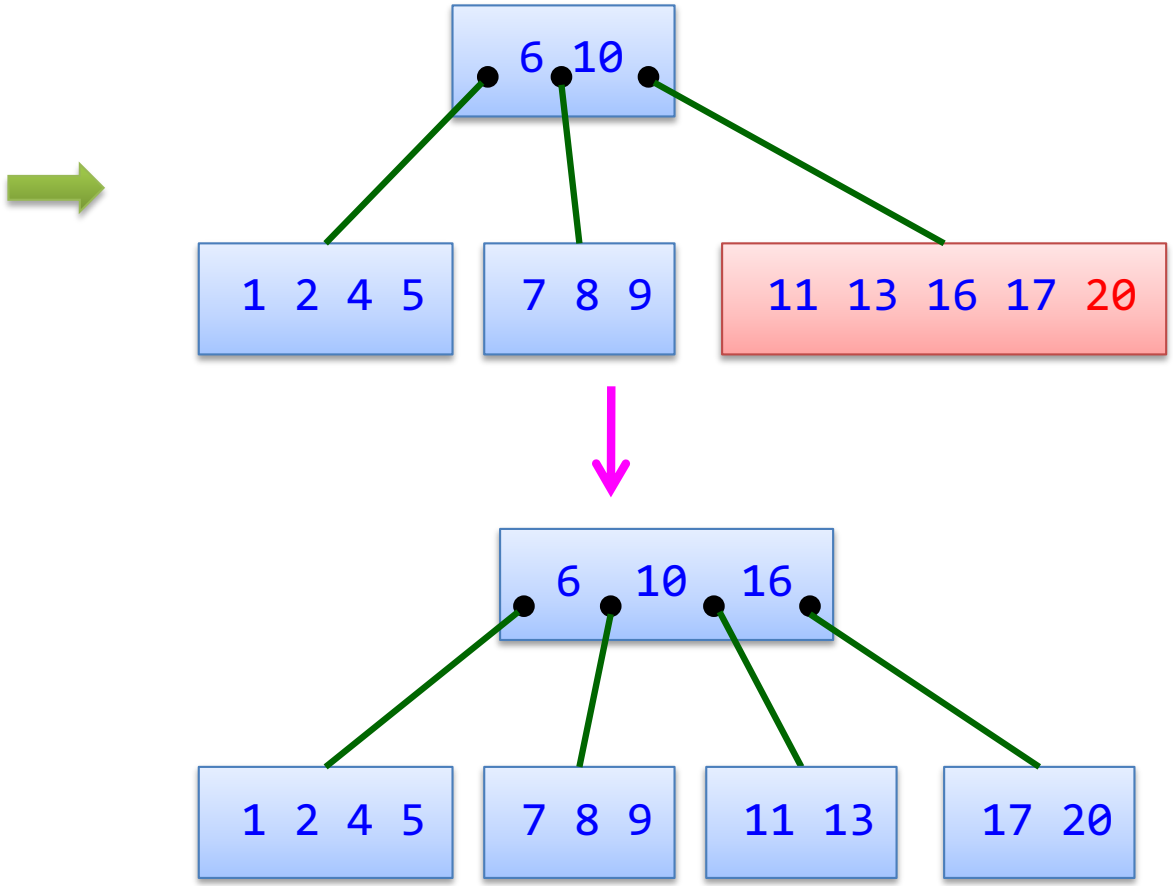



11
1 2 6 7 11
6
1 2
7 11
4,8,13
6
1 2 4
7 8 11 13
10
6
1 2 4
7 8 10 11 13
6 10
1 2 4
7 8
11 13

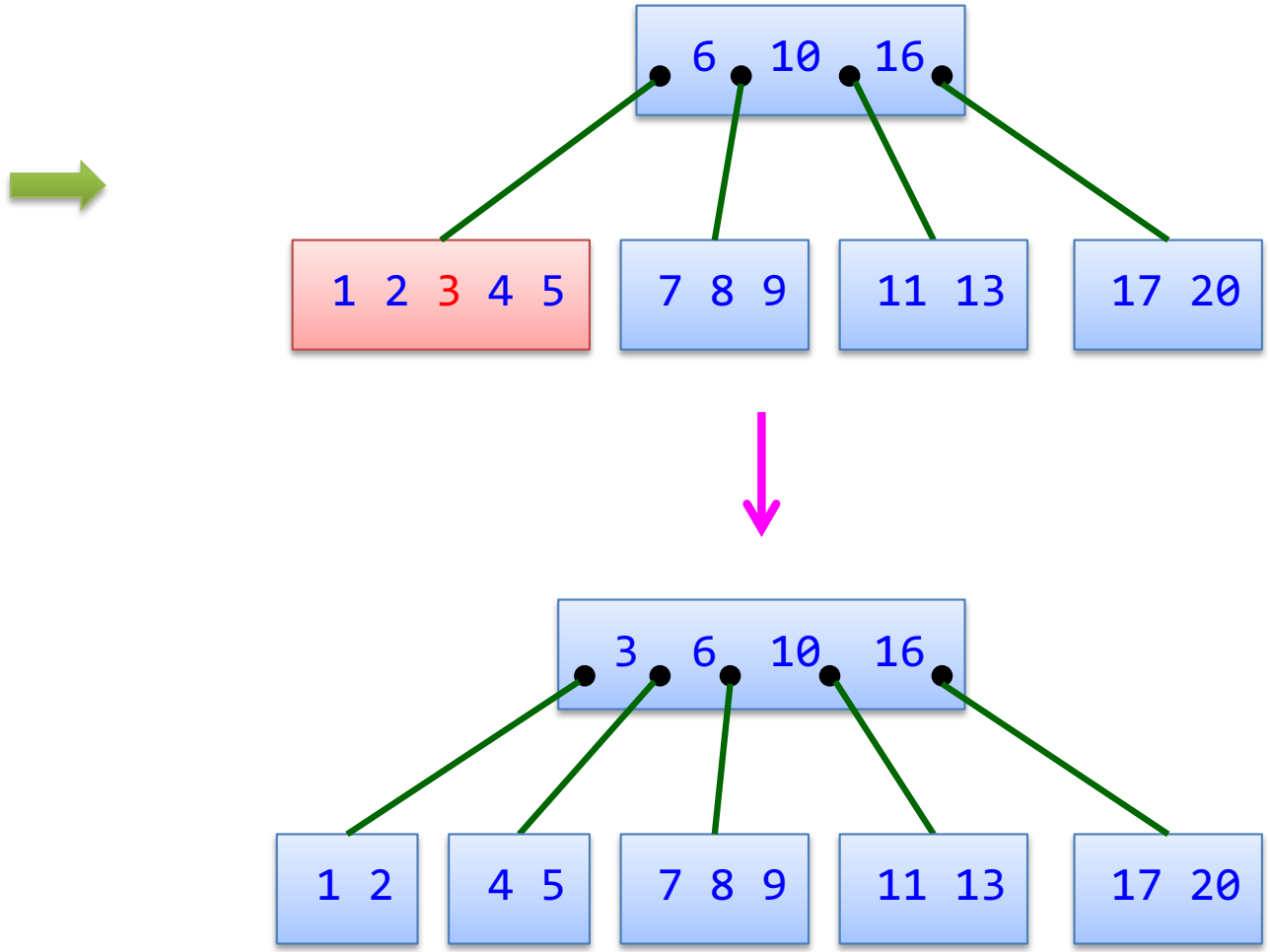
5,17,9,16


6 10
1 2 4 5
7 8 9
11 13 16 17

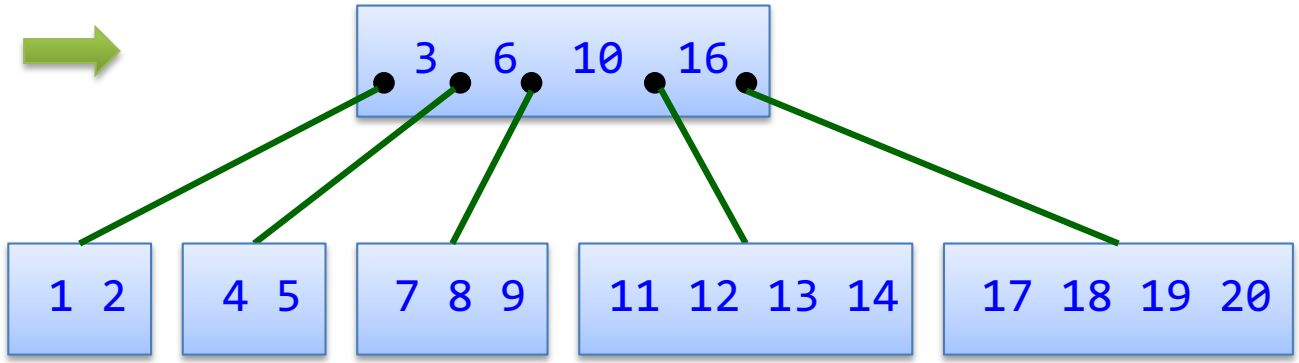
20

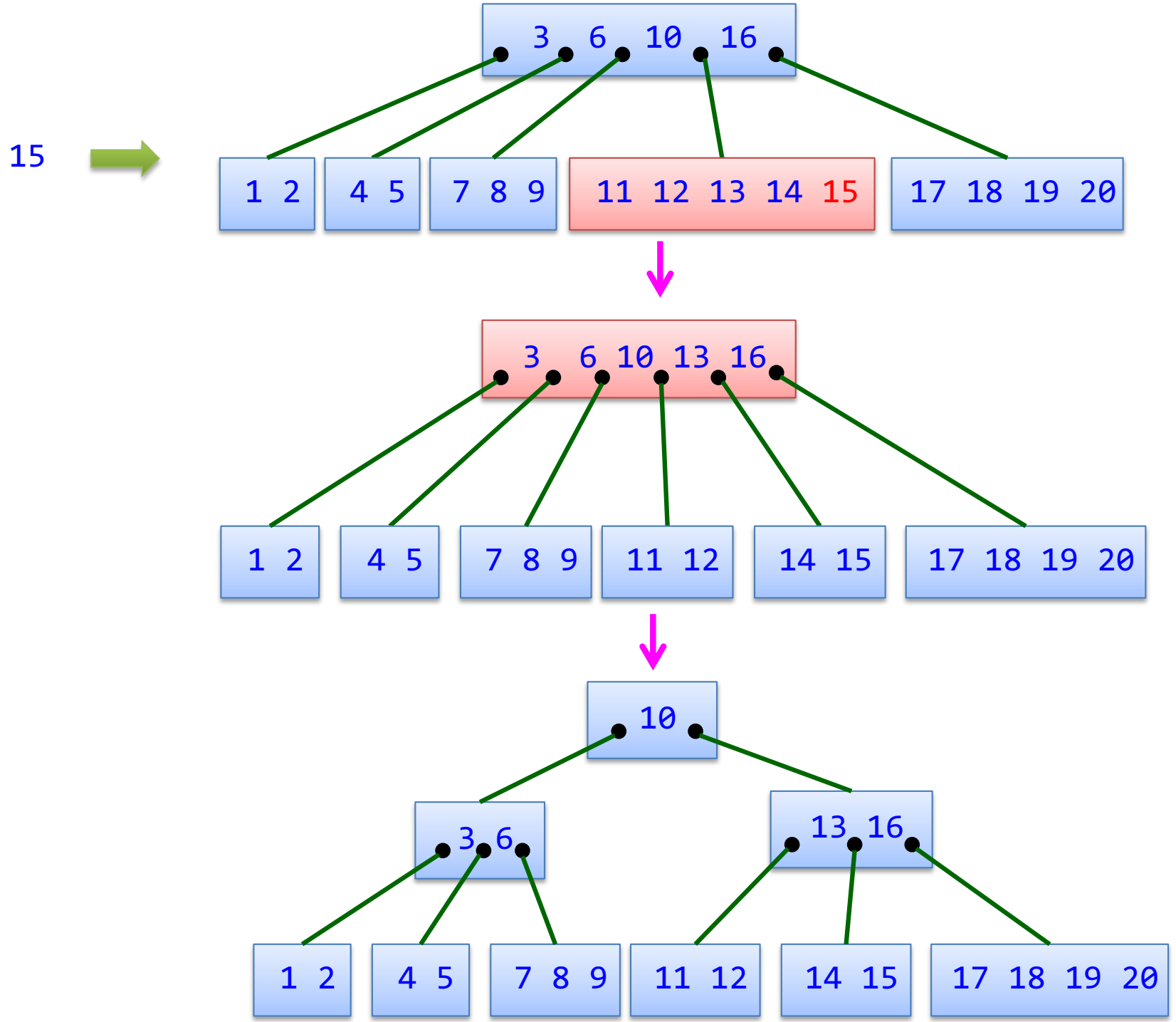

6 10
1 2 4 5
7 8 9
11 13 16 17 20
6 10 16
1 2 4 5
7 8 9
11 13
17 20

3

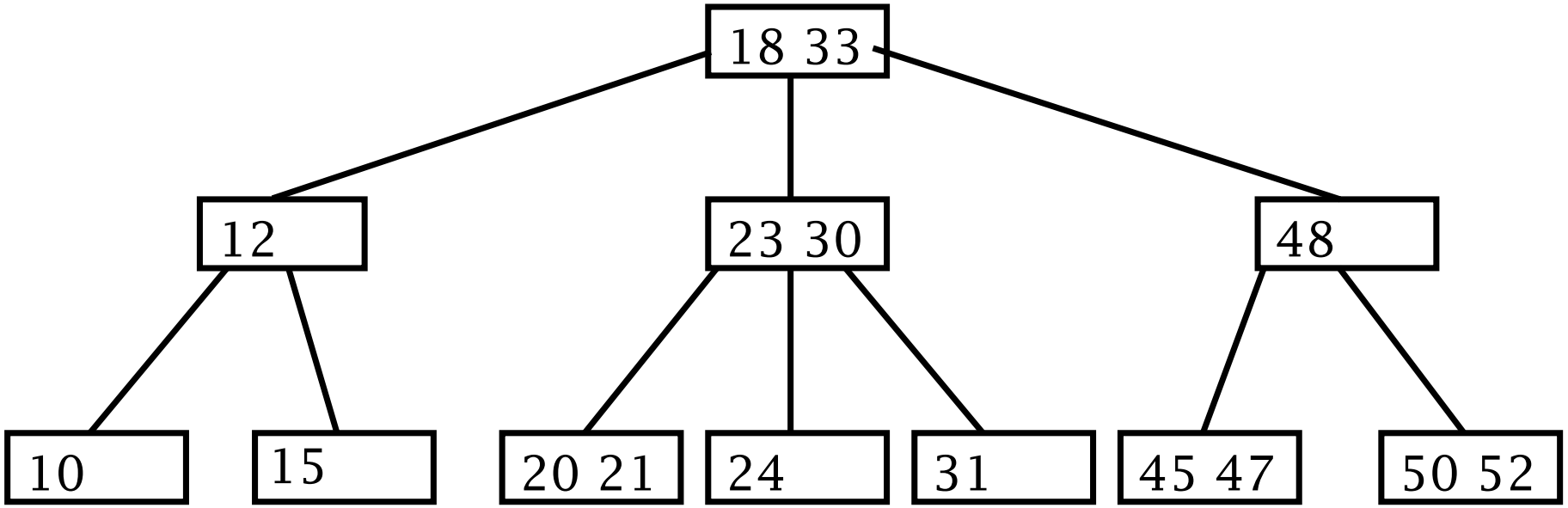


12,14,18,19




15
3 6 10 16
1 2
4 5
7 8 9
11 12 13 14 15
17 18 19 20
3 6 10 13 16
1 2
4 5
7 8 9
11 12
14 15
17 18 19 20
10
3 6
13 16
1 2
4 5
7 8 9
11 12
14 15
17 18 19 20

【练习】在下示3阶B树依次插入14，55，19。



3．B树的删除

- 利用前述的查找算法找出关键字*k*所在的结点*p*。
- 实施关键字*k*的删除操作。结点*p*分为两种情况，情况一是结点*p*是叶子结点，情况二是结点*p*不是叶子结点。

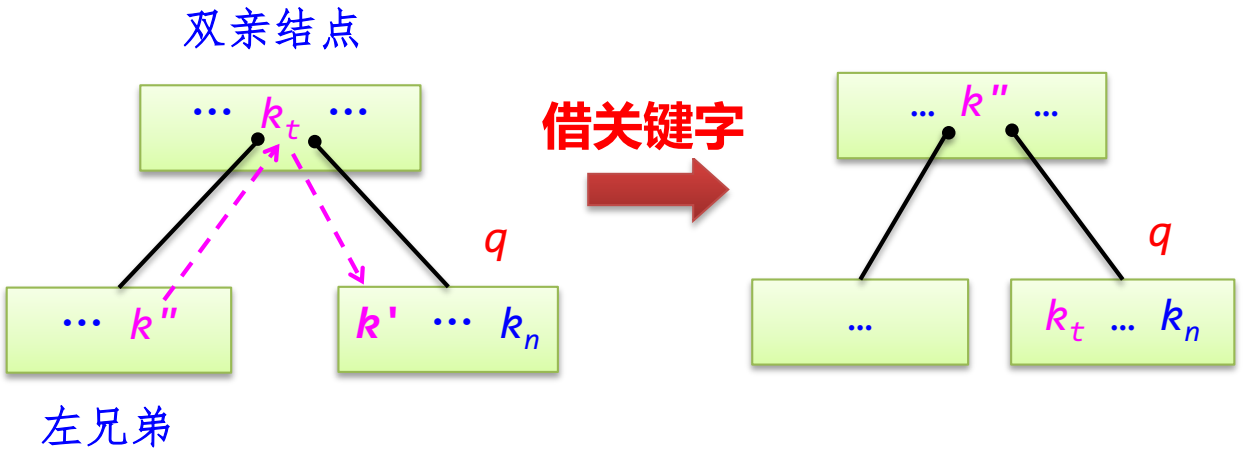
情况二转换为情况一

转换过程：当结点*p*不是叶子结点时，假设结点*p*中关键字key[*i*]=*k*（1≤*i*≤*n*），以*p*[*i*]（或*p*[*i*-1]）所指右子树（或左子树）中的最小关键字mink（或最大关键字maxk）来替代被删关键字key[*i*]（值替代），再删除关键字mink（或maxk）。

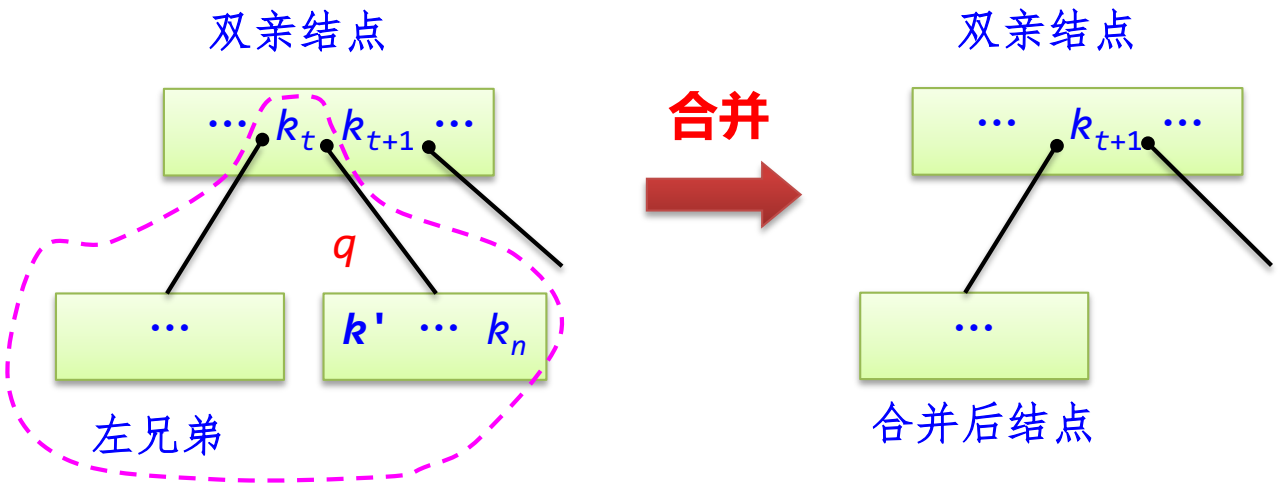
现在考虑情况1，即在m阶B树的某个叶子结点q中删除关键字k'=mink（或者k'=maxk），根据结点q中关键字个数n又分为以下三种子情况：

① 若n>Min（=$\lceil m/2 \rceil$-1），说明删除关键字k'后该结点仍满足B树的定义，则可直接从结点q中删除关键字k'。

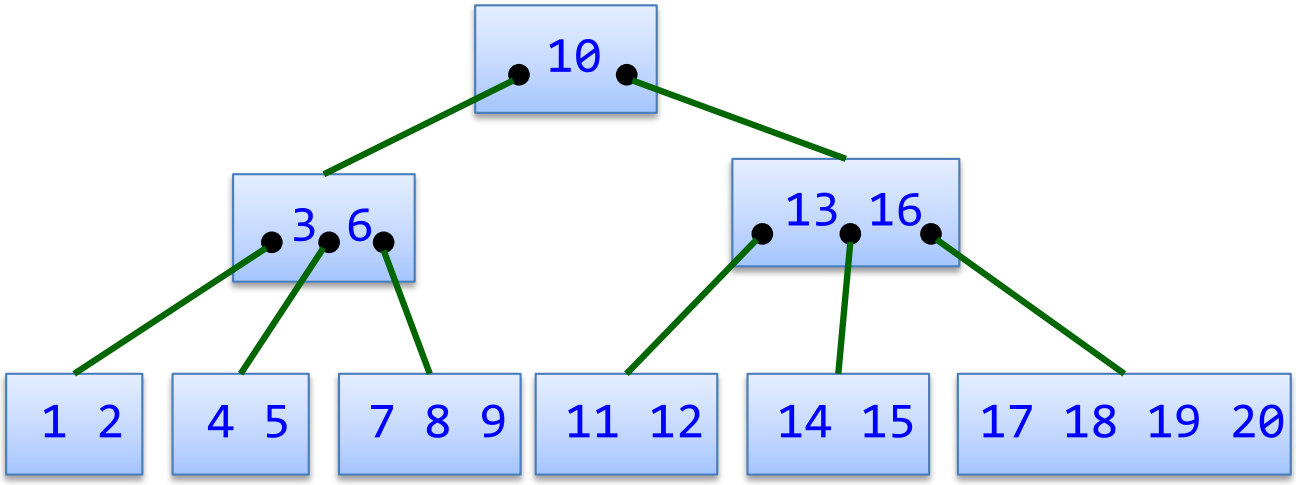
② 若n=Min，说明删除关键字k'后该结点不满足B树的定义，此时若结点q的左（或右）兄弟可以借 ⇨ 借一个关键字。



③ 假如结点**q**的关键字个数等于**Min**，并且该结点的左和右兄弟结点都不能借 ⇨ 合并。

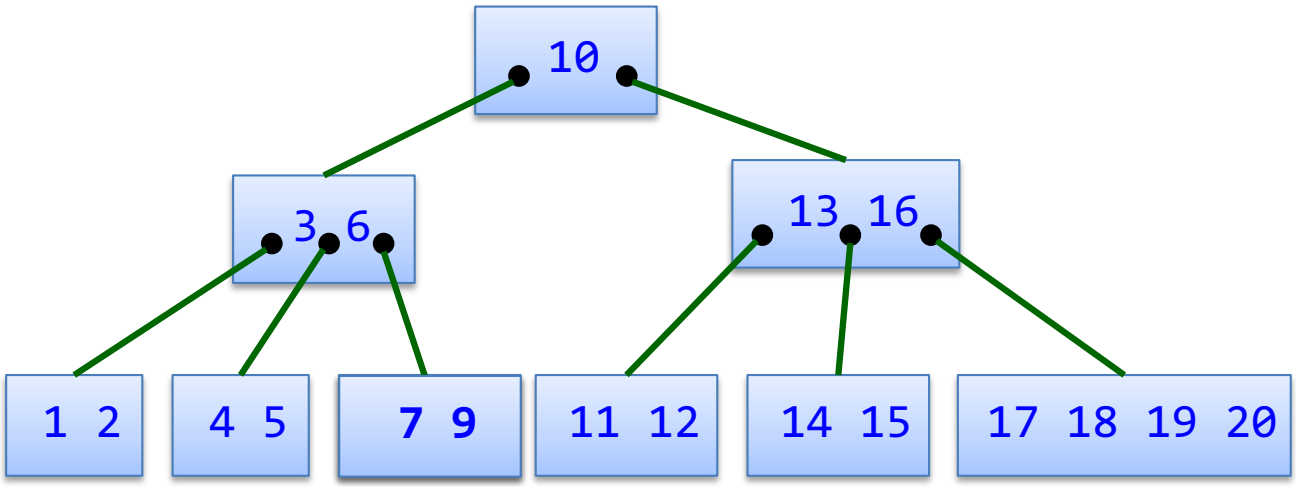


【例9.13】对于例9.12创建的如下最终5阶B树，给出删除8、16、15和4关键字的过程。



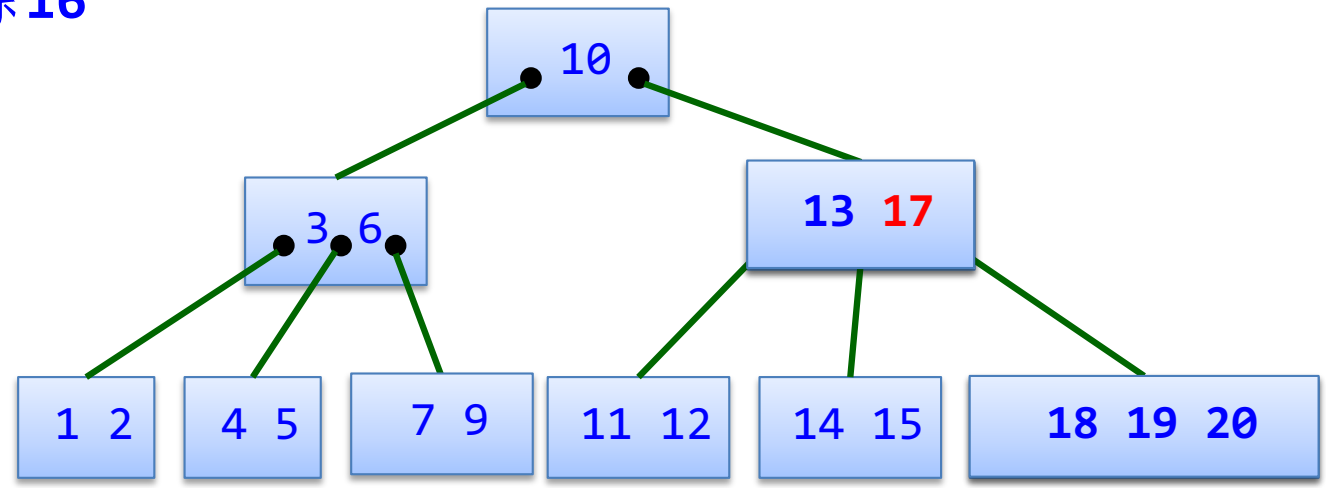
这里m=5，每个结点的关键字个数在2～4之间。

删除8



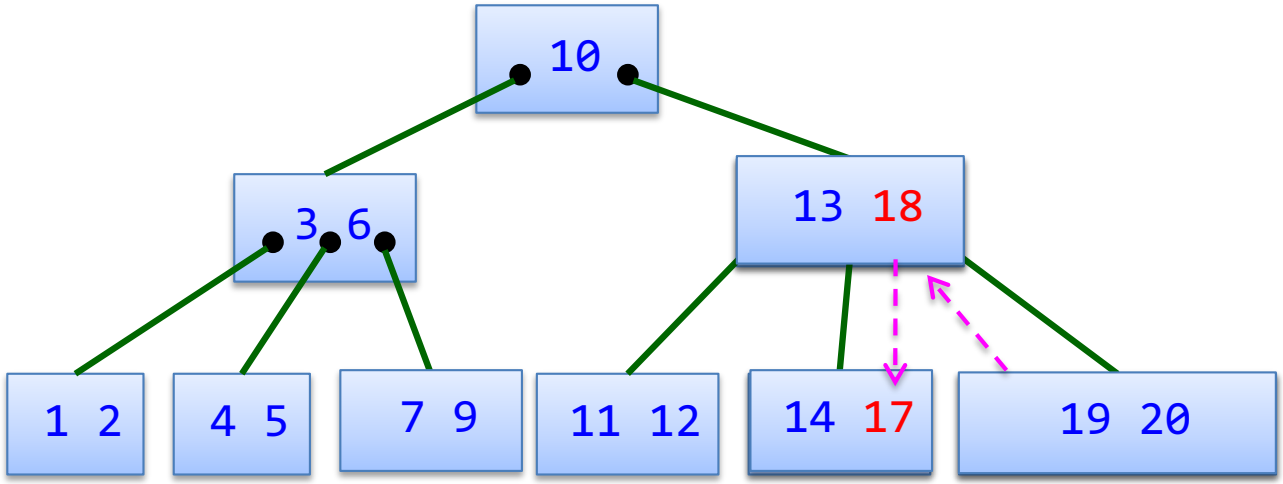
每个结点的关键字个数在2～4之间。

删除16



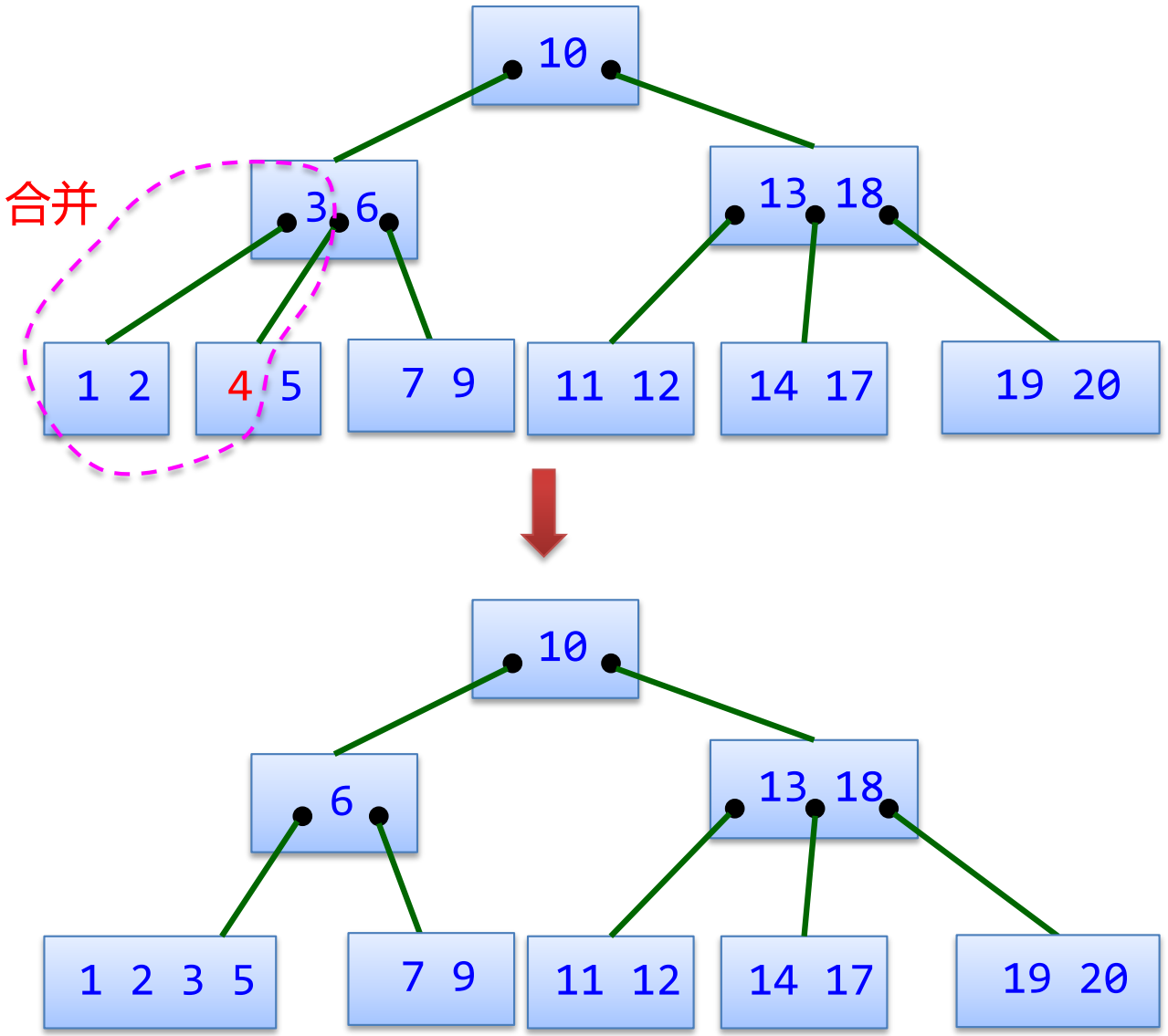
每个结点的关键字个数在**2**～**4**之间。

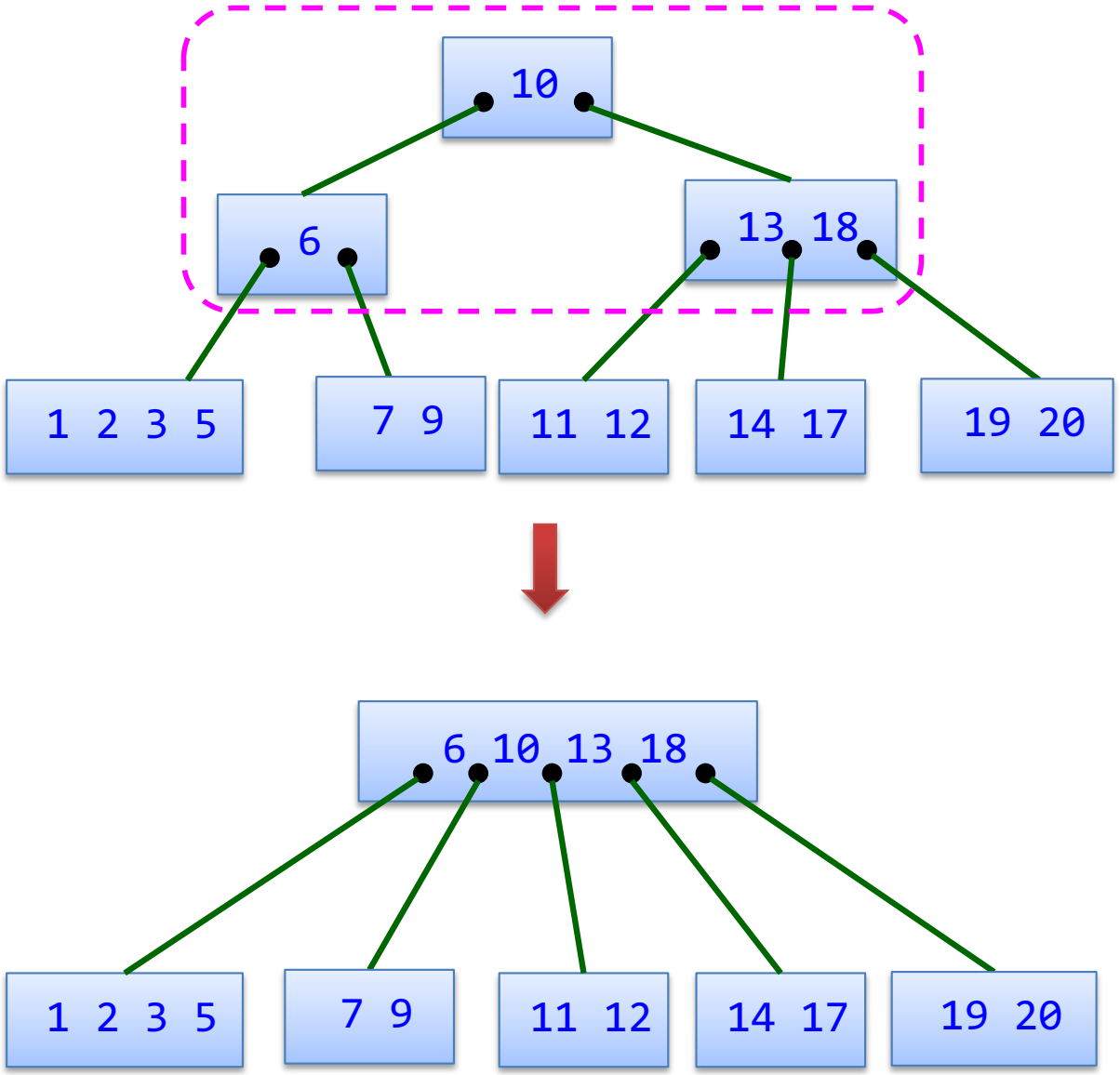
删除**15**



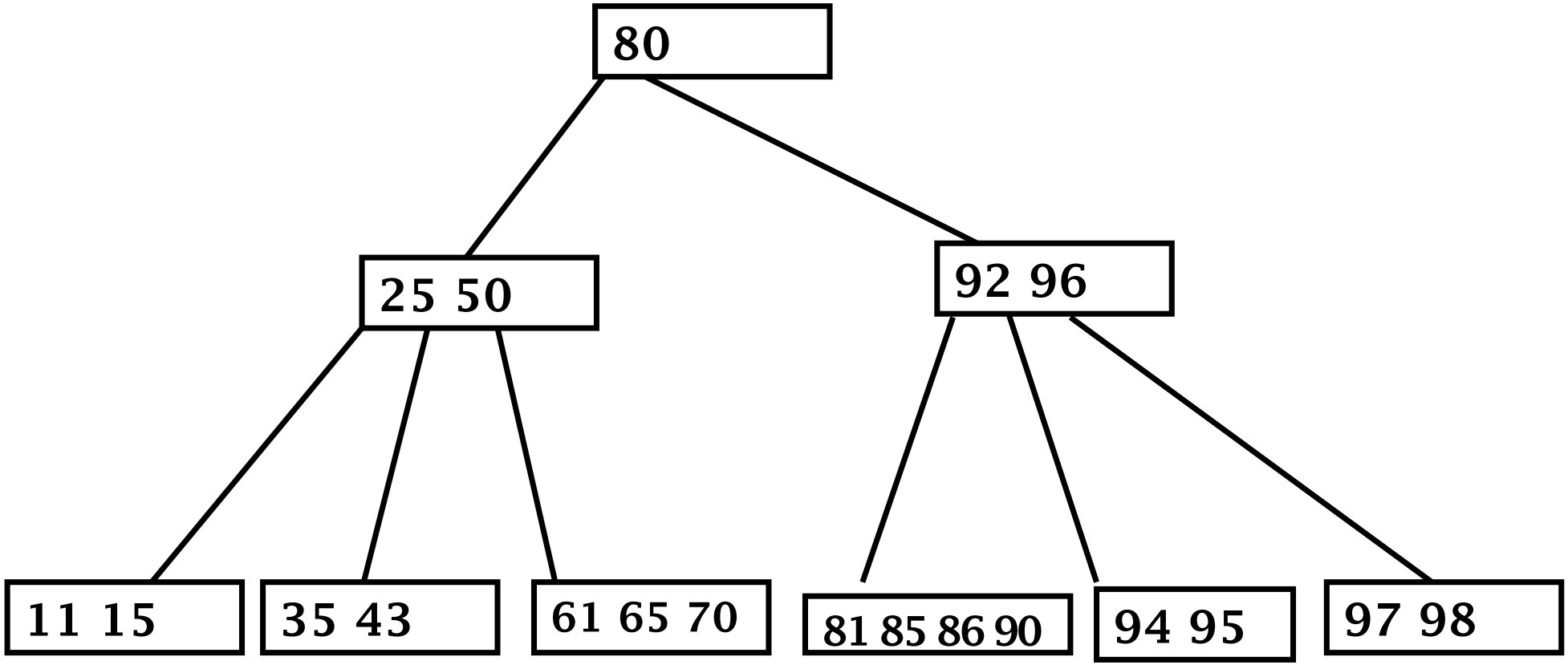
每个结点的关键字个数在2～4之间。

删除4




10
6
13 18
1 2 3 5
7 9
11 12
14 17
19 20
6 10 13 18
1 2 3 5
7 9
11 12
14 17
19 20

【练习】在下示5阶B树依次删除92，96。



9.3.5 B+树

在索引文件组织中，经常使用B树的一些变形，其中B+树是一种应用广泛的变形。一棵*m*阶B+树满足下列条件：

（1）每个分支结点至多有*m*棵子树。

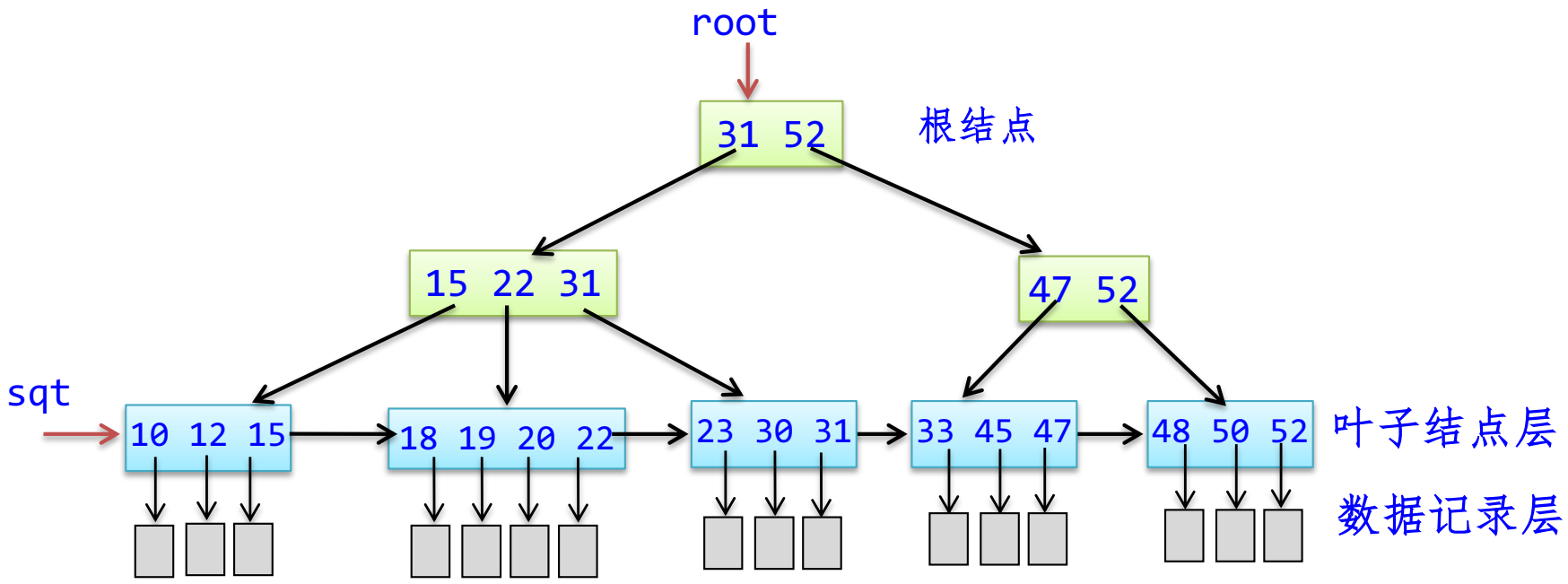
（2）根结点或者没有子树，或者至少有一棵子树。

（3）除根结点外，其他每个分支结点至少有$\lceil m/2 \rceil$棵子树。

（4）有*n*棵子树的结点有*n*个关键字。

（5）所有叶子结点包含全部关键字及指向相应数据记录的指针，而且叶子结点按关键字大小顺序链接（每个叶子结点的指针指向数据文件中的记录）。

（6）所有分支结点（可看成是索引）中仅包含各子树中最大关键字。

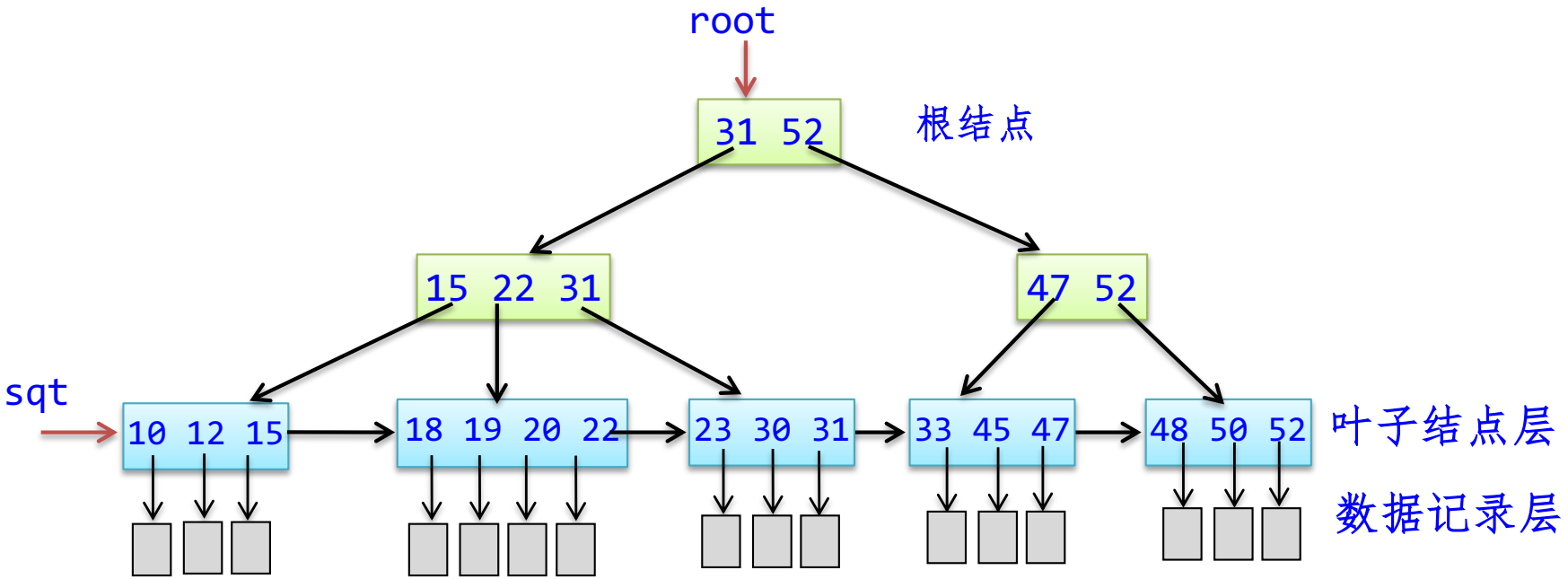


一棵4阶的B+树

*m*阶的B+树和*m*阶的B树的主要的差异

（1）在B+树中，具有*n*个关键字的结点对应*n*棵子树，即每个关键字对应一棵子树，而在B树中，具有*n*个关键字的结点对应*n*+1棵子树。

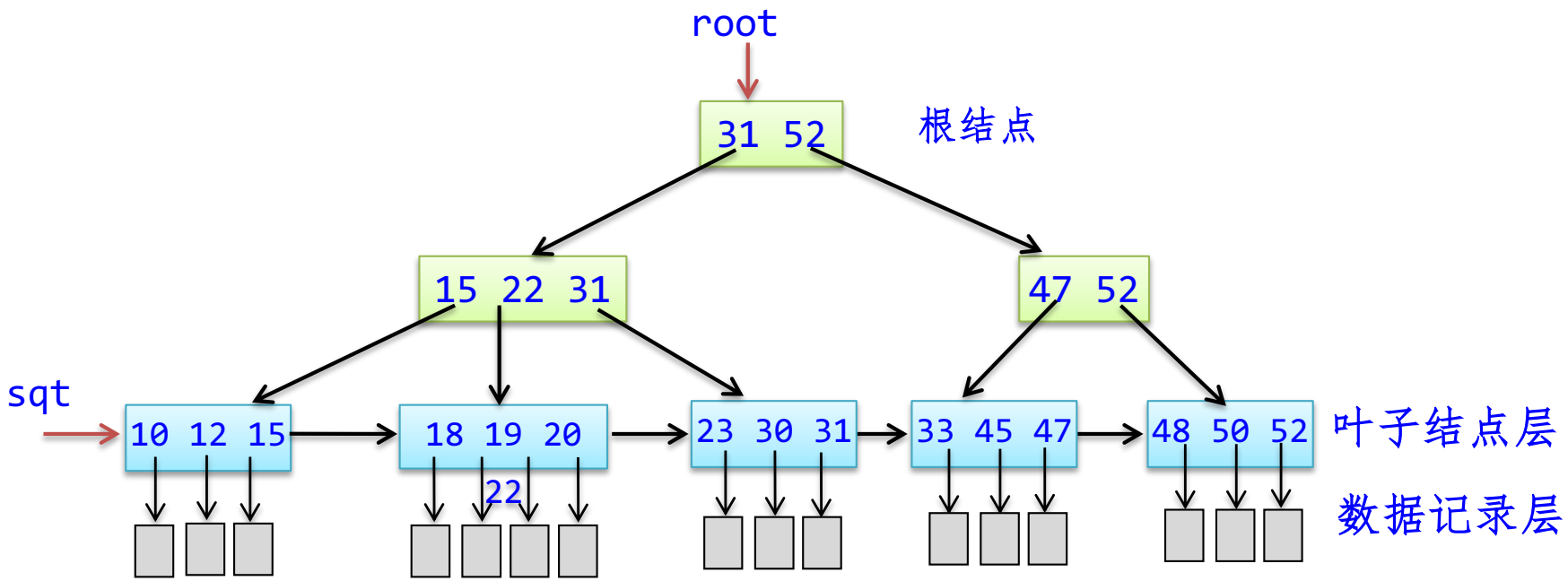
（2）在B+树中，每个结点（除根结点外）中的关键字个数*n*的取值范围是$\lceil m/2 \rceil \leqslant n \leqslant m$，根结点*n*的取值范围是$1 \leqslant n \leqslant m$。



一棵4阶的B+树

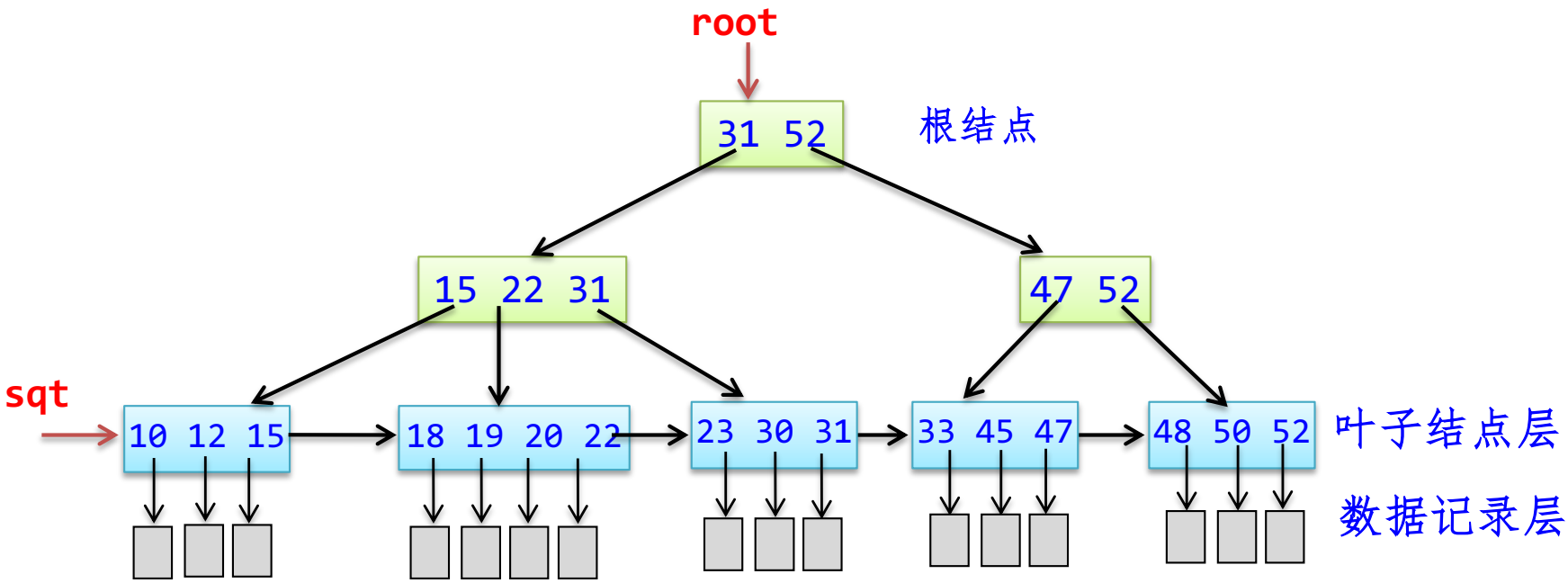
（3）B+树中的叶子结点层包含全部关键字，即其他非叶子结点中的关键字包含在叶子结点中，而在B树中，所有关键字是不重复的。

（4）B+树中所有非叶子结点仅起到索引的作用，即这些结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录。而在B树中，每个结点的关键字都含对应的记录。



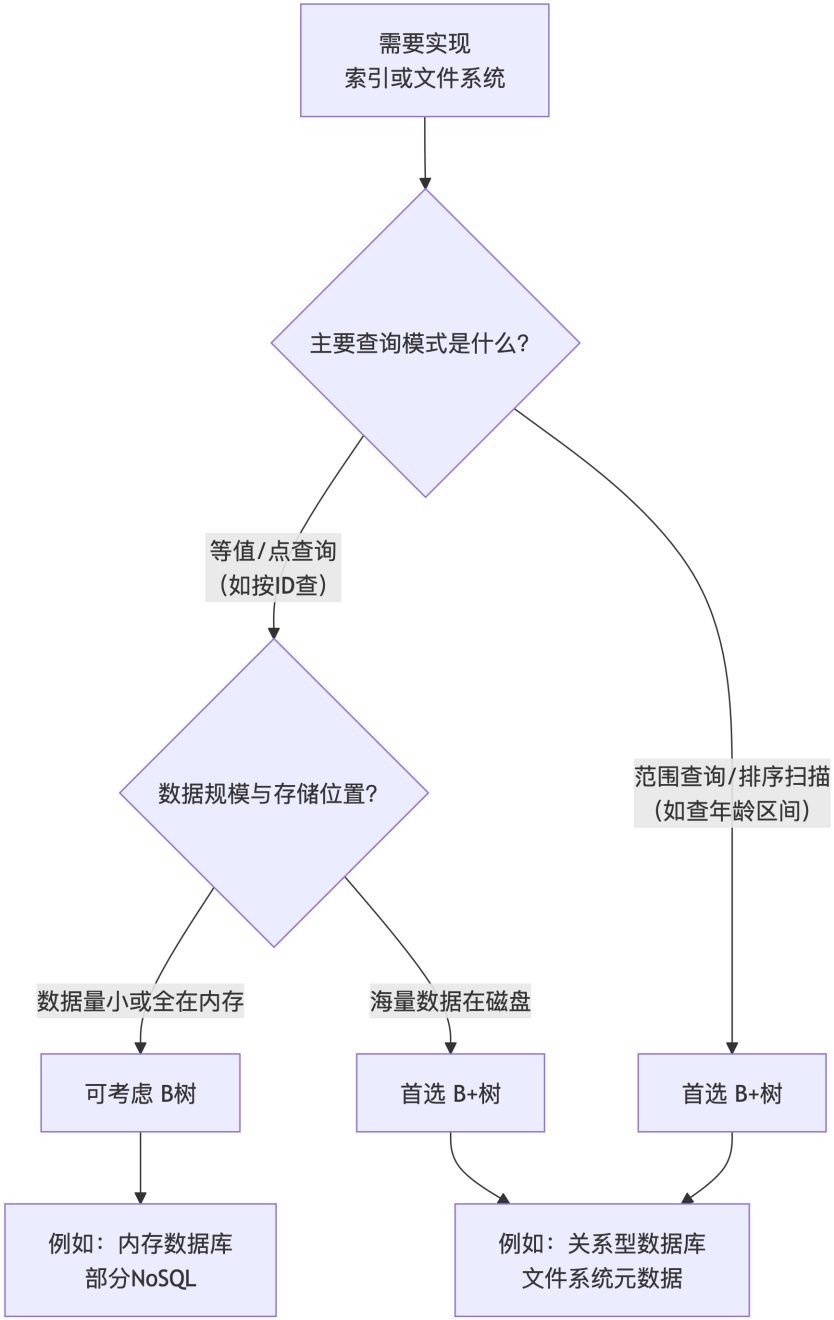
一棵4阶的B+树

（5）在B+树上有两个标识指针，一个是指向根结点的root，另一个是指向关键字最小叶子结点的sqt，所有叶子结点链接成一个不定长的线性链表，所以B+树既可以通过root随机查找，也可以通过sqt顺序查找。而在B树只能随机查找。



一棵4阶的B+树

特性维度	B树	B+树
数据存储位置	**所有节点**（包括内部节点）都可存放**完整数据记录**（键值对）。	**仅叶子节点**存放完整数据记录或数据指针；**内部节点仅存键**，作为导航索引。
叶子节点链接	叶子节点之间**没有**指针链接。	所有叶子节点通过**双向链表**串联，构成一个有序序列。
结构形态	树形结构。	**索引层** (内部节点) + **顺序数据层** (叶子节点链表)。
查询特点	查询可能在**任意节点**结束，性能不稳定。	任何查询都必须走到**叶子节点**，性能稳定；**非常适合范围查询**。
空间利用率	相对较低（因节点也存数据）。	更高（内部节点只存键，一个节点能索引更多孩子）。
应用场景	非关系型数据库(e.g.MongoDB)、文件系统	关系型数据库（如MySQL, PostgreSQL）


需要实现
索引或文件系统
主要查询模式是什么?
等值/点查询
(如按ID查)
范围查询/排序扫描
(如查年龄区间)
数据规模与存储位置?
数据量小或全在内存
海量数据在磁盘
可考虑 B树
首选 B+树
首选 B+树
例如：内存数据库
部分NoSQL
例如：关系型数据库
文件系统元数据

16. 【2016 统考真题】B+树不同于 B 树的特点之一是（　）。

A. 能支持顺序查找　　B. 结点中含有关键字

C. 根结点至少有两个分支　　D. 所有叶结点都在同一层上

17. 【2017 统考真题】下列应用中，适合使用 B+树的是（ ）。

A. 编译器中的词法分析

B. 关系数据库系统中的索引

C. 网络中的路由表快速查找

D. 操作系统的磁盘空闲块管理

22. 【2023 统考真题】下列关于非空 B 树的叙述中，正确的是（ ）。

I. 插入操作可能增加树的高度　　II. 删除操作一定会导致叶结点的变化

III. 查找某关键字总是要查找到叶结点　IV. 插入的新关键字最终位于叶结点中

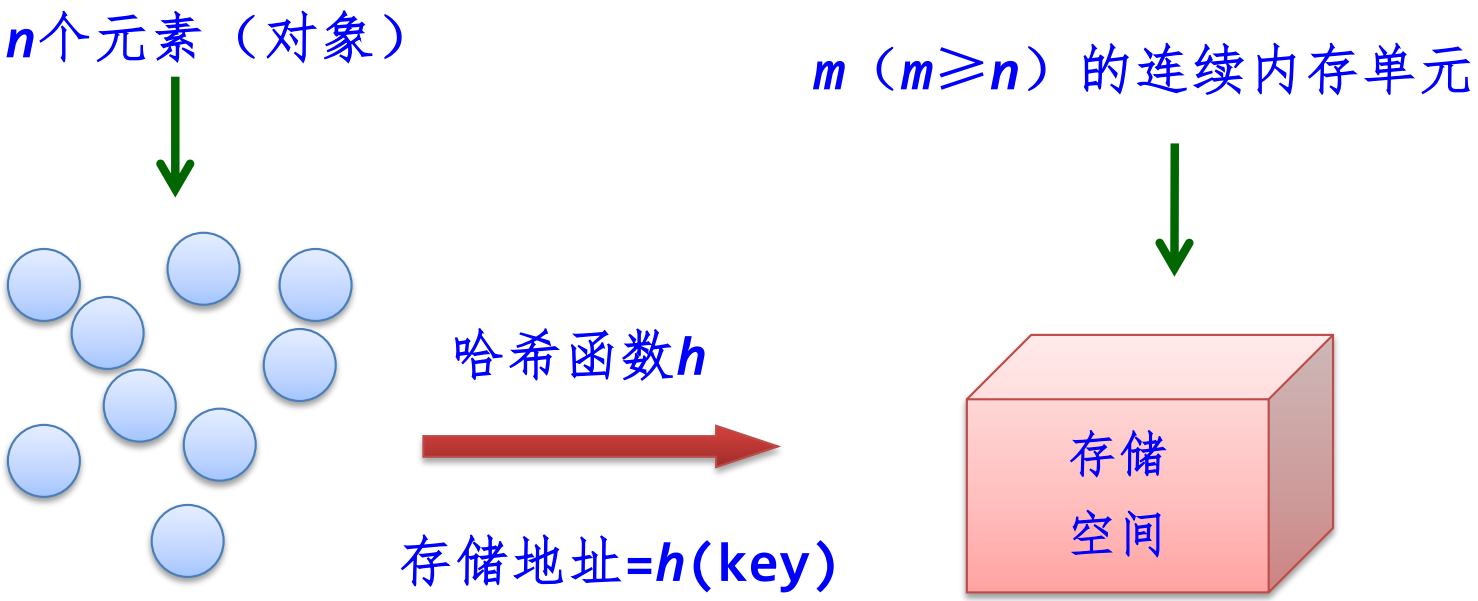
A. 仅 I　　B. 仅 I、II　　C. 仅 III、IV　　D. 仅 I、II、IV

9.4 哈希表查找

9.4.1 哈希表的基本概念

哈希表

- 设要存储的元素个数为n，设置一个长度为m（$m \geq n$）的连续内存单元。
- 以每个元素的关键字k_i（$0 \leqslant i \leqslant n-1$）为自变量，通过一个哈希函数$h$把$k_i$映射为内存单元的地址（或相对地址）$h(k_i)$。
- 并把该元素存储在这个内存单元中。


n个元素（对象）
m（m≥n）的连续内存单元
哈希函数h
存储地址=h(key)
存储
空间

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

n=7
m=12
h(学号)=学号-2018001

哈希表ha

哈希地址	学号	姓名	分数
0	2018001	王华	90
1			
2			
3			
4	2018005	李英	82
5	2018006	陈明	54
6	2018007	许兵	76
7			
8	2018009	张强	95
9	2018010	刘丽	62
10			
11	2018012	李萍	88

查找学号为2018010的学生分数：

- 先计算*h*(2018010)=2018010-2018001=9。
- 再取ha[9]元素的分数62即可。
- 对应的查找时间为O(1)。

- 对于两个不同的关键字k_i和k_j（$i \neq j$）出现$h(k_i)=h(k_j)$，这种现象称为哈希冲突。
- 将具有不同关键字而具有相同哈希地址的元素称为“同义词”，这种冲突也称为同义词冲突。

需要解决哈希冲突

9.4.2 哈希函数构造方法

- 构造哈希函数的目标：使得到的哈希地址尽可能均匀地分布在*m*个连续内存单元地址上，同时使计算过程尽可能简单以达到尽可能高的时间效率。
- 根据关键字的结构和分布的不同，有多种构造哈希函数的方法。

1．直接定址法

- 以关键字*k*本身或关键字加上某个数值常量c作为哈希地址的方法。即*h*(*k*)=*k*+c。
- 这种哈希函数计算简单，并且不可能有冲突发生。
- 当关键字的分布基本连续时，可用直接定址法的哈希函数；否则，若关键字分布不连续将造成内存单元的大量浪费。

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

n=7

m=12

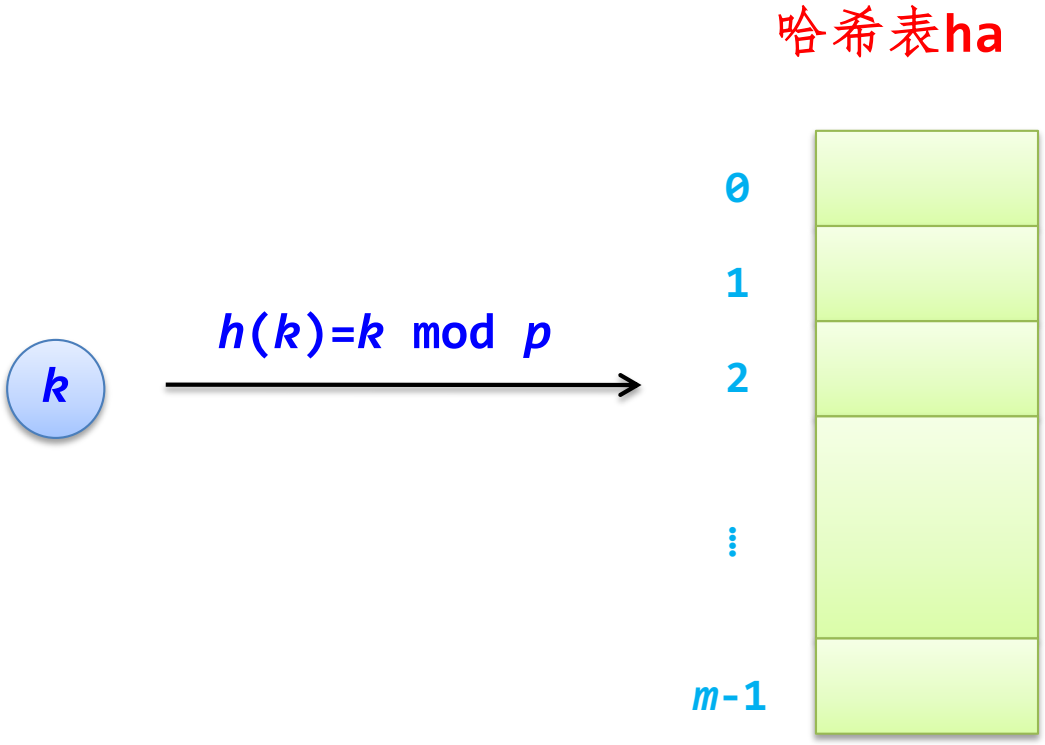
h(学号)=学号-2018001

哈希表ha

哈希地址	学号	姓名	分数
0	2018001	王华	90
1			
2			
3			
4	2018005	李英	82
5	2018006	陈明	54
6	2018007	许兵	76
7			
8	2018009	张强	95
9	2018010	刘丽	62
10			
11	2018012	李萍	88

2．除留余数法

- 用关键字k除以某个不大于哈希表长度m的数p所得的余数作为哈希地址的方法。
- 除留余数法的哈希函数$h(k)$为：$h(k)=k \bmod p$ (mod为求余运算，$p \leqslant m$)
- p最好是质数（素数）。



- $p \leqslant m$ ⇨保证地址有效
- p为质数 ⇨保证冲突尽可能小

3．数字分析法

- 提取关键字中取值较均匀的数字位作为哈希地址的方法。
- 适合于所有关键字值都已知的情况，并需要对关键字中每一位的取值分布情况进行分析。

位序	1	2	3	4	5	6	7	8
	9	2	3	1	7	6	0	2
	9	2	3	2	6	8	7	5
	9	2	7	3	9	6	2	8
	9	2	3	4	3	6	3	4
	9	2	7	0	6	8	1	6
	9	2	7	7	4	6	3	8
	9	2	3	8	1	2	6	2
	9	2	3	9	4	2	2	0

哈希地址的集合为{2，75，28，34，16，38，62，20}

大范围的数值 → 小范围的数值

如果关键字是字符串，需要将字符串转换为唯一的整数值，例如BKDRHash函数就是一种简单快捷的哈希转换算法（因Brian Kernighan与Dennis Ritchie在著名的《The C Programming Language》一书提出而得名）。

```
unsigned int BKDRHash(char* str)        //生成str字符串的哈希函数值
{  unsigned int seed=131;               //种子值，也可以为31、131、
                                        //1313、13131、131313等
   unsigned int hash = 0;
   while (*str)
      hash=hash*seed+(*str++);
   return hash;
}
```

9.4.3 哈希冲突解决方法

在哈希表中，虽然冲突很难避免，但发生冲突的可能性却有大有小。这主要与三个因素有关：

- 与装填因子有关。所谓装填因子α是指哈希表中已存入的元素数n与哈希地址空间大小m的比值，即$\alpha=n/m$。α越小，冲突的可能性就越小； 但α越小，存储空间的利用率就越低。
- 与所采用的哈希函数有关。
- 与解决冲突的方法有关。

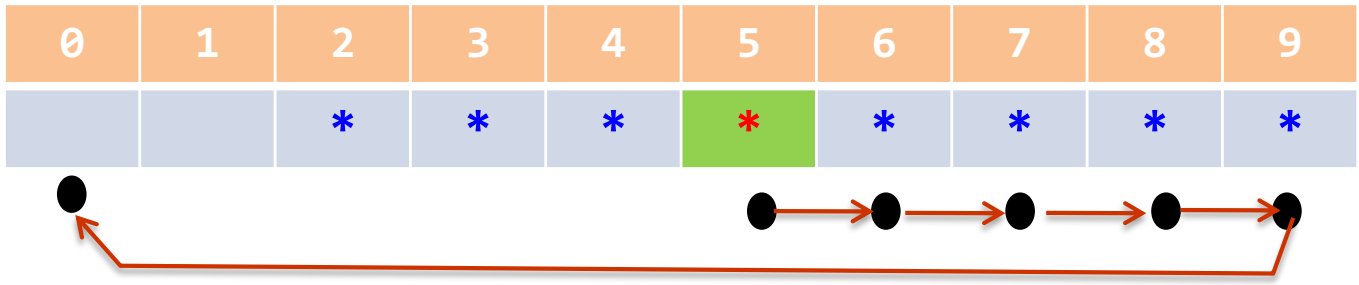
解决哈希冲突方法有许多，可分为开放定址法和拉链法两大类。

1．开放定址法

- 发生冲突时查找周围一个空位置存放元素（记录）。
- 设置一个查找周围一个空位置的函数。

（1）线性探测法

- 从发生冲突的地址（设为d）开始，依次循环探测d的下一个地址（当到达下标为$m-1$的哈希表表尾时，下一个探测的地址是表首地址0），直到找到一个空闲单元为止。
- 描述公式为：$d_0=h(k)$，$d_i=(d_{i-1}+1)\ \mathrm{mod}\ m$　$(1\leqslant i\leqslant m-1)$



问题：可能出现堆积现象：

n=6,m=10,关键字为（10,11,12,19,20,21）

哈希函数：h(key) = key % 9

10，11，12

0	1	2	3	4	5	6	7	8	9
	10	11	12						

- 19 ⇨ h(19)=19%9=1（冲突）
- d_0=1，d_1=(1+1)%10=2 （冲突）
- d_2=(2+1)%10=3 （冲突）
- d_3=(3+1)%10=4（将19放在4位置）

0	1	2	3	4	5	6	7	8	9
	10	11	12	19					

- 20 ⇨ h(20)=20%9=2（冲突）
- d_0=2，d_1=(2+1)%10=3 （冲突）
- d_2=(3+1)%10=4 （冲突）
- d_3=(4+1)%10=5（将20放在5位置）

哈希函数值不相同的多个记录争夺同一个后继哈希地址称为非同义词冲突

假设哈希表每个元素为[*k*，*v*]，其中*k*为关键字（为了方便计算哈希函数，假设关键字为int类型），*v*为对应的值（值类型为T），哈希表的元素类型如下：

```
#define NULLKEY -1                          //全局变量,空关键字
template <typename T>
struct HNode                                //哈希表元素类型
{  int key;                                 //关键字
   T value;                                 //数据值

   HNode():key(NULLKEY) {}                  //构造函数
   HNode(int k,T v)                         //重载构造函数
   {  key=k;
      value=v;
   }
};
```

设计哈希函数为$h(k)=k \% p$，哈希表长度为m，采用线性探测法解决冲突，包含插入算法insert的哈希表类HashTable1：

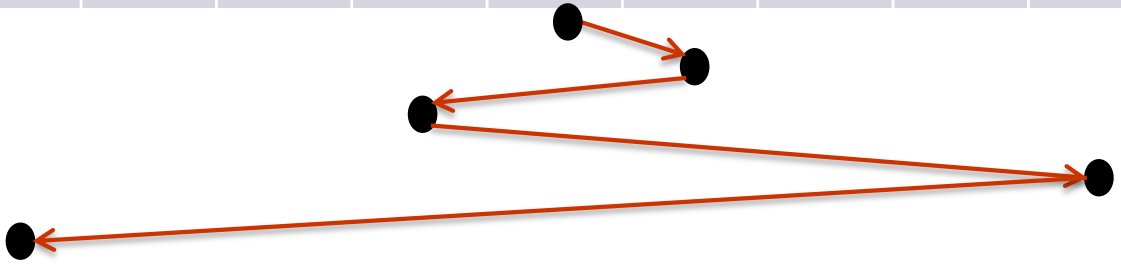
```
#define MAXM 100                    //哈希表最大长度
template <typename T>
class HashTable1                    //哈希表(除留余数法+线性探测法)
{  int n;                           //哈希表中元素个数
   int m;                           //哈希表长度
   int p;
   HNode<T> ha[MAXM];               //存放哈希表元素
public:
   HashTable1(int m,int p)          //哈希表构造函数
   {  this->m=m; this->p=p;
      for (int i=0;i<m;i++)         //初始化为空哈希表
          ha[i].key=NULLKEY;
      n=0;
   }
```

```
    void insert(int k,T v)                  //在哈希表中插入(k,v)
    {  int d=k % p;                         //求哈希函数值
       while (ha[d].key!=NULLKEY)           //找空位置
          d=(d+1) % m;                      //线性探测法查找空位置
       ha[d]=HNode<T>(k,v);                 //放置(k,v)
       n++;                                 //增加一个元素
    }
    //其他运算函数
};
```

（2）平方探测法

- 发生冲突时前后查找空位置。
- 描述公式为：$d_0=h(k)$，　$d_i=(d_0\pm i^2)$ mod m　$(1\leqslant i\leqslant m-1)$

0	1	2	3	4	5	6	7	8	9
		*	*	*	*	*	*	*	*



- 平方探测法可以避免出现堆积问题。
- 缺点是不能探测到哈希表上的所有单元，但至少能探测到一半单元。

【例9.14】假设哈希表ha长度m=13，采用除留余数法和线性探测法解决冲突建立关键字集合{16，74，60，43，54，90，46，31，29，88，77}的哈希表。

n=11，m=13，除留余数法的哈希函数为：

$$h(k)=k \bmod p$$

p应为小于等于m的素数，假设p取值13。

哈希函数：$h(k)=k \bmod 13$

解决冲突方法：线性探测法

- $h(16)=3$　　ha[3]=16,共1次探测
- $h(74)=9$　　ha[9]=74,共1次探测
- $h(60)=8$　　ha[8]=60,共1次探测
- $h(43)=4$　　ha[4]=43,共1次探测
- $h(54)=2$　　ha[2]=54,共1次探测
- $h(90)=12$　　ha[12]=90,共1次探测
- $h(46)=7$　　ha[7]=46,共1次探测
- $h(31)=5$　　ha[5]=31,共1次探测

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

- $h(29)=3$ 有冲突

  $d_0=3$，$d_1=(3+1)\ \%\ 13=4$ 仍有冲突

  $d_2=(4+1)\ \%\ 13=5$ 仍有冲突

  $d_3=(5+1)\ \%\ 13=6$ ha[6]=29,共4次探测
- $h(88)=10$ ha[10]=88,共1次探测
- $h(77)=12$ 有冲突

  $d_0=12$，$d_1=(12+1)\ \%\ 13=0$ ha[0]=77，共2次探测

哈希表ha[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

【2023 统考真题】现有长度为 5、初始为空的散列表 HT，散列函数 H(k)=(k+4)%5，用线性探查再散列法解决冲突。若将关键字序列 2022,12,25 依次插入 HT，然后删除关键字 25，则 HT 中查找失败的平均查找长度为（ ）。

A. 1　　B. 1.6　　C. 1.8　　D. 2.2

23．C

当采用开放定址法时，不能随便物理删除表中的已有元素，因为若删除元素，则可能截断其他具有相同散列地址的元素的查找地址。因此，当要删除一个元素时，可给它做一个删除标记。依次将 2022, 12, 25 插入散列表，然后删除 25，得到的散列表如下：

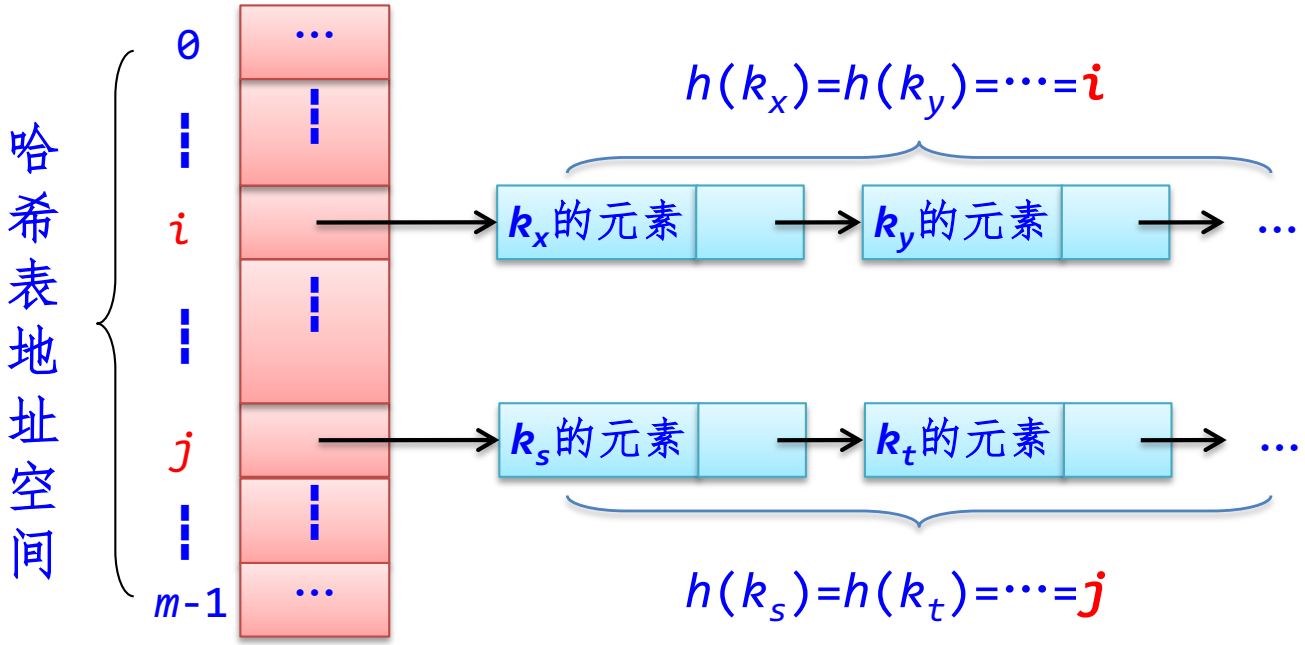
地址	0	1	2	3	4
关键字		2022	12		25（删除）
查找失败次数	1	3	2	1	2

当查找位置是删除标记时，应继续往后查找。

查找失败的平均查找长度为(1 + 3 + 2 + 1 + 2)/5 = 1.8。

2．拉链法

- 拉链法是把所有的同义词用单链表链接起来的方法。
- 在这种方法中，哈希表每个单元中存放的不再是元素本身，而是相应同义词单链表的首结点指针。
- 由于单链表中可插入任意多个结点，所以此时装填因子α根据同义词的多少既可以设定为大于1，也可以设定为小于或等于1，通常取$\alpha=0.75$。


哈希表地址空间
0
i
j
m-1
$h(k_x)=h(k_y)=\cdots=i$
k_x的元素
k_y的元素
k_s的元素
k_t的元素
$h(k_s)=h(k_t)=\cdots=j$

假设哈希表每个元素为[*k*，*v*]，其中*k*为关键字（为了方便计算哈希函数，关键字为int类型），*v*为对应的值（值类型为T），拉链法哈希表中单链表类型如下：

```
template <typename T>
struct HNode                        //单链表结点类
{  int key;                         //关键字
   T value;                         //数据值
   HNode<T>* next;                  //下一个结点指针
   HNode() {}                       //构造函数
   HNode(int k,T v)                 //重载构造函数
   {  key=k;
      value=v;
      next=NULL;
   }
};
```

假设哈希表长度为*m*，哈希函数为*h(k)=k % m*，采用拉链法解决冲突，包含插入算法**insert**的哈希表类**HashTable2**：

```
#define MAXM 100                          //哈希表最大长度
class HashTable2                          //哈希表(除留余数法+拉链法)
{  int n;                                 //哈希表中元素个数
   int m;                                 //哈希表长度
   HNode<T>* ha[MAXM];                    //存放哈希表中单链表首结点地址
public:
   HashTable2(int m)                      //哈希表构造函数
   {  this->m=m;
      for (int i=0;i<m;i++)
          ha[i]=NULL;
      n=0;
   }
   ~HashTable2()                          //析构函数：释放整个哈希表空间
   {
       //与销毁邻接表类似
   }
```

```
    void insert(int k,T v)          //在哈希表中插入(k,v)
    {   int d=k % m;                //求哈希函数值
        p=new HNode<T>(k,v);        //新建关键字k的结点p
        p->next=ha[d];              //采用头插法将p插入到ha[d]单链表中
        ha[d]=p;
        n++;                        //哈希表元素个数增1
    }
    //其他运算函数
};
```

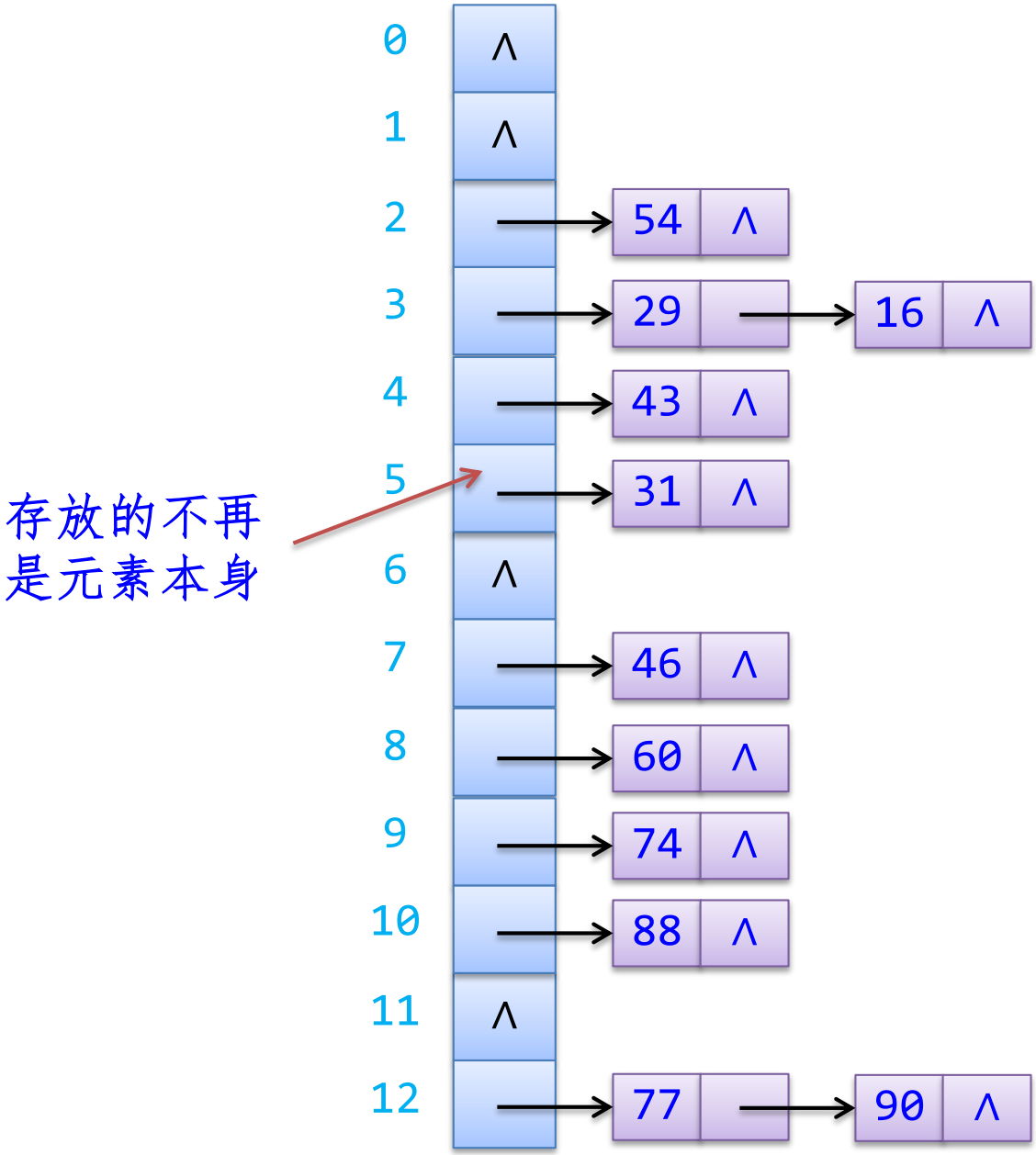
【例9.15】假设哈希表长度m=13，采用除留余数法和拉链法解决冲突建立关键字集合{16，74，60，43，54，90，46，31，29，88，77}的哈希表。

n=11，m=13，除留余数法的哈希函数为$h(k)=k \bmod p$，p应为小于等于m的素数，假设p取值13，当出现哈希冲突时采用拉链法解决冲突。

$h(16)=3$
$h(74)=9$
$h(60)=8$
$h(43)=4$
$h(54)=2$
$h(90)=12$
$h(46)=7$
$h(31)=5$
$h(29)=3$
$h(88)=10$
$h(77)=12$

$h(54)=2$
$h(16)=3$
$h(29)=3$
$h(43)=4$
$h(31)=5$
$h(46)=7$
$h(74)=9$
$h(60)=8$
$h(88)=10$
$h(90)=12$
$h(77)=12$

采用拉链法解决冲突建立的链表如下图所示。



9.4.4 哈希表查找及性能分析

1. 采用开放定址法建立的哈希表的查找

假设有元素类型为HashNode的哈希表ha[0..*m*-1]，哈希函数为*h*(*k*)=*k* % *p*，采用开放定址法中的线性探测法解决冲突，哈希表中空元素的关键字为常量NULLKEY。

```
int search(int k)      //查找关键字k,成功时返回其位置，否则返回-1
{  int d=k % p;                    //求哈希函数值
   while (ha[d].key!=NULLKEY && ha[d].key!=k)
      d=(d+1) % m;                 //线性探测法查找空位置
   if (ha[d].key==k)               //查找成功返回其位置
      return d;
   else                            //查找失败返回-1
      return -1;
}
```

示例

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

成功的查找：在ha中找到对应的关键字

- $h(77)=12$　　ha[12]=90≠77

  $d_0=12$，$d_1=(12+1)\ \%\ 13=0$　　ha[0]=77，共2次比较

比较的次数=探测次数

$$ASL_{succ}=\frac{9\times1+1\times2+4\times1}{11}=1.364$$

哈希表ha[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90

不成功的查找：在ha中找不到对应的关键字*x*

- $h(x)=12$　　ha[12]≠*x*

  $d_0=12$，$d_1=(12+1)\ \%\ 13=0$　　ha[0]≠*x*

  $d_2=(0+1)\ \%\ 13=1$　　ha[1]为空，表示查找失败，共3次比较

确定查找失败，一定有比较到空为止!

哈希表ha中查找失败的所有情况的探测次数

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2	1	10	9	8	7	6	5	4	3	2	1	3

$$ASL_{unsucc} = \frac{2+1+10+9+8+7+6+5+4+3+2+1+3}{13} = 4.692$$

【例9.16】 将关键字序列(7,8,30,11,18,9,14)散列存储到散列表中，散列表的存储空间是一个下标从0开始的一维数组，散列函数为：$H(key)=(key\times 3) \bmod 7$，处理冲突采用线性探测再散列法，要求装填（载）因子为0.7。

（1）请画出所构造的散列表。

（2）分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

说明：本题为2010年全国考研题。

（1）$n=7$，$\alpha=0.7=n/m$，则$m=n/0.7=10$。

计算各关键字存储地址的过程如下：

$H(7)=7\times3 \bmod 7=0$

$H(8)=8\times3 \bmod 7=3$

$H(30)=30\times3 \bmod 7=6$

$H(11)=11\times3 \bmod 7=5$

$H(18)=18\times3 \bmod 7=5$　　冲突

$d_1=(5+1) \bmod 10=6$　　仍冲突

$d_2=(6+1) \bmod 10=7$

$H(9)=9\times3 \bmod 7=6$　冲突

$d_1=(6+1) \bmod 10=7$　　仍冲突

$d_2=(7+1) \bmod 10=8$

$H(14)=14\times3 \bmod 7=0$　　冲突

$d_1=(0+1) \bmod 10=1$

构造的哈希表：

下标	0	1	2	3	4	5	6	7	8	9
关键字	**7**	**14**		**8**		**11**	**30**	**18**	**9**	
探测次数	**1**	**2**		**1**		**1**	**1**	**3**	**3**	

（2）在等概率情况下：

$$ASL_{成功}=(1+2+1+1+1+3+3)/7=12/7=1.71$$

不成功的情况下所有探测次数：

下标	0	1	2	3	4	5	6	7	8	9
关键字	**7**	**14**		**8**		**11**	**30**	**18**	**9**	
探测次数	**3**	**2**	**1**	**2**	**1**	**5**	**4**	**3**	**2**	**1**

所以有：

$$ASL_{不成功}=(3+2+1+2+1+5+4)/7=18/7=2.57$$

2．采用拉链法建立的哈希表的查找

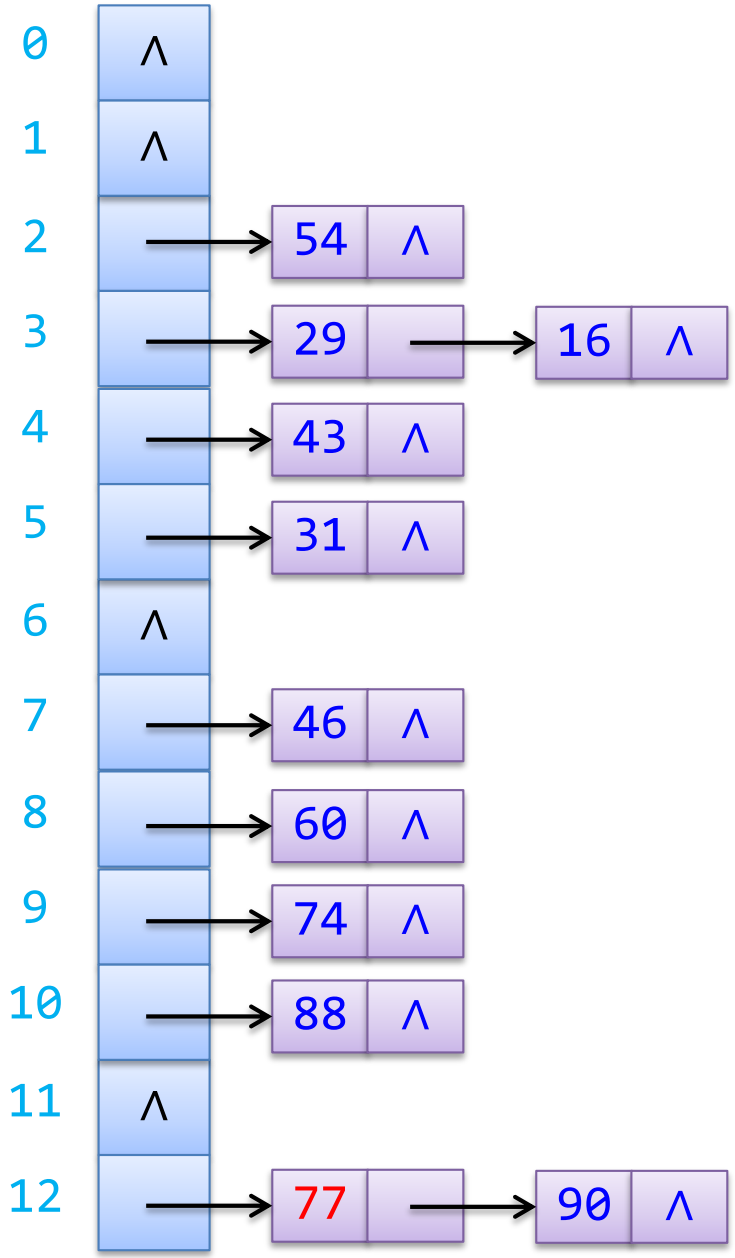
假设哈希表中单链表结点类型为HNode，哈希函数为$h(k)=k \% m$，采用拉链法解决冲突。

```
HNode<T>* search(int k)          //查找关键字k,成功时返回其地址,否则返回空
{  int d=k % m;                  //求哈希函数值
   HNode<T>* p=ha[d];            //p指向ha[d]单链表的首结点
   while (p!=NULL && p->key!=k)//查找key为k的结点p
      p=p->next;
   return p;                     //返回p（查找失败时p=NULL）
}
```

- 成功（找到关键字为k的结点）时：返回该结点引用（地址）
- 失败（没有找到关键字为k的结点）时：返回NULL

成功的查找：在ha中找到对应的关键字

示例

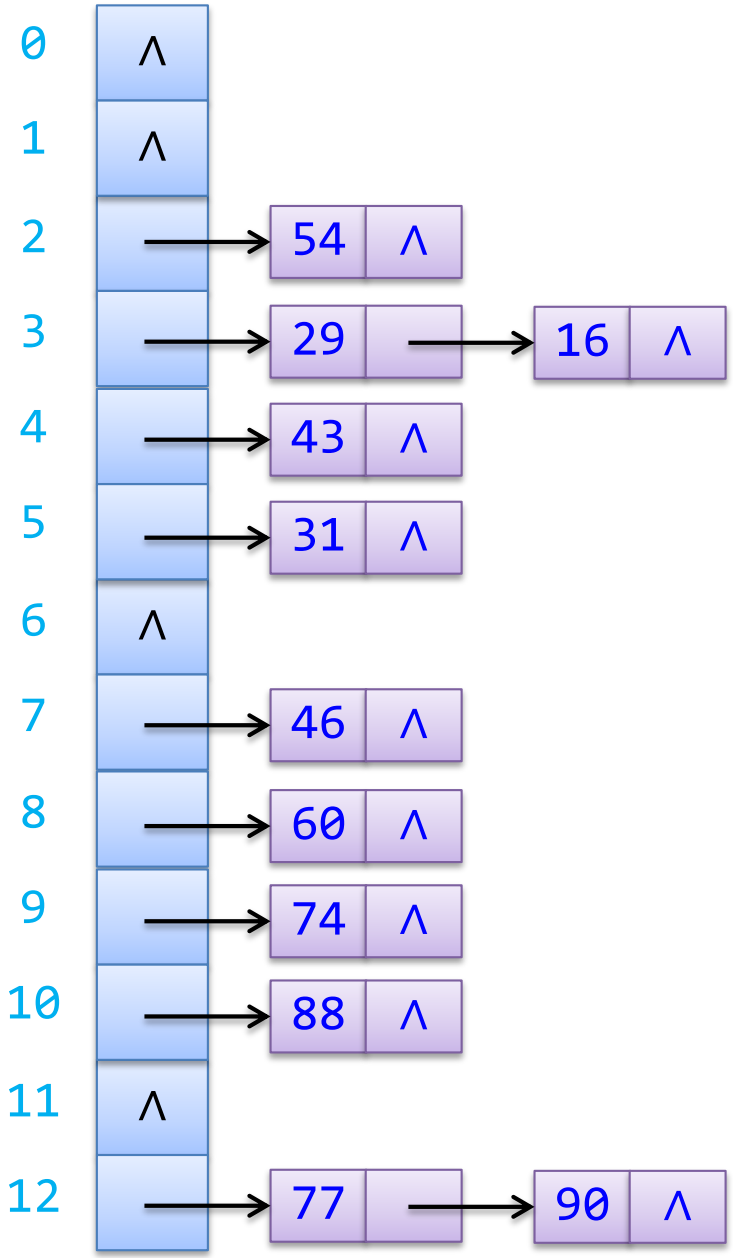


- $h(77)=12$，在ha[12]的单链表中
- 共1次比较

比较次数=对应结点在单链表中的序号（从1开始）

$$ASL_{succ}=\frac{1\times 9+2\times 2}{11}=1.18$$

不成功的查找：在ha中找不到对应的关键字*x*



- $h(x)=3$，在ha[3]的单链表中
- 共2次比较

比较的次数=对应单链表中的结点个数

$$ASL_{unsucc}=\frac{7\times1+2\times2}{13}=0.846$$

$$=\alpha$$

一般情况的ASL

- *n*个关键字的构造顺序不同得到的哈希表不同
- 平均查找长度ASL也不同
- 考虑所有顺序构造哈希表的平均情况

平均情况下的平均查找长度：

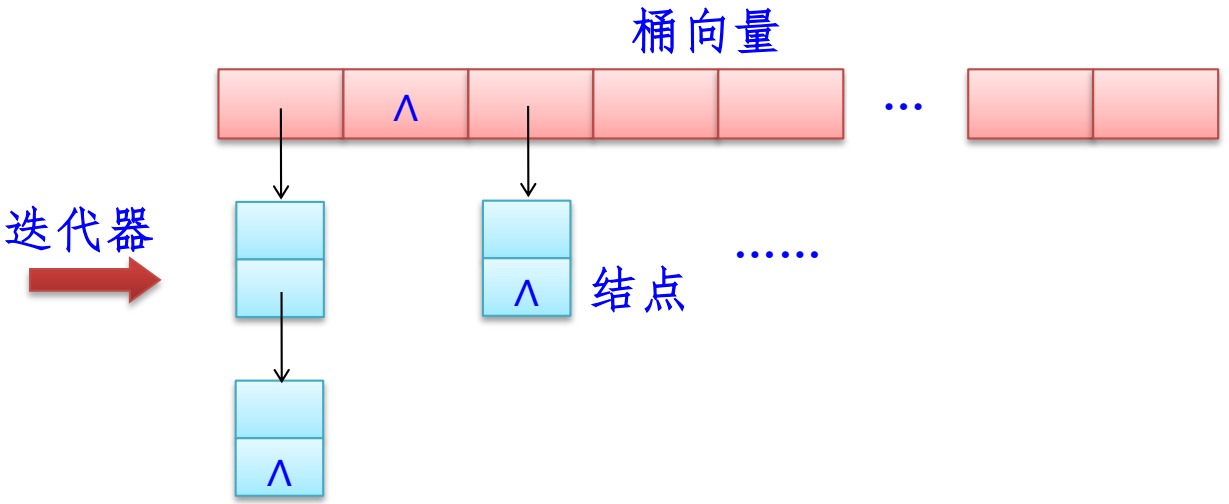
解决冲突的方法	平均查找长度ASL	
	成功的查找	不成功的查找
线性探测法	$\frac{1}{2}(1+\frac{1}{1-\alpha})$	$\frac{1}{2}(1+\frac{1}{(1-\alpha)^2})$
平方探测法	$-\frac{1}{\alpha}\log_e(1-\alpha)$	$\frac{1}{1-\alpha}$
拉链法	$1+\frac{\alpha}{2}$	$\alpha+e^{-\alpha}\approx\alpha$

9.4.5* STL中的哈希表

- 在C++11中新增加了4个关联容器，分别是unordered_map，unordered_set，unordered_multimap和unordered_multiset。
- 它们与map/multimap和set/multiset功能基本类似，主要区别这4个新增关联容器底层采用哈希表实现，查找性能更高。下面主要讨论unordered_map容器。

1. unordered_map容器的概述

- unordered_map容器的哈希结构如图所示，每个关键字为key元素通过一些哈希函数映射到一个特定位置，采用拉链法解决冲突。
- 哈希空间由桶向量构成，其中每个元素就是一个桶，指向对应的同义词单链表。每个哈希桶中可能没有结点，也可能有多个结点。



unorederd_map类模板如下：

```
template < class Key,                                    //关键字类型
           class T,                                      //值类型
           class Hash = hash<Key>,                       //哈希函数
           class Pred = equal_to<Key>,                   //相等比较函数
           class Alloc = allocator< pair<const Key,T> >    //分配器
         > class unordered_map;
```

unorederd_map的主要成员函数与map的大致相同，使用方法也与map的类似。但unorederd_map容器具有如下特点：

① 关联性：unorederd_map是一个关联容器，其中的元素根据关键字来引用，而不是根据索引来引用。

② 无序性：由于采用哈希结构，unordered_map中的元素不会根据其关键字值或映射值按任何特定顺序排序，而是根据其哈希值组织到桶中，以允许通过键值直接快速访问各个元素（按关键字查找的平均时间复杂度大致为O(1)）。

③ 唯一性：unorederd_map容器中的元素的关键字是唯一的。

2．使用unordered_map容器

1）创建unorederd_map容器

- 创建完整的unorederd_map容器比较复杂。
- 最简单的是像map一样仅给出<key，value>的类型，并且可以使用“{}”进行初始化。例如，以下语句创建一个hmap容器（其中int关键字为学号，string值为姓名）并通过初始化插入一个元素：

```
unordered_map<int,string> hmap={{1,"Mary"}};   //创建hmap并初始化
```

2）插入元素

向unorederd_map中插入元素主要有两种方法，分别是用insert函数插入pair数据和用数组方式插入数据。例如以下两个语句对应两种插入方法：

```
hmap.insert(pair<int, string>(2, "Smith"));
hmap[3] = "John";
```

3）获取一个关键字对应的值

直接使用unorederd_map[关键字]获取该关键字对应的值。例如：

```
string xm=hmap[4];
```

与map一样，执行该语句时，只有当hmap中有这个关键字（4）时才会成功返回学号4的姓名，否则会自动插入一个元素，其关键字为4，对应值为string类型的默认值空串。

4）按关键字查找元素

（1）用count函数来判定关键字是否出现，其返回值要么是0，要么是1。其缺点是无法定位关键字对应元素出现位置。例如：

```
if (hmap.count(1)!=0)    //判断hmap中是否存在关键字为1的元素
   cout << "查找成功" << endl;
else
   cout << "查找失败" << endl;
```

（2）用find函数来定位元素出现位置，它返回的一个迭代器，当查找成功时返回元素所在位置的迭代器，查找失败时返回end()函数值。例如：

```
auto it=hmap.begin();    //初始化时auto自动识别为迭代器类型
it=hmap.find(2);
if (it!=hmap.end())      //在hmap查找关键字为2的元素
   cout << "查找成功，其姓名是" << it->second << endl;
else
   cout << "查找失败" << endl;
```

5）删除元素

主要使用erase函数删除map中的元素。例如：

```
auto it=hmap.begin();
hmap.erase(it);                    //删除第一个元素
hmap.erase(2);                     //删除关键字为2的元素
```

6）修改元素值

在unorederd_map中修改元素值非常简单，与map类似。例如：

```
hmap[1]="June";                    //将学号1的姓名改为"June"
```